TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

HCM·UTE

ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG CỤM ĐỒNG HỒ KỸ THUẬT SỐ (IPC) VÀ BỘ ĐIỀU KHIỂN TRUNG TÂM (VCU) CHO MÔ HÌNH XE Ô TÔ ĐIỆN

SVTH: **LÊ THÀNH NGUYÊN**
MSSV:22119204
TĂNG THÂN NHẤT
MSSV:22119208

GVHD: **THS. NGUYỄN VĂN PHÚC**

TP. HỒ CHÍ MINH – 07/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

HCM·UTE

ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG CỤM ĐỒNG HỒ KỸ THUẬT SỐ (IPC) VÀ BỘ ĐIỀU KHIỂN TRUNG TÂM (VCU) CHO MÔ HÌNH XE Ô TÔ ĐIỆN

SVTH: **LÊ THÀNH NGUYÊN**
MSSV:22119204
TĂNG THÂN NHẤT
MSSV:22119208

GVHD: **THS. NGUYỄN VĂN PHÚC**

TP. HỒ CHÍ MINH – 07/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ-THÔNG TIN

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự Do – Hạnh phúc

Tp. HCM, ngày 20 tháng 07 năm 2026

BẢN XÁC NHẬN VIỆC CHỈNH SỬA KHÓA LUẬN TỐT NGHIỆP

1. **Tên đề tài**: Thiết kế và triển khai hệ thống cụm đồng hồ kĩ thuật số (IPC) và bộ điều khiển trung tâm (VCU) cho mô hình xe ô tô điện
2. Thông tin sinh viên

   + Họ tên sinh viên: Lê Thành Nguyên MSSV: 22119204

   + Họ tên sinh viên: Tăng Thân Nhất MSSV: 22119208
3. **Giảng viên hướng dẫn**: Ths. Nguyễn Văn Phúc
4. **Ngành**: Công Nghệ Kỹ Thuật Máy Tính
5. **Thông tin bảo vệ**

   + Hội đồng: 03

   + Phòng: A117

   + Thời gian bảo vệ: ngày 7 tháng 7 năm 2026
6. Các nội dung chỉnh sửa trong cáo cáo khóa luận tốt nghiệp theo ý kiến của giảng viên phản biện và các thành viên Hội đồng bảo vệ:

TT	Ý kiến của giảng viên phản biện và các thành viên Hội đồng bảo vệ	Kết quả chỉnh sửa, bổ sung	Ghi chú
1	Bỏ các nội dung để song ngữ	Đã bỏ các song ngữ của nội dung để song ngữ không cần thiết	
2	Lưu đồ 3.24, 3.25, 3.30	Các lưu đồ 3.24, 3.25, 3.30 đã được sửa đúng tính hợp lý về nội dung và hình thức của lưu đồ	
3	Chưa đánh giá định lượng về đáp ứng thời gian thực	Đã thêm nội dung đáp ứng định lượng về thời gian thực ở mục: 4.5 Đánh giá định lượng về đáp ứng thời gian thực	
4	Làm rõ điểm cải tiến của đề tài so với các đề tài trước đây	Đã thêm điểm cải tiến của đề tài so với các đề tài trước ở cuối mục: 5.1.1 Kết quả đạt được	
5	Trích dẫn các TLTK và thứ tự trích dẫn phù hợp	Đã có trích dẫn TLTK, đã sửa hình thức của các trích dẫn này phù hợp	

6	Tất cả hình và bảng cần được giới thiệu và phân tích trong các đoạn liên quan	Đã cật nhật đầy đủ nội dung của tất cả các hình và bảng	
7	Cải thiện chất lượng hình ảnh	Đã cải thiện chất lượng hình ảnh trong chương 4 Kết quả thực hiện	

Giảng viên hướng dẫn
(Ký và ghi rõ họ tên)

Nguyễn Văn Phúc

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

Lê Thành Nguyên

Tăng Thân Nhất

Xác nhận của Bộ môn

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG CỤM ĐỒNG HỒ KỸ THUẬT SỐ (IPC) VÀ BỘ ĐIỀU KHIỂN TRUNG TÂM (VCU) CHO MÔ HÌNH XE Ô TÔ ĐIỆN

Sinh viên: + Tăng Thân Nhất MSSV: 22119208

+ Lê Thành Nguyên MSSV: 22119204

Hướng dẫn: ThS. Nguyễn Văn Phúc

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề rõ ràng, mục tiêu cụ thể

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phương pháp hợp lý

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Kết quả phù hợp với mục tiêu

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Đề xuất mang tính cải tiến

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Bố cục rõ ràng, đúng mẫu

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Kỹ năng làm việc nhóm tốt

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Có thực hiện trích dẫn

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Tỉ lệ đạo văn đạt yêu cầu

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Tinh thần và thái độ học tập nghiêm túc, khả năng nghiên cứu tốt

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo đạt yêu cầu, nhóm sinh viên được phép bảo vệ khóa luận tốt nghiệp

Tp. HCM, ngày 26 tháng 06 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)

Nguyễn Văn Phúc

---

* *Giáo viên hướng dẫn có thể ghi tiếp vào mặt sau*

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án này, nhóm chúng em đã nhận được rất nhiều sự giúp đỡ và động viên quý báu.

Trước hết, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Thầy Nguyễn Văn Phúc, người đã tận tình hướng dẫn, định hướng và góp ý cho chúng em trong từng giai đoạn của đồ án. Những lời chỉ bảo của Thầy không chỉ giúp chúng em hoàn thành đề tài mà còn giúp chúng em trưởng thành hơn về tư duy kỹ thuật.

Chúng em cũng xin cảm ơn quý Thầy Cô trong Khoa/Bộ môn Kỹ thuật máy tính - Viễn thông, Trường đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh đã truyền đạt cho chúng em những kiến thức nền tảng vững chắc trong suốt những năm học vừa qua.

Cuối cùng, chúng em xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn ở bên, ủng hộ và tạo điều kiện để chúng em hoàn thành đồ án này.

Do kiến thức và thời gian còn hạn chế, đồ án không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý của quý Thầy Cô để đề tài được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

LỜI CAM ĐOAN

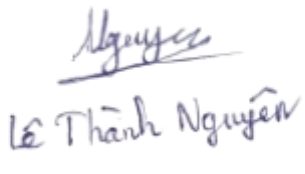
Nhóm tác giả gồm Lê Thành Nguyên và Tăng Thân Nhất xin được cam đoan rằng đồ án tốt nghiệp với đề tài "Thiết kế và thi công hệ thống cụm đồng hồ kỹ thuật số (IPC) và bộ điều khiển trung tâm (VCU) cho mô hình xe điện" do nhóm trực tiếp thực hiện dưới sự hướng dẫn tận tình của ThS. Nguyễn Văn Phúc. Toàn bộ ý tưởng, quá trình nghiên cứu, thiết kế phần cứng cũng như phát triển phần mềm đều là kết quả lao động trí tuệ của nhóm, không sao chép, không mua bán hay nhờ người khác làm thay dưới bất kỳ hình thức nào.

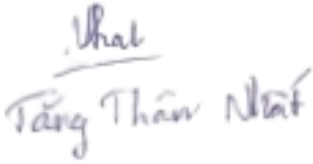
Mọi tài liệu tham khảo, số liệu, hình ảnh và thông tin được sử dụng trong đồ án đều có nguồn gốc rõ ràng, được trích dẫn cụ thể và đầy đủ theo quy định hiện hành của Nhà trường. Các kết quả thể hiện trong báo cáo là trung thực, phản ánh đúng tiến trình nghiên cứu và tương ứng với sản phẩm thực tế mà nhóm đã hoàn thiện.

Nếu có bất kỳ sự vi phạm nào liên quan đến tính trung thực của đồ án, nhóm sinh viên xin hoàn toàn chịu trách nhiệm trước Khoa Điện – Điện tử và Ban Giám hiệu Nhà trường theo đúng quy chế đào tạo hiện hành.

Sinh viên thực hiện đồ án tốt nghiệp

Sinh viên 1	Sinh viên 2
Lê Thành Nguyên	Tăng Thân Nhất





TÓM TẮT

Đồ án thực hiện thiết kế và thi công hệ thống cụm đồng hồ kỹ thuật số (IPC) và bộ điều khiển trung tâm (VCU) cho mô hình xe điện, giao tiếp qua mạng CAN Bus 500 kbit/s. Hệ thống tổ chức theo kiến trúc phân tầng phỏng theo xe điện thực tế: nút điều khiển ECU xây dựng trên vi điều khiển STM32F103C8T6 chạy hệ điều hành thời gian thực FreeRTOS, đảm nhiệm đọc tín hiệu người lái, điều khiển động cơ và đo lường thông số; bộ điều khiển trung tâm VCU xây dựng trên Raspberry Pi 4 (Linux), điều hành xe bằng máy trạng thái và xử lý logic an toàn; giao diện cụm đồng hồ thiết kế bằng Qt/QML hiển thị thông số vận hành theo thời gian thực. Toàn hệ thống quán triệt nguyên tắc mọi quyết định điều khiển tập trung tại VCU, còn STM32 chỉ thực thi.

Đề tài được triển khai theo hướng bottom-up và kiểm thử end-to-end theo từng kịch bản vận hành. Kết quả, hệ thống hoạt động ổn định với máy trạng thái sáu trạng thái, điều khiển hai chiều, an toàn nhiều lớp và chẩn đoán lỗi DTC; giao diện cập nhật mượt theo dữ liệu CAN; mô phỏng đầy đủ các tình huống khởi động, lái xe, đổi chế độ, sạc pin và xử lý sự cố, đạt được các mục tiêu đề ra.

MỤC LỤC

DANH MỤC HÌNH.....XIII

DANH MỤC BẢNG.....XVI

DANH MỤC CÁC TỪ VIẾT TẮT.....XVII

CHƯƠNG 1 GIỚI THIỆU ĐỀ TÀI.....1

1.1 ĐẶT VẤN ĐỀ..... 1

1.2 TỔNG QUAN CÁC CÔNG TRÌNH LIÊN QUAN..... 1

1.2.1 Nghiên cứu và ứng dụng trên thế giới.....1

1.2.2 Tình hình nghiên cứu trong nước.....2

1.3 MỤC TIÊU..... 3

1.4 PHƯƠNG PHÁP THỰC HIỆN..... 3

1.5 GIỚI HẠN ĐỀ TÀI..... 4

1.6 BỐ CỤC BÁO CÁO..... 4

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT.....6

2.1 CHUẨN TRUYỀN THÔNG CAN BUS..... 6

2.1.1 Lịch sử hình thành và vai trò trong xe điện..... 6

2.1.2 Đặc điểm kỹ thuật của CAN Bus..... 6

2.1.3 Cấu trúc khung dữ liệu CAN.....8

2.1.4 Cơ chế phân xử bus..... 9

2.1.5 Quá trình truyền và nhận dữ liệu trên mạng CAN.....10

2.2 CHUẨN GIAO THỨC I2C.....11

2.2.1 Tổng quan..... 11

2.2.2 Cấu trúc bus và nguyên lý hoạt động..... 11

2.2.3 Định địa chỉ và khung truyền dữ liệu.....12

2.2.4 Các chế độ tốc độ..... 13

2.3 CHUẨN GIAO THỨC SPI.....13

2.3.1 Tổng quan..... 13

2.3.2 Cấu trúc bus và nguyên lý hoạt động.....14

2.3.3 Các chế độ hoạt động..... 15

2.4 CHUẨN GIAO THỨC UART.....16
2.4.1 Tổng quan.....16
2.4.2 Cấu trúc khung truyền dữ liệu.....16
2.4.3 Tốc độ baud và cơ chế đồng bộ.....17
2.5 GIỚI THIỆU VỀ STM32CUBEIDE.....18
2.5.1 Tổng quan.....18
2.5.2 Các tính năng chính.....18
2.6 GIỚI THIỆU HỆ ĐIỀU HÀNH THỜI GIAN THỰC FREERTOS.....19
2.6.1 Tổng quan về hệ điều hành thời gian thực.....19
2.6.2 Giới thiệu FreeRTOS.....20
2.6.3 Các thành phần và cơ chế chính.....20
2.7 GIỚI THIỆU VỀ QT/QML.....22
2.7.1 Tổng quan về Qt.....22
2.7.2 Ngôn ngữ QML và Qt Quick.....22
CHƯƠNG 3 THIẾT KẾ VÀ THI CÔNG HỆ THỐNG.....24
3.1 KIẾN TRÚC TỔNG THỂ HỆ THỐNG.....24
3.1.1 Sơ đồ khối tổng thể hệ thống.....24
3.1.2 Phân chia chức năng theo ba tầng.....25
3.1.3 Các nguyên tắc thiết kế cốt lõi.....26
3.1.4 Luồng dữ liệu tổng thể.....27
3.2 THIẾT KẾ PHẦN CỨNG.....28
3.2.1 Sơ đồ khối phần cứng tổng thể.....28
3.2.2 Khối ECU Node (STM32F103C8T6).....28
3.2.3 Khối nguồn pin.....30
3.2.4 Khối truyền động.....33
3.2.5 Khối input.....36
3.2.6 Khối truyền thông CAN.....38
3.2.7 Khối VCU/IPC.....40
3.2.8 Pin mapping tổng hợp và cấu hình ngoại vi.....43
3.3 THIẾT KẾ GIAO THỨC TRUYỀN THÔNG CAN.....46

3.3.1 Tham số bus và nguyên tắc thiết kế..... 46
3.3.2 Bảng thông điệp CAN..... 47
3.3.3 Đặc tả bố cục byte của từng khung..... 47
3.3.4 Quy ước mã hóa trường liệt kê và cờ lỗi..... 49
3.3.5 Lập lịch truyền và lọc nhận..... 50
3.4 THIẾT KẾ VÀ THI CÔNG PHẦN MỀM STM32 (FREERTOS)..... 50
3.4.1 Kiến trúc phần mềm phân chia module..... 50
3.4.2 Thiết kế tác vụ..... 51
3.4.3 Cơ chế đồng bộ dữ liệu thời gian thực..... 52
3.4.4 Module Input Manager..... 53
3.4.5 Module Motor Control..... 54
3.4.6 Module Battery Manager..... 56
3.4.7 Module CAN Handler..... 58
3.5 THIẾT KẾ VÀ THI CÔNG PHẦN MỀM VCU..... 60
3.5.1 Kiến trúc tiến trình và mô hình đa luồng..... 60
3.5.2 VehicleData - kho dữ liệu trung tâm thread-safe..... 61
3.5.3 Truyền thông CAN trên Linux (SocketCAN)..... 61
3.5.4 Máy trạng thái sáu trạng thái..... 63
3.5.5 Kiến trúc xử lý lỗi hai tầng và logic an toàn..... 66
3.5.6 Hệ thống chẩn đoán DtcManager (Tầng B)..... 68
3.6 THIẾT KẾ VÀ THI CÔNG GIAO DIỆN IPC (QT/QML)..... 70
3.6.1 Lớp cầu nối VcuBridge (C++ ↔ QML)..... 70
3.6.2 Điều khiển màn hình theo trạng thái..... 71
3.6.3 Thiết kế các màn hình..... 72
3.6.4 Hoạt ảnh chuyển màn hình..... 72
3.6.5 Hệ thống cảnh báo và chẩn đoán..... 73
3.6.6 Bố cục cụm đồng hồ..... 73
CHƯƠNG 4 KẾT QUẢ THỰC HIỆN..... 75
4.1 KẾT QUẢ THI CÔNG PHẦN CỨNG..... 75
4.2 KẾT QUẢ TRUYỀN THÔNG CAN..... 78

4.2.1 Khởi động và log UART..... 78
4.2.2 Kết quả VCU nhận gói tin CAN từ ECU..... 78
4.3 KẾT QUẢ HOẠT ĐỘNG VCU VÀ GIAO DIỆN IPC..... 80
4.3.1 Khởi động và tự kiểm tra hệ thống..... 80
4.3.2 Trạng thái STANDBY và READY..... 82
4.3.3 Trạng thái DRIVING và các chế độ lái..... 83
4.3.4 Trạng thái sạc pin..... 85
4.3.5 Kết quả phát hiện lỗi và chẩn đoán..... 87
4.4 ĐÁNH GIÁ MỨC ĐỘ TIN CẬY..... 94
4.4.1 Độ tin cậy truyền thông và đồng bộ dữ liệu..... 94
4.4.2 Độ tin cậy của logic điều khiển và máy trạng thái..... 95
4.4.3 Độ tin cậy của các cơ chế an toàn và chẩn đoán..... 95
4.4.4 Đối chiếu với mục tiêu đề ra..... 96
4.5 ĐÁNH GIÁ ĐỊNH LƯỢNG VỀ ĐÁP ỨNG THỜI GIAN THỰC..... 96
CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN..... 98
5.1 KẾT LUẬN..... 98
5.1.1 Kết quả đạt được..... 98
5.1.2 Hạn chế..... 99
5.2 HƯỚNG PHÁT TRIỂN..... 100
TÀI LIỆU THAM KHẢO..... 101

DANH MỤC HÌNH

Hình 2.1 Biểu đồ điện áp CANH và CANL của CAN tốc độ cao (High-Speed CAN theo ISO 11898-2)..... 7
Hình 2.2 Cấu trúc của một khung dữ liệu CAN (Data Frame)..... 8
Hình 2.3 Minh hoạ quá trình arbitration giữa ba node: Node 1 và Node 2 và Node 3 phát đồng thời; Node 3 thắng..... 9
Hình 2.4 Sơ đồ kết nối bus I2C với một master và nhiều slave..... 11
Hình 2.5 Điều kiện START và STOP trên bus I2C..... 12
Hình 2.6 Cấu trúc một khung truyền I2C đầy đủ..... 13
Hình 2.7 Sơ đồ kết nối SPI giữa master và nhiều slave..... 14
Hình 2.8 Bốn chế độ SPI theo CPOL và CPHA..... 15
Hình 2.9 Sơ đồ kết nối UART giữa hai thiết bị..... 16
Hình 2.10 Cấu trúc khung truyền dữ liệu UART..... 17
Hình 2.11 Giao diện cấu hình chân và ngoại vi trong STM32CubeIDE... 19
Hình 2.12 Sơ đồ chuyển trạng thái của một tác vụ trong FreeRTOS..... 21
Hình 2.13 Biểu tượng của nền tảng Qt..... 22
Hình 3.1 Sơ đồ khối tổng thể hệ thống VCU-IPC..... 25
Hình 3.2 Luồng dữ liệu tổng thể của hệ thống..... 27
Hình 3.3 Sơ đồ khối phần cứng tổng thể của hệ thống..... 28
Hình 3.4 Sơ đồ nối dây khối ECU Node..... 29
Hình 3.5 STM32F103C8T6 (board Blue Pill)..... 29
Hình 3.6 Sơ đồ nối dây khối nguồn pin..... 31
Hình 3.7 Mạch bảo vệ pin BMS 2S 20A..... 31
Hình 3.8 Mạch sạc Pin 2S 2A Cổng TypeC..... 32
Hình 3.9 Mạch Opto Cách Ly 2 Kênh PC817..... 33
Hình 3.10 Sơ đồ nối dây khối truyền động..... 34
Hình 3.11 Driver motor TB6612FNG..... 35
Hình 3.12 GA12-N20 động cơ giảm tốc có encoder 6VDC, 100 rpm..... 35
Hình 3.13 Thông số kỹ thuật của GA12-N20 động cơ giảm tốc 36
Hình 3.14 Sơ đồ nối dây khối input..... 37

Hình 3.15 Sơ đồ nối dây khối truyền thông Can..... 38
Hình 3.16 Mạch truyền thông Can Bus TJA1050..... 39
Hình 3.17 Mạch chuyển giao tiếp Can Bus MCP2515..... 39
Hình 3.18 Sơ đồ nối dây khối VCU/IPC..... 41
Hình 3.19 Raspberry Pi 4 Model B 4GB..... 41
Hình 3.20 Màn hình Waveshare 7 inch HDMI 42
Hình 3.21 Mạch thời gian thực RTC DS3231..... 42
Hình 3.22 Sơ đồ nguyên lý tổng thể cho phần cứng..... 45
Hình 3.23 Sơ đồ kiến trúc phân lớp phần mềm STM32..... 51
Hình 3.24 Lưu đồ giải thuật cho tác vụ Input Manager..... 54
Hình 3.25 Lưu đồ giải thuật cho tác vụ Motor Control..... 55
Hình 3.26 Lưu đồ giải thuật cho tác vụ Battery Manager..... 57
Hình 3.27 Lưu đồ giải thuật cho Task_CANTransmit..... 59
Hình 3.28 Lưu đồ giải thuật cho Task_CANReceive..... 59
Hình 3.29 Kiến trúc tiến trình và mô hình đa luồng của VCU..... 60
Hình 3.30 Lưu đồ giải thuật cho luồng CanReceive..... 62
Hình 3.31 Lưu đồ giải thuật cho luồng CanSender..... 63
Hình 3.32 Lưu đồ vòng lặp StateMachine..... 65
Hình 3.33 Lưu đồ giải thật của Tầng A trong kiến trúc xử lý lỗi..... 67
Hình 3.34 Lưu đồ giải thật của Tầng B trong kiến trúc xử lý lỗi..... 69
Hình 3.35 Cơ chế chuyển C++ qua QML thông qua lớp cầu nối VcuBridge..... 71
Hình 4.1 Mô hình hệ thống VCU–IPC cho mô hình xe điện..... 75
Hình 4.2 Màn hình giao diện hệ thống..... 77
Hình 4.3 Log khởi động của ECU node trên cổng UART..... 78
Hình 4.4 Lưu lượng khung CAN bắt được trên Raspberry Pi..... 79
Hình 4.5 Giao diện trạng thái OFF..... 80
Hình 4.6 Giao diện hiệu ứng khởi động hệ thống..... 81
Hình 4.7 Giao diện chu trình tự kiểm tra..... 81
Hình 4.8 Giao diện trạng thái STANDBY..... 82
Hình 4.9 Giao diện trạng thái READY..... 83

Hình 4.10 Giao diện trạng thái DRIVING – chế độ ECO.....83
Hình 4.11 Giao diện trạng thái DRIVING – chế độ NORMAL.....84
Hình 4.12 Giao diện trạng thái DRIVING – chế độ SPORT.....85
Hình 4.13 Giao diện trạng thái CHARGING.....86
Hình 4.14 Kiểm chứng quá trình nạp pin.....86
Hình 4.15 Hiệu ứng kết thúc sạc.....87
Hình 4.16 Màn hình FAULT khi khi động cơ bị kẹt.....88
Hình 4.17 Màn hình FAULT mất truyền thông CAN.....88
Hình 4.18 Hiển thị khi cảm biến điện áp bị ngắt.....89
Hình 4.19 Giao diện cảnh báo pin yếu.....90
Hình 4.20 Menu service mô phỏng lỗi (Fault Injection).....90
Hình 4.21 Màn hình Lỗi CRITICAL (P0AA6) 91
Hình 4.22 Màn hình bật đèn MIL khi lỗi MEDIUM (P0560).....92
Hình 4.23 Màn hình chế độ limp-home khi lỗi HIGH (P0A3F).....92
Hình 4.24 Màn hình bảng chi tiết các mã lỗi DTC.....93
Hình 4.25 Nội dung file lưu trữ DTC.....93

DANH MỤC BẢNG

Bảng 2.1 Thông số kỹ thuật chính của CAN 2.0 theo ISO 11898 [9][10]...8
Bảng 3.1 Phân bổ chức năng giữa các tầng của hệ thống..... 25
Bảng 3.2 Bảng chân lý điều khiển chiều quay motor qua TB6612FNG... 34
Bảng 3.3 Thành phần khối input.....37
Bảng 3.4 Pin mapping tổng hợp khối STM32F103C8T6..... 43
Bảng 3.5 Cấu hình ngoại vi STM32 trên STM32CubeMX..... 44
Bảng 3.6 Bảng thông điệp CAN của hệ thống..... 47
Bảng 3.7 Bố cục byte khung 0x100 — Speed & Accel..... 47
Bảng 3.8 Bố cục byte khung 0x101 — Battery.....48
Bảng 3.9 Bố cục byte khung 0x102 — Hardware Inputs.....48
Bảng 3.10 Bố cục byte khung 0x103 — Odometer..... 48
Bảng 3.11 Bố cục byte khung 0x200 — VCU Command..... 48
Bảng 3.12 Mã hóa các trường liệt kê.....49
Bảng 3.13 Mã hóa bitmask trường fault_flags..... 49
Bảng 3.14 Thiết kế các tác vụ firmware STM32.....52
Bảng 3.15 Bảng tra SoC theo điện áp pack 2S (nội suy tuyến tính giữa các điểm).....56
Bảng 3.16 Các điều kiện chuyển trạng thái của máy trạng thái VCU..... 64
Bảng 3.17 Danh mục lỗi tầng A (Bảo vệ) do máy trạng thái xử lý..... 66
Bảng 3.18 Danh mục mã chẩn đoán DTC của hệ thống..... 68
Bảng 3.19 Ánh xạ trạng thái xe sang màn hình hiển thị và hiệu ứng chuyển cảnh.....72
Bảng 3.20 Các đèn báo trên thanh trên cùng của cụm đồng hồ..... 73
Bảng 4.1 Đối chiếu giải mã các khung CAN..... 79
Bảng 4.2 Đối chiếu kết quả với mục tiêu đề ra..... 96
Bảng 4.3 Chu kỳ hoạt động của các tác vụ/luồng chính trên cả hai nền tảng..... 97

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Từ đầy đủ
ACK	Acknowledge
ADC	Analog-to-Digital Converter
APB1	Advanced Peripheral Bus 1
ARM	Advanced RISC Machines
BMS	Battery Management System
bxCAN	Basic Extended CAN
CAN	Controller Area Network
CAN FD	CAN with Flexible Data-rate
CANH / CANL	CAN High / CAN Low
CMSIS	Cortex Microcontroller Software Interface Standard
CPOL / CPHA	Clock Polarity / Clock Phase
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DAC	Digital-to-Analog Converter
DC	Direct Current
DLC	Data Length Code
DMA	Direct Memory Access
DTC	Diagnostic Trouble Code

ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable ROM
EOF	End of Frame
EV	Electric Vehicle
FIFO	First In First Out
FOC	Field-Oriented Control
FreeRTOS	Free Real-Time Operating System
FSM	Finite State Machine
GCC	GNU Compiler Collection
GDB	GNU Debugger
GND	Ground
GPIO	General-Purpose Input/Output
GPS	Global Positioning System
HAL	Hardware Abstraction Layer
HDMI	High-Definition Multimedia Interface
HMI	Human-Machine Interface
HV	High Voltage
I2C	Inter-Integrated Circuit
ID	Identifier
IDE	Integrated Development Environment

IFS	Inter-Frame Space
IPC	Instrument Panel Cluster
ISO	International Organization for Standardization
ISR	Interrupt Service Routine
LCD	Liquid Crystal Display
LL	Low-Layer
MCU	Microcontroller Unit
MIL	Malfunction Indicator Lamp
MISRA	Motor Industry Software Reliability Association
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
MOSI / MISO	Master Out Slave In / Master In Slave Out
NACK	Negative Acknowledge
NXP	NXP Semiconductors
OBD	On-Board Diagnostics
OCV	Open-Circuit Voltage
OEM	Original Equipment Manufacturer
PID	Proportional-Integral-Derivative
PWM	Pulse Width Modulation
QML	Qt Modeling Language
RAM	Random Access Memory

RTC	Real-Time Clock
RTOS	Real-Time Operating System
RTR	Remote Transmission Request
SBC	Single-Board Computer
SCK	Serial Clock
SDA / SCL	Serial Data / Serial Clock
SoC	State of Charge
SOF	Start of Frame
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
SS	Slave Select
TIM	Timer
TTL	Transistor-Transistor Logic
UART	Universal Asynchronous Receiver/Transmitter
USART	Universal Synchronous/Asynchronous Receiver/Transmitter
USB	Universal Serial Bus
VCU	Vehicle Control Unit
VDC	Volts Direct Current

CHƯƠNG 1

GIỚI THIỆU ĐỀ TÀI

1.1 ĐẶT VẤN ĐỀ

Theo báo cáo Global EV Outlook 2024 của Cơ quan Năng lượng Quốc tế (IEA), doanh số xe điện toàn cầu đạt gần 14 triệu chiếc trong năm 2023, chiếm 18% tổng lượng ô tô bán ra – tăng gấp đôi so với chỉ hai năm trước đó [1]. Sự tăng trưởng mạnh mẽ này kéo theo nhu cầu cấp bách về nguồn nhân lực kỹ thuật có khả năng hiểu và triển khai các hệ thống điện tử nhúng đặc thù của xe điện, đặc biệt là hai thành phần cốt lõi: Vehicle Control Unit (VCU) và Instrument Panel Cluster (IPC).

VCU đóng vai trò trung tâm điều phối toàn bộ hoạt động của xe điện – từ điều khiển motor, quản lý năng lượng pin, xử lý tín hiệu người lái đến giám sát trạng thái hệ thống và phát hiện lỗi [2]. IPC là giao diện trực quan duy nhất giữa xe và người lái, hiển thị thời gian thực các thông số như tốc độ, mức pin và cảnh báo an toàn thông qua chuẩn truyền thông CAN Bus [3]. Hai thành phần này hoạt động liên kết chặt chẽ và là nền tảng của mọi kiến trúc xe điện hiện đại.

Tuy nhiên, đối với sinh viên tại Việt Nam, các hệ thống VCU và IPC thương mại hầu như là "hộp đen" – không có tài liệu nội bộ, không thể can thiệp và học hỏi từ bên trong. Điều này tạo ra khoảng cách lớn giữa lý thuyết được học và thực tế hoạt động của một hệ thống xe điện đầy đủ. Đồ án này được thực hiện với mục tiêu lấp đầy khoảng cách đó: tự tay thiết kế, triển khai và mô phỏng một hệ thống IPC – VCU hoàn chỉnh trên phần cứng thực, từ đó hiểu từ gốc rễ cách các thành phần này hoạt động và phối hợp với nhau trong một xe điện.

1.2 TỔNG QUAN CÁC CÔNG TRÌNH LIÊN QUAN

1.2.1 Nghiên cứu và ứng dụng trên thế giới

Hướng tiếp cận xây dựng IPC và VCU trên nền tảng nhúng mở đã được cộng đồng kỹ thuật quốc tế triển khai qua nhiều dự án có tài liệu công khai.

Dự án DES Instrument Cluster của chương trình SEA:ME (Software Engineering in Automotive and Mobility) là một ví dụ điển hình: nhóm tác giả xây dựng ứng dụng IPC bằng Qt chạy trên Raspberry Pi, nhận dữ liệu tốc độ từ cảm biến thông qua CAN Bus sử dụng MCP2515 và TJA1050 – bộ đôi phần cứng giống hệt với đề tài này [4]. Điểm khác biệt là dự án SEA:ME chỉ hiển thị tốc độ đơn thuần, chưa tích hợp VCU với state machine, quản lý pin hay điều khiển motor.

Một công trình đáng chú ý khác là nghiên cứu "Implementation of Automotive Embedded System" (2022) sử dụng STM32F103 với FreeRTOS để xây dựng một VCU tích hợp nhiều hệ thống con gồm Motor Control, Steer-by-Wire và Battery Management System [5]. Nghiên cứu này chứng minh tính khả thi của việc chạy nhiều subsystem song song trên một vi điều khiển tầm trung với RTOS, đồng thời chỉ ra rằng kiến trúc tập trung giúp giảm chi phí phần cứng và cải thiện khả năng phối hợp giữa các hệ thống so với kiến trúc đa ECU độc lập.

Về phía giao diện đồ họa, Bamboo Apps (2024) đã công bố một proof-of-concept kết hợp FreeRTOS trên Arduino với Qt trên Raspberry Pi qua CAN Bus thực để mô phỏng kiểm thử ứng dụng ô tô [6]. Nghiên cứu này xác nhận rằng kiến trúc MCU (FreeRTOS) – CAN – SBC (Qt/Linux) là hoàn toàn khả thi và đủ hiệu năng cho ứng dụng giáo dục và nghiên cứu. Qt cũng được xác nhận là framework IPC chủ đạo trong ngành ô tô, được sử dụng bởi BMW Group, Honda, Volvo và nhiều OEM lớn khác thông qua Qt Automotive Suite [7].

1.2.2 Tình hình nghiên cứu trong nước

Tại Việt Nam, các đề tài liên quan đến CAN Bus và hệ thống ô tô bắt đầu xuất hiện nhiều hơn ở bậc đại học trong vài năm gần đây, chủ yếu tập trung vào thu thập và giám sát dữ liệu.

Đề tài "Ứng dụng chuẩn truyền thông CAN thiết kế hệ thống giám sát trên ô tô" (Đàm Thuận An, Lâm Quang Vinh – Trường Đại học Công nghệ Kỹ thuật TP.HCM, 2025) sử dụng kiến trúc Raspberry Pi 4 + MCP2515 + Arduino để thu thập dữ liệu mô phỏng từ mạng CAN rồi hiển thị lên LCD 7 inch và đẩy lên

Firebase. Đây là công trình gần nhất với đề tài hiện tại về mặt phần cứng. Tuy nhiên, phạm vi của công trình đó giới hạn ở việc giám sát thụ động các thông số mô phỏng, chưa có VCU thực sự điều khiển xe, chưa có state machine vận hành và chưa triển khai motor control thực.

1.3 MỤC TIÊU

Mục tiêu chính của đề tài là thiết kế và triển khai một hệ thống cụm đồng hồ hiển thị (IPC) và bộ điều khiển trung tâm (VCU) cho mô hình xe điện, giao tiếp với nhau qua chuẩn truyền thông CAN Bus. Để đạt được mục tiêu này, đề tài cần thực hiện các công việc sau:

Xây dựng nút điều khiển ECU có chức năng đọc tín hiệu người lái, điều khiển động cơ, đo các thông số vận hành và truyền dữ liệu lên mạng CAN.

Xây dựng bộ điều khiển trung tâm VCU nhận dữ liệu từ mạng CAN, điều hành hoạt động của xe theo máy trạng thái và xử lý logic điều khiển, an toàn.

Thiết kế giao diện cụm đồng hồ IPC hiển thị trực quan các thông số vận hành của xe theo thời gian thực.

Tích hợp các thành phần và kiểm thử hệ thống qua các kịch bản vận hành thực tế.

1.4 PHƯƠNG PHÁP THỰC HIỆN

Đề tài được tiếp cận theo ba giai đoạn kế tiếp nhau: nghiên cứu lý thuyết, triển khai phần cứng và phần mềm, sau đó tích hợp và kiểm thử toàn hệ thống.

Giai đoạn nghiên cứu lý thuyết tập trung vào việc nắm vững các nguyên lý hoạt động của CAN Bus (ISO 11898), kiến trúc bxCAN peripheral trên STM32, cơ chế lập lịch task của FreeRTOS và mô hình lập trình Qt/QML. Tài liệu tham khảo chính là datasheet gốc từ nhà sản xuất, tài liệu chuẩn ISO và các dự án mã nguồn mở tương tự đã được kiểm chứng [4][6].

Giai đoạn triển khai được thực hiện theo hướng bottom-up: xây dựng và kiểm thử từng khối chức năng nhỏ độc lập trước khi ghép lại. Cụ thể, firmware STM32 được phát triển và kiểm thử theo thứ tự: ADC → GPIO → PWM/Encoder → CAN Tx/Rx → FreeRTOS task integration. Phía Raspberry Pi

cũng tương tự: SocketCAN → CAN frame parsing → state machine → Qt/QML data binding. Mỗi bước được xác nhận hoạt động đúng qua UART debug log và công cụ candump trên Linux trước khi chuyển sang bước tiếp theo.

Giai đoạn tích hợp và kiểm thử chạy hệ thống end-to-end theo từng kịch bản vận hành được xác định trước (khởi động, lái xe, đổi chế độ, sạc pin, fault injection). Kết quả kiểm thử được ghi nhận định tính (hành vi đúng/sai) và định lượng (thời gian đáp ứng CAN, độ mượt hiển thị IPC) để làm cơ sở đánh giá ở Chương 4.

1.5 GIỚI HẠN ĐỀ TÀI

Hệ thống sử dụng động cơ DC GA12-N20 để mô phỏng hành vi truyền động thay cho động cơ công suất lớn của xe điện thực tế.

Đề tài không áp dụng các thuật toán điều khiển và ước lượng nâng cao (điều khiển PID đầy đủ, điều khiển theo hướng từ thông FOC, ước lượng dung lượng pin bằng phương pháp đếm điện lượng hoặc bộ lọc Kalman); dung lượng pin (SoC) được ước lượng theo phương pháp tra bảng điện áp.

Hệ thống chưa hướng tới đáp ứng tiêu chuẩn an toàn chức năng ISO 26262 và chưa tích hợp lên xe điện thương mại thực tế.

1.6 BỐ CỤC BÁO CÁO

Nội dung đề tài được trình bày trong 05 chương, cụ thể như sau:

Chương 1 Giới thiệu đề tài: Trình bày lý do hình thành đề tài, tình hình nghiên cứu trong và ngoài nước, mục tiêu, phương pháp nghiên cứu và giới hạn của đề tài.

Chương 2 Cơ sở lý thuyết: Giới thiệu các kiến thức nền tảng liên quan đến đề tài, bao gồm các chuẩn giao thức truyền thông (CAN, I2C, SPI, UART) và các nền tảng, công cụ phần mềm được sử dụng trong quá trình thiết kế và triển khai hệ thống.

Chương 3 Thiết kế và thi công hệ thống: Trình bày chi tiết cấu trúc hệ thống thông qua sơ đồ khối, thiết kế và thi công phần cứng và phần mềm, sơ đồ nối chân cùng nguyên lý hoạt động của các khối chức năng.

Chương 4 Kết quả thực hiện: Trình bày các kết quả đạt được sau khi triển khai và thi công mô hình hệ thống, từ đó phân tích, nhận xét và đánh giá mức độ đáp ứng của hệ thống so với các yêu cầu đã đặt ra.

Chương 5 Kết luận và hướng phát triển: Tổng hợp những kết quả đạt được và những hạn chế còn tồn tại của đề tài, đồng thời đề xuất các hướng phát triển trong tương lai.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 CHUẨN TRUYỀN THÔNG CAN BUS

2.1.1 Lịch sử hình thành và vai trò trong xe điện

Controller Area Network (CAN) là giao thức truyền thông nối tiếp do Robert Bosch GmbH khởi phát năm 1983 nhằm giải quyết bài toán ngày càng nhiều dây nối trong xe hơi hiện đại, và chính thức công bố tại hội nghị SAE Congress tháng 2 năm 1986 tại Detroit [8]. Đến năm 1991 Bosch phát hành CAN 2.0 và đến năm 1993 giao thức được ISO chuẩn hoá thành ISO 11898, trở thành tiêu chuẩn de facto trong ngành ô tô toàn cầu [8][9]. Mercedes-Benz là hãng đầu tiên đưa CAN vào xe sản xuất đại trà với dòng W140 S-Class năm 1991 [11].

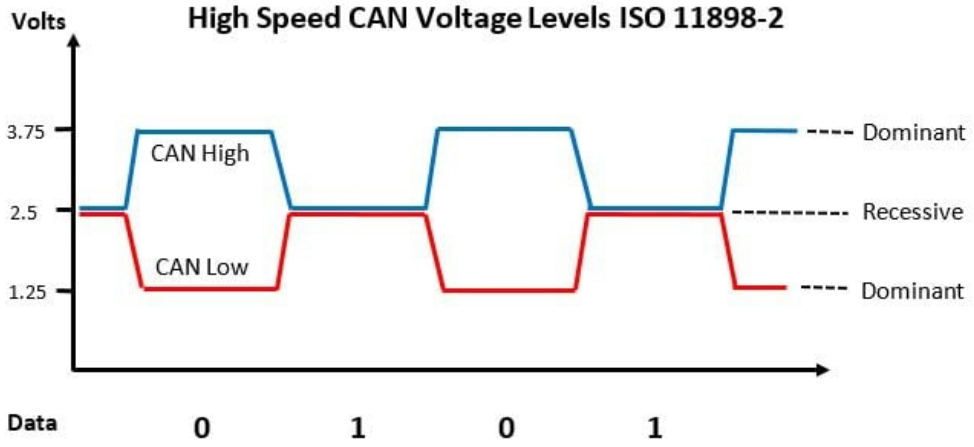
Trong xe điện, CAN Bus đóng vai trò mạch thần kinh trung ương kết nối VCU với BMS, Motor Control Unit, On-Board Charger và IPC [2]. Khả năng ưu tiên thông điệp theo ID đảm bảo các lệnh điều khiển quan trọng như dừng khẩn cấp hoặc cắt công suất luôn được xử lý trước, bất kể lưu lượng bus hiện tại – đây là tính chất thiết yếu mà các giao thức thông thường không đảm bảo được [10].

2.1.2 Đặc điểm kỹ thuật của CAN Bus

CAN Bus sử dụng kiến trúc multi-master: bất kỳ node nào cũng có thể phát khi bus rảnh, và khi xung đột xảy ra thì cơ chế phân xử không phá huỷ tự động nhường quyền cho node có ID nhỏ hơn (ưu tiên cao hơn) mà không cần truyền lại. Tín hiệu truyền theo dạng vi sai trên cặp dây CANH/CANL, giúp loại trừ nhiễu điện từ chung mode rất hiệu quả trong môi trường xe điện có nhiều nguồn nhiễu từ inverter và motor.

Về mặt vật lý, CAN sử dụng cặp dây xoắn với tín hiệu vi sai: CANH (CAN High) và CANL (CAN Low). Ở trạng thái recessive (logic 1), cả hai dây ở mức 2.5V nên hiệu điện thế vi sai xấp xỉ 0V. Ở trạng thái dominant (logic 0), CANH được kéo lên 3.5V và CANL bị kéo xuống 1.5V, tạo hiệu điện thế vi sai khoảng

2V. Nguyên tắc quan trọng: trạng thái dominant luôn "thắng" trạng thái recessive trên bus – đây là nền tảng của cơ chế arbitration.



Hình 2.1 Biểu đồ điện áp CANH và CANL của CAN tốc độ cao (High-Speed CAN theo ISO 11898-2)

Mạng CAN có thể hoạt động ở hai chế độ vật lý chính: CAN tốc độ cao (High-Speed CAN, ISO 11898-2) với tốc độ lên đến 1 Mbit/s, bus hai dây vi sai với trở kháng đặc tính 60Ω – đây là chế độ sử dụng trong đề tài; và CAN tốc độ thấp có khả năng chịu lỗi (Low-Speed/Fault-Tolerant CAN, ISO 11898-3) với tốc độ tối đa 125 kbit/s, có thể tiếp tục hoạt động khi một trong hai dây bị đứt hoặc ngắn mạch.

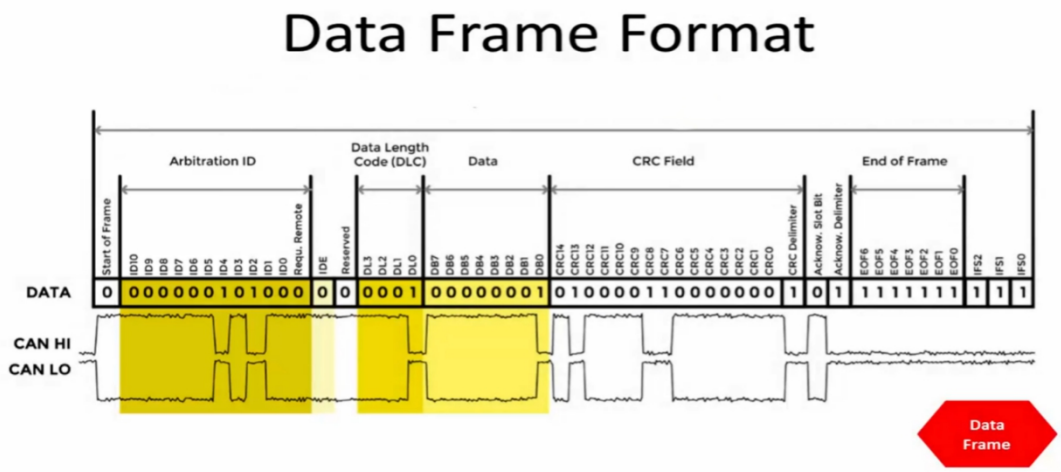
Về tốc độ và khoảng cách, hai thông số này tỉ lệ nghịch với nhau do ràng buộc thời gian truyền lan tín hiệu trong cơ chế arbitration: ở 1 Mbit/s chiều dài bus tối đa khoảng 40m; ở 500 kbit/s khoảng 100m; ở 125 kbit/s có thể đạt 500m. Trong đề tài, khoảng cách giữa STM32 và Raspberry Pi chỉ vài chục centimét nên chọn 500 kbit/s là hoàn toàn hợp lý và cho độ trễ thấp.

Bảng 2.1 Thông số kỹ thuật chính của CAN 2.0 theo ISO 11898 [9][10]

Thông số	Giá trị / Mô tả
Tốc độ dữ liệu tối đa	1 Mbit/s (CAN 2.0); lên đến 8 Mbit/s với CAN FD
Chiều dài bus tối đa	40 m ở 1 Mbit/s; 500 m ở 125 kbit/s
Loại ID frame	Standard 11-bit (CAN 2.0A) và Extended 29-bit (CAN 2.0B)
Phát hiện lỗi	CRC 15-bit, Bit Stuffing, ACK Field, Form Check, Bit Monitoring
Điện áp vi sai	Recessive: CANH ≈ CANL ≈ 2.5V; Dominant: CANH = 3.5V, CANL = 1.5V
Điện trở termination	120 Ω tại mỗi đầu bus theo ISO 11898-2
Kiến trúc truy cập bus	CSMA/NDA – không cần master trung tâm

2.1.3 Cấu trúc khung dữ liệu CAN

CAN định nghĩa bốn loại khung với chức năng khác nhau: Data Frame mang dữ liệu thực tế, Remote Frame để yêu cầu node khác gửi dữ liệu, Error Frame thông báo lỗi phát hiện được, và Overload Frame yêu cầu thêm thời gian trì hoãn.



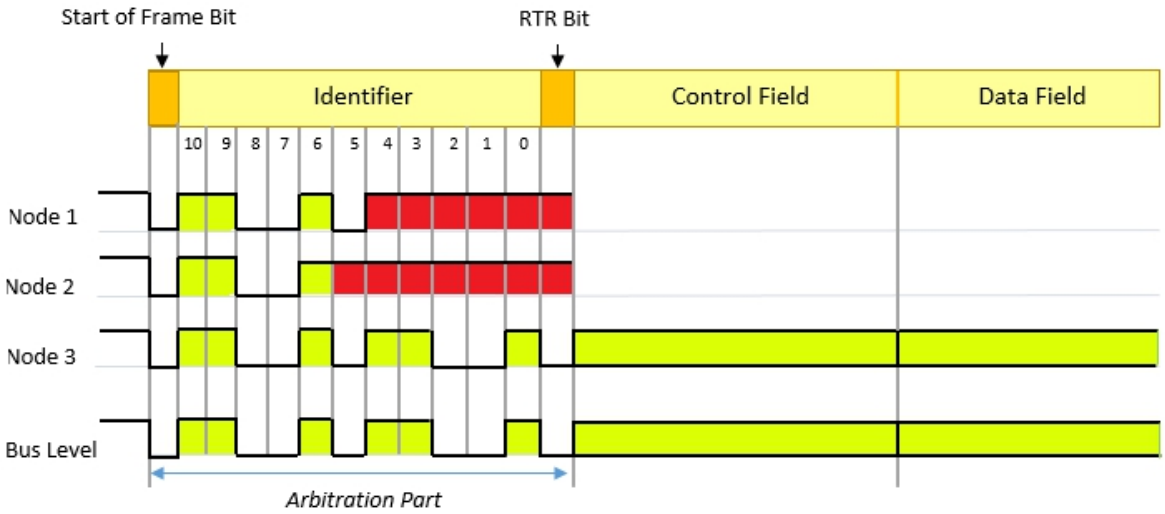
Hình 2.2 Cấu trúc của một khung dữ liệu CAN (Data Frame)

Một Data Frame CAN gồm các trường theo thứ tự: SOF (1 bit) đánh dấu bắt đầu, Arbitration Field (11 hoặc 29 bit ID + RTR) xác định ưu tiên, Control Field (6 bit) chứa DLC (Data Length Code, 0–8 byte), Data Field (0–64 bit payload), CRC Field (15 bit + delimiter) phát hiện lỗi, ACK Field (2 bit) xác nhận, và EOF (7 bit recessive) kết thúc khung. Khi hai node cùng phát một lúc, mỗi node so sánh bit nó gửi với bit trên bus: node nào gửi recessive (1) nhưng thấy dominant (0) biết mình thua phân xử và dừng phát ngay – đây là cơ chế arbitration không phá huỷ dữ liệu của node thắng [10].

Remote Frame có cấu trúc tương tự Data Frame nhưng không có Data Field và bit RTR được set recessive (1). Node gửi Remote Frame với một ID cụ thể để yêu cầu node sở hữu ID đó phát Data Frame tương ứng. Trong thực tế, Remote Frame ít được dùng trong các hệ thống ô tô hiện đại vì dễ gây nhầm lẫn.

2.1.4 Cơ chế phân xử bus

Cơ chế arbitration là điểm đặc biệt nhất của CAN, cho phép multi-master mà không cần coordination từ master trung tâm. Khi bus rảnh (trạng thái idle – tất cả recessive), bất kỳ node nào cũng có thể bắt đầu phát frame bằng cách gửi bit SOF (dominant). Nếu hai node cùng phát đồng thời, quá trình arbitration diễn ra bit-by-bit trong suốt Arbitration Field (phần ID):



Hình 2.3 Minh hoạ quá trình arbitration giữa ba node: Node 1 và Node 2 và Node 3 phát đồng thời; Node 3 thắng

Mỗi node vừa phát bit vừa đọc lại trạng thái bus ngay sau đó. Nếu node phát recessive (1) nhưng đọc lại thấy dominant (0), tức là có node khác đang phát dominant đồng thời – node đó biết mình thua arbitration và dừng phát ngay lập tức, chuyển sang chế độ nhận. Node phát dominant tiếp tục phát bình thường. Do dominant luôn thắng recessive và ID nhỏ hơn có nhiều bit dominant hơn ở vị trí cao, node có ID nhỏ hơn luôn thắng arbitration. Đây là arbitration không phá huỷ: node thắng không cần phát lại, node thua sẽ thử lại khi bus rảnh sau đó. Điều này đảm bảo thông điệp ưu tiên cao (ID nhỏ) luôn được truyền đi trước.

2.1.5 Quá trình truyền và nhận dữ liệu trên mạng CAN

Quá trình truyền một Data Frame diễn ra theo các bước sau: Ứng dụng nạp dữ liệu (ID + payload) vào Transmit Mailbox của CAN controller. Controller chờ bus rảnh (phát hiện 11 bit recessive liên tiếp – IFS + EOF của frame trước). Khi bus rảnh, controller bắt đầu phát SOF (1 bit dominant) và lần lượt phát các trường. Trong Arbitration Field, controller đồng thời đọc lại bus; nếu thua arbitration thì huỷ phát và chờ. Nếu thắng arbitration, controller tiếp tục phát Control + Data + CRC. Sau khi phát CRC Delimiter (recessive), controller phát ACK Slot (recessive) và đọc lại; nếu bus dominant thì xác nhận frame được nhận thành công. Phát ACK Delimiter + EOF hoàn tất frame.

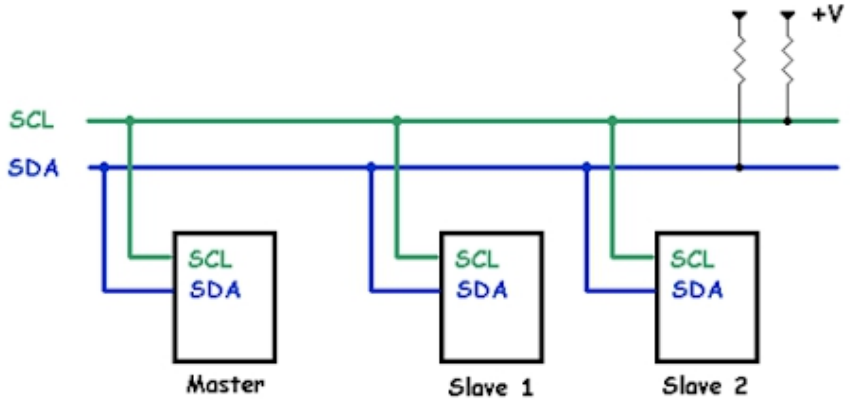
Quá trình nhận diễn ra song song ở tất cả node trên bus: Mỗi node liên tục giám sát bus. Khi phát hiện SOF (dominant), node bắt đầu đồng bộ clock và đọc từng bit. Sau khi nhận xong Arbitration Field, controller so sánh ID nhận được với danh sách Acceptance Filter đã cấu hình trước. Nếu ID khớp với filter, node tiếp tục đọc Data Field vào Receive Buffer và phát dominant vào ACK Slot để xác nhận. Nếu ID không khớp, node vẫn tiếp tục đọc nhưng không lưu dữ liệu và không phát ACK. Sau khi nhận xong, controller kích hoạt ngắt để thông báo cho ứng dụng có dữ liệu mới.

2.2 CHUẨN GIAO THỨC I2C

2.2.1 Tổng quan

I2C (Inter-Integrated Circuit) là một chuẩn giao tiếp nối tiếp đồng bộ do hãng Philips Semiconductors (nay là NXP Semiconductors) phát triển từ năm 1982, ban đầu nhằm kết nối các vi mạch trên cùng một bo mạch với số chân tối thiểu [12]. Đặc trưng nổi bật của I2C là chỉ sử dụng hai đường tín hiệu dùng chung cho mọi thiết bị: SDA (Serial Data – đường dữ liệu) và SCL (Serial Clock – đường xung nhịp). Nhờ số dây ít và cơ chế định địa chỉ tích hợp, một bus I2C có thể kết nối nhiều thiết bị mà không làm tăng số chân của vi điều khiển, rất phù hợp để ghép nối các cảm biến, EEPROM, đồng hồ thời gian thực và các IC ngoại vi tốc độ trung bình.

I2C hoạt động theo mô hình chủ – tớ. Thiết bị chủ (thường là vi điều khiển) khởi tạo và điều khiển quá trình truyền, đồng thời phát xung nhịp trên đường SCL; các thiết bị tớ phản hồi khi được gọi đúng địa chỉ. Bus cho phép nhiều slave và cũng hỗ trợ nhiều master, trong đó cơ chế phân xử bảo đảm chỉ một master chiếm bus tại một thời điểm [12].



Hình 2.4 Sơ đồ kết nối bus I2C với một master và nhiều slave

2.2.2 Cấu trúc bus và nguyên lý hoạt động

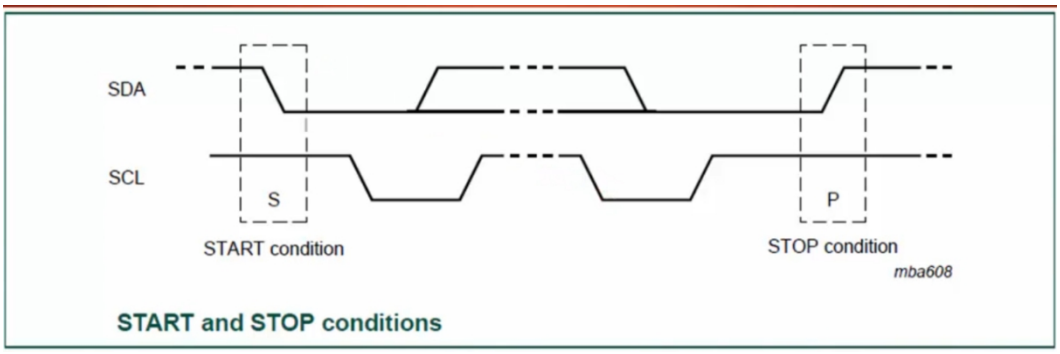
Cả hai đường SDA và SCL đều được thiết kế theo kiểu cực máng hở: thiết bị chỉ có thể kéo đường tín hiệu xuống mức thấp, còn mức cao do điện trở kéo lên nối tới nguồn tạo ra. Vì vậy mỗi bus I2C bắt buộc có điện trở kéo lên trên cả

SDA và SCL; khi không có thiết bị nào tác động, bus ở mức cao (trạng thái rảnh). Thiết kế open-drain cho phép nhiều thiết bị cùng nối vào một đường mà không gây xung đột điện, đồng thời là cơ sở cho cơ chế báo nhận và phân xử của giao thức [13].

Trong một phiên truyền, master sinh xung nhịp trên SCL và quy ước thời điểm dữ liệu hợp lệ: mức trên đường SDA chỉ được phép thay đổi khi SCL ở mức thấp, và phải giữ ổn định trong suốt thời gian SCL ở mức cao. Ngoài ra, giao thức còn hỗ trợ cơ chế kéo dãn xung nhịp, cho phép một slave tạm giữ SCL ở mức thấp để báo rằng nó cần thêm thời gian xử lý trước khi tiếp tục, giúp các thiết bị chậm vẫn theo kịp master.

2.2.3 Định địa chỉ và khung truyền dữ liệu

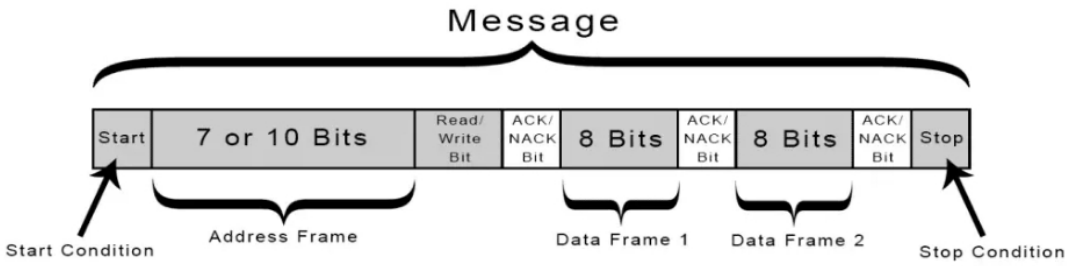
Mỗi phiên giao tiếp được đóng khung bởi hai điều kiện đặc biệt do master tạo ra trên bus. Điều kiện bắt đầu xảy ra khi SDA chuyển từ mức cao xuống thấp trong lúc SCL đang ở mức cao; điều kiện kết thúc xảy ra khi SDA chuyển từ thấp lên cao trong lúc SCL ở mức cao. Hai điều kiện này khác biệt với quy tắc thay đổi dữ liệu thông thường nên được mọi thiết bị nhận biết rõ ràng [12].



Hình 2.5 Điều kiện START và STOP trên bus I2C

Sau điều kiện START, master gửi đi byte địa chỉ gồm 7 bit địa chỉ của slave cần giao tiếp và 1 bit R/W xác định chiều truyền (đọc hay ghi). Slave nào có địa chỉ trùng khớp sẽ đáp lại bằng một bit báo nhận ACK (kéo SDA xuống thấp ở xung nhịp thứ chín); nếu không có thiết bị nào đáp, bus giữ mức cao tương ứng tín hiệu NACK. Sau đó, dữ liệu được truyền theo từng byte 8 bit, gửi bit có trọng số cao trước, và mỗi byte cũng được kết thúc bằng một bit ACK/NACK từ phía

thiết bị nhận. Giao thức còn cho phép điều kiện START lặp lại để master đổi chiều truyền hoặc chuyển sang giao tiếp với thiết bị khác mà không nhả bus [13].



Hình 2.6 Cấu trúc một khung truyền I2C đầy đủ

Với địa chỉ 7 bit, một bus I2C về lý thuyết có thể định danh tới 128 địa chỉ, trong đó một số địa chỉ được dành riêng cho mục đích đặc biệt nên số thiết bị thực dùng nhỏ hơn. Trường hợp cần nhiều thiết bị hơn, chuẩn còn hỗ trợ chế độ địa chỉ 10 bit để mở rộng không gian định danh [12].

2.2.4 Các chế độ tốc độ

Chuẩn I2C định nghĩa nhiều chế độ tốc độ để phù hợp với từng loại ứng dụng: chế độ tiêu chuẩn tới 100 kbit/s, chế độ nhanh tới 400 kbit/s, chế độ nhanh mở rộng tới 1 Mbit/s, và chế độ tốc độ cao tới 3,4 Mbit/s [12]. Các chế độ này tương thích ngược, nên thiết bị tốc độ cao có thể hoạt động trên bus tốc độ thấp hơn.

Nhìn chung, I2C có ưu điểm là số dây ít, hỗ trợ nhiều thiết bị trên cùng một bus và tích hợp sẵn cơ chế định địa chỉ cùng báo nhận, giúp đơn giản hoá phần cứng. Bù lại, do dùng đường open-drain và điện trở kéo lên, tốc độ và khoảng cách truyền của I2C bị hạn chế hơn so với một số chuẩn khác, đồng thời phù hợp nhất cho giao tiếp giữa các vi mạch trong phạm vi gần [13].

2.3 CHUẨN GIAO THỨC SPI

2.3.1 Tổng quan

SPI (Serial Peripheral Interface) là một chuẩn giao tiếp nối tiếp đồng bộ do hãng Motorola phát triển từ giữa thập niên 1980, được dùng rộng rãi để kết nối vi điều khiển với các ngoại vi tốc độ cao như bộ nhớ flash, màn hình, cảm biến và các bộ chuyển đổi ADC/DAC [14]. Khác với I2C dùng hai dây, SPI sử dụng cơ

chế truyền song công toàn phần, cho phép dữ liệu đi và về đồng thời, nhờ đó đạt tốc độ cao hơn đáng kể.

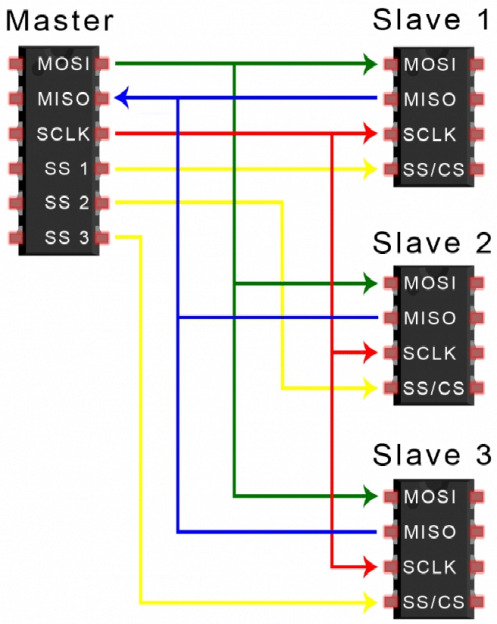
SPI hoạt động theo mô hình master – slave, trong đó thiết bị chủ phát xung nhịp và chủ động khởi tạo mọi giao dịch. Giao thức sử dụng bốn đường tín hiệu:

SCLK (Serial Clock): xung nhịp đồng bộ do master tạo ra.

MOSI (Master Out – Slave In): dữ liệu truyền từ master tới slave.

MISO (Master In – Slave Out): dữ liệu truyền từ slave về master.

SS/CS (Slave Select / Chip Select): đường chọn chip, tích cực mức thấp, dùng để kích hoạt slave cần giao tiếp.



Hình 2.7 Sơ đồ kết nối SPI giữa master và nhiều slave

2.3.2 Cấu trúc bus và nguyên lý hoạt động

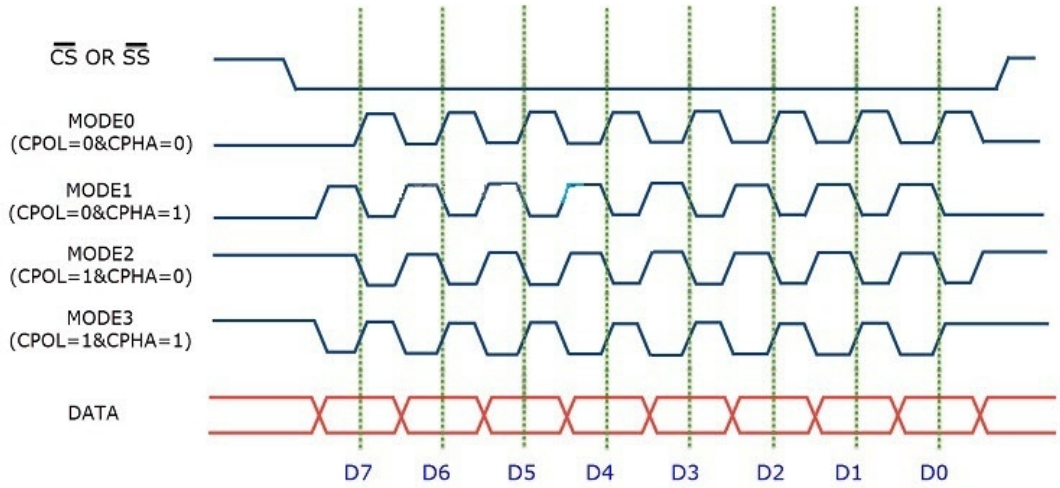
Điểm khác biệt cơ bản của SPI so với I2C là không sử dụng cơ chế định địa chỉ. Thay vào đó, mỗi slave được chọn bằng một đường SS/CS riêng: khi master kéo đường SS của một slave xuống mức thấp, slave đó được kích hoạt và tham gia trao đổi dữ liệu, các slave còn lại ở trạng thái nghỉ. Do đó, khi cần kết nối nhiều slave, master phải dùng thêm một đường SS cho mỗi thiết bị.

Quá trình truyền dữ liệu dựa trên nguyên lý thanh ghi dịch ở cả hai phía. Tại mỗi nhịp của SCLK, master và slave đồng thời dịch ra một bit trên đường gửi và

dịch vào một bit trên đường nhận; sau tám nhịp, hai bên đã hoán đổi xong một byte. Cơ chế này khiến mỗi giao dịch SPI luôn diễn ra theo cả hai chiều cùng lúc, kể cả khi ứng dụng chỉ cần truyền theo một hướng. Dữ liệu thường được truyền với bit có trọng số cao đi trước.

2.3.3 Các chế độ hoạt động

SPI có bốn chế độ hoạt động được xác định bởi hai tham số: cực tính xung nhịp CPOL (Clock Polarity – quy định mức nghỉ của SCLK là cao hay thấp) và pha xung nhịp CPHA (Clock Phase – quy định dữ liệu được lấy mẫu ở sườn xung thứ nhất hay thứ hai). Bốn tổ hợp của CPOL và CPHA tạo thành chế độ 0 đến chế độ 3 [15]. Master và slave bắt buộc phải được cấu hình cùng một chế độ thì mới truyền nhận chính xác.



Hình 2.8 Bốn chế độ SPI theo CPOL và CPHA

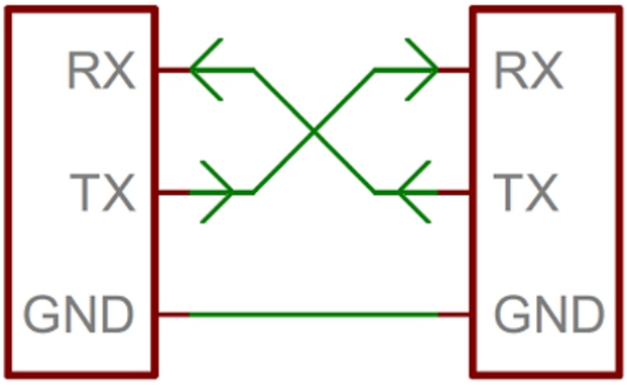
Nhìn chung, SPI có ưu điểm là tốc độ truyền cao, truyền song công toàn phần và phần cứng đơn giản nhờ không cần định địa chỉ hay cơ chế báo nhận phức tạp. Đổi lại, SPI tốn nhiều chân hơn khi số slave tăng (mỗi slave cần một đường SS), không có cơ chế xác nhận dữ liệu tích hợp, và phù hợp nhất cho giao tiếp trong phạm vi gần trên cùng một bo mạch.

2.4 CHUẨN GIAO THỨC UART

2.4.1 Tổng quan

UART (Universal Asynchronous Receiver/Transmitter) là một chuẩn truyền thông nối tiếp không đồng bộ, được sử dụng rất phổ biến để trao đổi dữ liệu giữa hai thiết bị như vi điều khiển với máy tính, mô-đun GPS, mô-đun Bluetooth hay phục vụ gỡ lỗi qua cổng console [16]. Khác với SPI và I2C vốn là các chuẩn đồng bộ có đường xung nhịp chung, UART không truyền tín hiệu xung nhịp; thay vào đó, hai thiết bị thoả thuận trước về tốc độ truyền và định dạng khung để có thể hiểu nhau.

UART chỉ cần hai đường tín hiệu dữ liệu, cho phép truyền song công toàn phần: TX (Transmit – đường phát) và RX (Receive – đường thu). Khi kết nối hai thiết bị, đường TX của thiết bị này nối với đường RX của thiết bị kia và ngược lại, đồng thời hai thiết bị phải dùng chung đường đất (GND). Ở trạng thái rảnh, đường truyền giữ ở mức cao.



Hình 2.9 Sơ đồ kết nối UART giữa hai thiết bị

2.4.2 Cấu trúc khung truyền dữ liệu

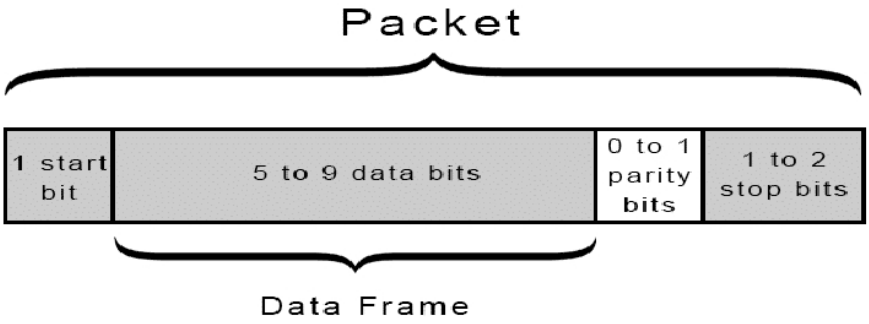
Do không có xung nhịp chung, UART đóng gói dữ liệu thành từng khung có cấu trúc cố định để bên nhận nhận biết được điểm bắt đầu và kết thúc. Một khung UART tiêu chuẩn gồm các thành phần:

Bit bắt đầu (Start bit): một bit mức thấp báo hiệu khung bắt đầu; sườn xuống của bit này giúp bên nhận xác định thời điểm để bắt đầu lấy mẫu.

Các bit dữ liệu (Data bits): thường là 8 bit (có thể từ 5 đến 9 bit tuỳ cấu hình), truyền bit có trọng số thấp trước.

Bit chẵn lẻ (Parity bit): một bit tuỳ chọn dùng để kiểm tra lỗi đơn giản theo quy tắc chẵn hoặc lẻ.

Bit kết thúc (Stop bit): một hoặc hai bit mức cao đánh dấu kết thúc khung và đưa đường truyền về trạng thái rảnh.



Hình 2.10 Cấu trúc khung truyền dữ liệu UART

2.4.3 Tốc độ baud và cơ chế đồng bộ

Tốc độ truyền của UART được đặc trưng bởi baud rate — số bit truyền trong một giây (ví dụ 9600, 57600 hay 115200 bit/s). Vì hai thiết bị không chia sẻ xung nhịp, cả bên phát và bên thu bắt buộc phải cấu hình cùng một baud rate và cùng định dạng khung; nếu sai lệch vượt quá ngưỡng cho phép, dữ liệu nhận được sẽ bị lỗi [17]. Để bù cho việc không có xung nhịp chung, bên thu thường lấy mẫu mỗi bit ở giữa khoảng thời gian bit (dựa trên bộ định thời nội bộ chạy nhanh hơn baud rate nhiều lần), nhờ đó giảm sai số do lệch pha.

Cần phân biệt UART (giao thức logic ở mức tín hiệu TTL/CMOS của vi điều khiển) với các chuẩn điện như RS-232 hay RS-485; khi cần truyền theo các chuẩn điện này, hệ thống phải bổ sung mạch chuyển mức tín hiệu.

Nhìn chung, UART có ưu điểm là đơn giản, ít dây, không cần đường xung nhịp và được hỗ trợ rộng rãi, đặc biệt thuận tiện cho việc gỡ lỗi và truyền dữ liệu giữa hai thiết bị. Hạn chế của UART là chỉ kết nối điểm – điểm giữa hai thiết bị, đòi hỏi hai bên phải khớp tốc độ baud, đồng thời tốc độ và khoảng cách truyền bị giới hạn so với một số chuẩn khác.

2.5 GIỚI THIỆU VỀ STM32CUBEIDE

2.5.1 Tổng quan

STM32CubeIDE là môi trường phát triển tích hợp chính thức do hãng STMicroelectronics phát hành, phục vụ việc lập trình, biên dịch và gỡ lỗi cho các dòng vi điều khiển STM32. Đây là một công cụ miễn phí, chạy được trên nhiều hệ điều hành (Windows, Linux và macOS), được xây dựng trên nền tảng Eclipse/CDT cùng bộ trình biên dịch GNU C/C++ (GCC) cho lõi ARM và trình gỡ lỗi GDB. Nhờ kế thừa hệ sinh thái Eclipse, STM32CubeIDE cung cấp đầy đủ các tính năng của một IDE chuyên nghiệp như trình soạn thảo mã thông minh, quản lý dự án, gợi ý cú pháp và tích hợp công cụ gỡ lỗi.

Điểm nổi bật của STM32CubeIDE là gộp chung khả năng cấu hình đồ hoạ của công cụ STM32CubeMX vào trong cùng một môi trường. Nhờ đó, toàn bộ quy trình phát triển — từ cấu hình phần cứng, sinh mã khởi tạo, viết mã ứng dụng, biên dịch cho đến nạp và gỡ lỗi trên chip — đều được thực hiện trong một phần mềm duy nhất, giúp rút ngắn thời gian và giảm sai sót khi chuyển đổi giữa nhiều công cụ rời rạc.

2.5.2 Các tính năng chính

STM32CubeIDE cung cấp một quy trình làm việc khép kín với các nhóm tính năng tiêu biểu:

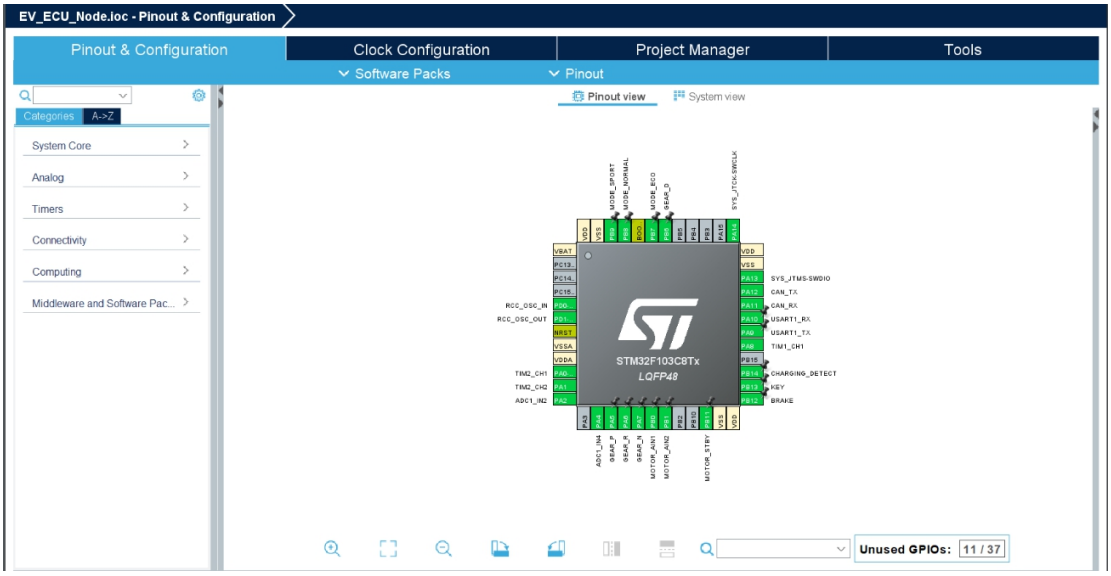
Cấu hình đồ hoạ và sinh mã tự động: thông qua giao diện CubeMX tích hợp, người dùng có thể gán chức năng cho từng chân, thiết lập cây xung nhịp, bật và cấu hình các ngoại vi cũng như các lớp phần mềm trung gian. Sau khi cấu hình, công cụ tự động sinh ra mã khởi tạo bằng thư viện HAL hoặc LL, giúp lập trình viên không phải thao tác trực tiếp với các thanh ghi ở giai đoạn khởi tạo.

Soạn thảo và biên dịch: trình soạn thảo hỗ trợ tô màu cú pháp, tự động hoàn thiện mã và điều hướng dự án; trình biên dịch GCC thực hiện biên dịch, liên kết và tạo ra tệp nạp cho vi điều khiển.

Nạp chương trình và gỡ lỗi: STM32CubeIDE làm việc trực tiếp với mạch nạp/gỡ lỗi ST-LINK, cho phép nạp firmware và thực hiện các thao tác gỡ lỗi như

đặt điểm dừng, chạy từng bước, theo dõi biến, xem nội dung thanh ghi và bộ nhớ theo thời gian thực.

Hỗ trợ phần mềm trung gian: công cụ tích hợp sẵn nhiều gói middleware phổ biến như hệ điều hành thời gian thực FreeRTOS, ngăn xếp USB, hệ thống tệp và các thư viện truyền thông, cho phép thêm vào dự án và cấu hình ngay trong giao diện đồ hoạ.



Hình 2.11 Giao diện cấu hình chân và ngoại vi trong STM32CubeIDE

Với việc hợp nhất các bước cấu hình, lập trình và gỡ lỗi trong một môi trường thống nhất cùng khả năng sinh mã tự động, STM32CubeIDE giúp quá trình phát triển phần mềm nhúng cho STM32 trở nên trực quan và hiệu quả, đặc biệt phù hợp với các dự án cần tích hợp nhiều ngoại vi và phần mềm trung gian.

2.6 GIỚI THIỆU HỆ ĐIỀU HÀNH THỜI GIAN THỰC FREERTOS

2.6.1 Tổng quan về hệ điều hành thời gian thực

Hệ điều hành thời gian thực là một loại hệ điều hành được thiết kế để xử lý các tác vụ và sự kiện trong những ràng buộc thời gian xác định và có thể dự đoán được. Khác với hệ điều hành thông thường vốn ưu tiên thông lượng trung bình, đặc trưng quan trọng nhất của RTOS là tính tất định: hệ thống phải bảo đảm tác vụ quan trọng được đáp ứng đúng hạn, thay vì chỉ chạy "nhanh nhất có thể".

Trong các hệ thống nhúng, RTOS cho phép chia chương trình thành nhiều tác vụ chạy gần như song song, mỗi tác vụ đảm nhận một chức năng riêng. Một

thành phần lõi gọi là bộ lập lịch chịu trách nhiệm quyết định tác vụ nào được sử dụng vi xử lý tại mỗi thời điểm, dựa trên độ ưu tiên và trạng thái của các tác vụ. Nhờ mô hình đa tác vụ này, lập trình viên có thể tổ chức phần mềm rõ ràng theo chức năng và bảo đảm các nhiệm vụ có yêu cầu thời gian thực được xử lý kịp thời.

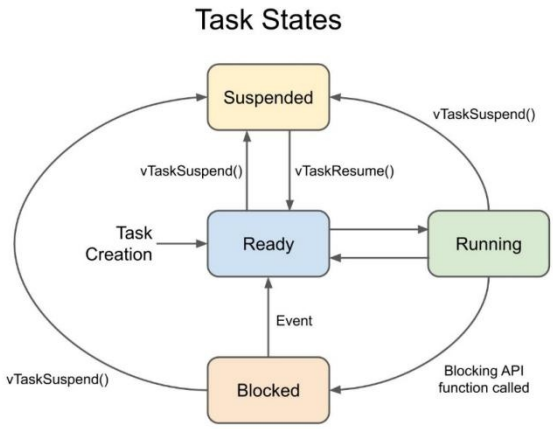
2.6.2 Giới thiệu FreeRTOS

FreeRTOS là một nhân thời gian thực mã nguồn mở, gọn nhẹ, được phát triển dành riêng cho các vi điều khiển và hệ thống nhúng có tài nguyên hạn chế. Phần mềm ban đầu do Richard Barry phát triển và hiện được Amazon Web Services bảo trợ, phát hành theo giấy phép mã nguồn mở nên có thể sử dụng miễn phí kể cả trong sản phẩm thương mại. Với kích thước nhỏ gọn và khả năng chuyển đổi trên nhiều kiến trúc vi xử lý khác nhau, FreeRTOS trở thành một trong những RTOS phổ biến nhất trong lĩnh vực nhúng.

FreeRTOS cung cấp các dịch vụ cốt lõi của một nhân thời gian thực như quản lý tác vụ, lập lịch, đồng bộ và truyền thông giữa các tác vụ, cùng cơ chế quản lý bộ nhớ động. Bên cạnh đó, để chuẩn hoá giao diện lập trình, các nhà sản xuất vi điều khiển thường cung cấp thêm một lớp bao như CMSIS-RTOS của ARM, giúp mã nguồn dễ chuyển đổi giữa các nền tảng.

2.6.3 Các thành phần và cơ chế chính

Tác vụ và bộ lập lịch. Mỗi tác vụ trong FreeRTOS là một luồng thực thi độc lập, có mức ưu tiên và vùng ngăn xếp riêng. Tại một thời điểm, một tác vụ có thể ở các trạng thái: đang chạy, sẵn sàng, bị chặn chờ sự kiện hoặc tạm dừng. FreeRTOS sử dụng cơ chế lập lịch theo độ ưu tiên có quyền ưu tiên chiếm dụng: tác vụ có độ ưu tiên cao hơn khi chuyển sang trạng thái sẵn sàng sẽ lập tức chiếm quyền sử dụng vi xử lý; các tác vụ cùng mức ưu tiên chia sẻ thời gian theo kiểu xoay vòng.



Hình 2.12 Sơ đồ chuyển trạng thái của một tác vụ trong FreeRTOS

Ngắt định thời (tick). Nhân RTOS dựa trên một ngắt định thời tuần hoàn gọi là tick để đếm thời gian hệ thống, tạo độ trễ chính xác và phục vụ việc xoay vòng giữa các tác vụ cùng độ ưu tiên. Tần số tick có thể cấu hình tuỳ theo yêu cầu của ứng dụng.

Đồng bộ và truyền thông liên tác vụ. Khi nhiều tác vụ cùng chia sẻ dữ liệu hoặc tài nguyên, FreeRTOS cung cấp các đối tượng nhân để phối hợp một cách an toàn:

Hàng đợi (Queue): truyền dữ liệu giữa các tác vụ theo mô hình producer – consumer.

Cờ báo hiệu (Semaphore): gồm semaphore nhị phân và semaphore đếm, dùng để đồng bộ tác vụ với sự kiện hoặc quản lý tài nguyên.

Khoá loại trừ tương hỗ (Mutex): bảo vệ tài nguyên dùng chung, có cơ chế kế thừa độ ưu tiên nhằm hạn chế hiện tượng đảo ngược độ ưu tiên.

Ngoài ra, FreeRTOS còn hỗ trợ nhiều phương án cấp phát bộ nhớ động (heap) khác nhau để phù hợp với từng mức yêu cầu về tài nguyên và độ phức tạp của hệ thống. Nhờ tập hợp các cơ chế trên, FreeRTOS cho phép xây dựng phần mềm nhúng đa tác vụ vừa có cấu trúc rõ ràng, vừa đáp ứng được các ràng buộc thời gian thực.

2.7 GIỚI THIỆU VỀ QT/QML

2.7.1 Tổng quan về Qt

Qt là một nền tảng phát triển ứng dụng đa nền tảng, chủ yếu dựa trên ngôn ngữ C++, ban đầu do công ty Trolltech phát triển và hiện được duy trì bởi The Qt Company. Qt cho phép xây dựng cả ứng dụng có giao diện đồ hoạ (GUI) lẫn ứng dụng không giao diện, với cùng một mã nguồn có thể biên dịch và chạy trên nhiều hệ điều hành khác nhau như Windows, Linux, macOS, Android cũng như các hệ thống nhúng. Nhờ tính đa nền tảng và bộ thư viện phong phú, Qt được sử dụng rộng rãi trong nhiều lĩnh vực, trong đó có giao diện người – máy (HMI) cho ô tô và các thiết bị nhúng.

Qt được tổ chức thành nhiều mô-đun phục vụ các nhu cầu khác nhau, từ dựng giao diện, xử lý mạng, đa phương tiện cho đến truy cập cơ sở dữ liệu. Trong đó, hai hướng xây dựng giao diện phổ biến là Qt Widgets (giao diện truyền thống cho ứng dụng máy tính để bàn) và Qt Quick (giao diện hiện đại, mượt mà, phù hợp cho màn hình cảm ứng và thiết bị nhúng).



Hình 2.13 Biểu tượng của nền tảng Qt

2.7.2 Ngôn ngữ QML và Qt Quick

QML (Qt Modeling Language) là một ngôn ngữ khai báo dùng để thiết kế giao diện người dùng, thuộc mô-đun Qt Quick. Thay vì mô tả tuần tự từng bước dựng giao diện như cách lập trình mệnh lệnh thông thường, QML mô tả giao diện dưới dạng một cây các phần tử với các thuộc tính, giúp việc xây dựng và bảo trì giao diện trực quan hơn. QML hỗ trợ cơ chế ràng buộc thuộc tính, cho phép một thuộc tính tự động cập nhật khi giá trị mà nó phụ thuộc thay đổi, đồng thời có thể nhúng mã JavaScript để xử lý các logic đơn giản ngay trong giao diện.

Một mô hình kiến trúc thường gặp khi dùng Qt Quick là tách phần giao diện (viết bằng QML) khỏi phần xử lý nghiệp vụ (viết bằng C++): lớp C++ đảm nhận tính toán và quản lý dữ liệu, sau đó cung cấp dữ liệu cho lớp QML hiển thị. Cách phân tách này giúp giao diện và logic phát triển độc lập, dễ mở rộng và bảo trì.

CHƯƠNG 3

THIẾT KẾ VÀ THI CÔNG HỆ THỐNG

3.1 KIẾN TRÚC TỔNG THỂ HỆ THỐNG

Hệ thống được xây dựng theo mô hình phân tầng theo kiến trúc của một xe điện thực tế, trong đó các chức năng đọc tín hiệu, ra quyết định điều khiển và hiển thị thông tin được phân chia cho ba khối xử lý độc lập, giao tiếp với nhau qua mạng CAN Bus. Mục này trình bày sơ đồ khối tổng thể, vai trò của từng tầng, các nguyên tắc thiết kế cốt lõi và luồng dữ liệu xuyên suốt hệ thống, làm nền tảng cho các thiết kế chi tiết ở những mục tiếp theo.

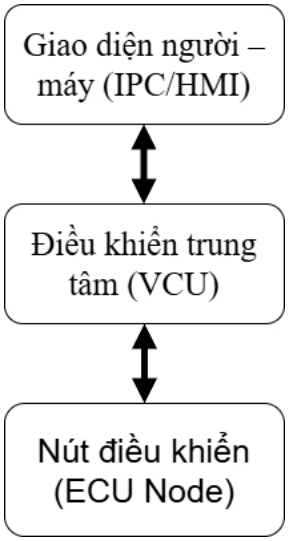
3.1.1 Sơ đồ khối tổng thể hệ thống

Kiến trúc tổng thể của hệ thống được tổ chức thành ba tầng chức năng như trình bày ở Hình 3.1: tầng nút điều khiển (ECU Node) chạy trên vi điều khiển STM32F103C8T6, tầng điều khiển trung tâm (VCU) và tầng giao diện người – máy (IPC/HMI) cùng chạy trên máy tính nhúng Raspberry Pi 4B.

Tầng ECU Node đảm nhiệm toàn bộ phần tương tác trực tiếp với phần cứng: đọc các tín hiệu input từ người lái, đo điện áp pack pin, đọc encoder để xác định tốc độ, đồng thời điều khiển động cơ thông qua driver TB6612FNG. STM32 trao đổi dữ liệu với phần còn lại của hệ thống thông qua bộ thu phát TJA1050.

Mạng CAN Bus tốc độ 500 kbps đóng vai trò xương sống truyền thông, kết nối STM32 với Raspberry Pi. Phía Raspberry Pi sử dụng module điều khiển CAN MCP2515 giao tiếp với ECU Node qua bus SPI.

Trên Raspberry Pi, bao gồm 2 thành phần phần mềm: khối VCU thực thi state machine, logic an toàn và chẩn đoán; khối IPC/HMI dựng giao diện cụm đồng hồ số bằng Qt/QML và xuất ra màn hình Waveshare 7 inch qua cổng HDMI. Việc đặt VCU và IPC trên cùng một nền tảng cho phép hai khối chia sẻ dữ liệu trực tiếp trong bộ nhớ tiến trình thay vì phải truyền qua mạng, nhờ đó giảm độ trễ và đơn giản hóa việc đồng bộ.



Hình 3.1 Sơ đồ khối tổng thể hệ thống VCU-IPC

3.1.2 Phân chia chức năng theo ba tầng

Việc phân chia chức năng tuân theo nguyên tắc mỗi tầng chỉ đảm nhiệm đúng nhóm nhiệm vụ phù hợp với năng lực phần cứng của nó. STM32 là vi điều khiển thời gian thực, mạnh về điều khiển ngoại vi mức thấp và đảm bảo thời gian đáp ứng ổn định, nên được giao toàn bộ phần đọc cảm biến và điều khiển chấp hành. Raspberry Pi có năng lực tính toán và đồ họa cao hơn nhiều, phù hợp để xử lý logic phức tạp và xuất kết quả ra giao diện. Bảng 3.1 tóm tắt sự phân bổ này.

Bảng 3.1 Phân bổ chức năng giữa các tầng của hệ thống

Tầng	Nền tảng	Chức năng chính
ECU Node	STM32F103C8T6 + FreeRTOS	Đọc input số và analog; điều khiển motor (PWM, chiều quay); đo điện áp pin và ước lượng SoC; đọc encoder rồi tính tốc độ và quãng đường; đóng gói và truyền nhận khung CAN; giám sát watchdog lệnh điều khiển.
VCU	Raspberry Pi 4B (Linux, C++)	Nhận và giải mã khung CAN; thực thi state machine 6 trạng thái; đánh giá lỗi và quyết định lệnh điều khiển; quản lý chẩn đoán DTC và lưu trữ quãng đường.

Tầng	Nền tảng	Chức năng chính
IPC / HMI	Raspberry Pi 4B (Qt/QML)	Hiển thị thời gian thực tốc độ, SoC, chế độ lái và trạng thái xe; thể hiện cảnh báo lỗi và danh sách mã chẩn đoán; chuyển cảnh giao diện theo trạng thái xe.

3.1.3 Các nguyên tắc thiết kế cốt lõi

Toàn bộ kiến trúc được thiết kế theo bốn nguyên tắc, vừa phản ánh tiêu chuẩn trong các hệ thống nhúng ô tô, vừa phù hợp với phạm vi đồ án.

Thứ nhất là nguyên tắc nguồn quyết định duy nhất. Mọi quyết định điều khiển — cho phép motor chạy hay không, mức công suất bao nhiêu, xe đang ở trạng thái nào — đều do state machine trên VCU đưa ra. STM32 không tự quyết định mà chỉ thực thi đúng lệnh nhận được qua khung CAN 0x200. Cách phân quyền này tránh xung đột logic giữa hai bộ xử lý và giúp việc kiểm thử, gỡ lỗi dễ tìm ra nguyên nhân.

Thứ hai là nguyên tắc tách biệt mối quan tâm. Ba nhóm nhiệm vụ — tương tác phần cứng, ra quyết định và hiển thị — được cô lập ở ba tầng và giao tiếp với nhau qua những giao diện rõ ràng: khung CAN giữa STM32 và VCU, cơ chế binding thuộc tính giữa VCU và IPC. Nhờ đó, mỗi tầng có thể được phát triển và kiểm thử độc lập với phần còn lại của hệ thống.

Thứ ba là nguyên tắc đáp ứng thời gian thực. Các tác vụ quan trọng được tổ chức thành những chu kỳ cố định và gán mức ưu tiên phù hợp: STM32 gửi dữ liệu tốc độ mỗi 20 ms, state machine trên VCU chạy lại mỗi 20 ms. Việc cố định chu kỳ giúp hệ thống có hành vi ổn định và phản ứng kịp thời với thao tác của người lái.

Thứ tư là nguyên tắc an toàn ưu tiên. Hệ thống luôn khởi động ở trạng thái OFF an toàn nhất; khi phát hiện lỗi nghiêm trọng hoặc khi STM32 mất lệnh điều khiển từ VCU quá thời gian cho phép, động cơ bị cắt công suất ngay lập tức. Trong state machine, trạng thái lỗi luôn được dành mức ưu tiên xử lý cao nhất.

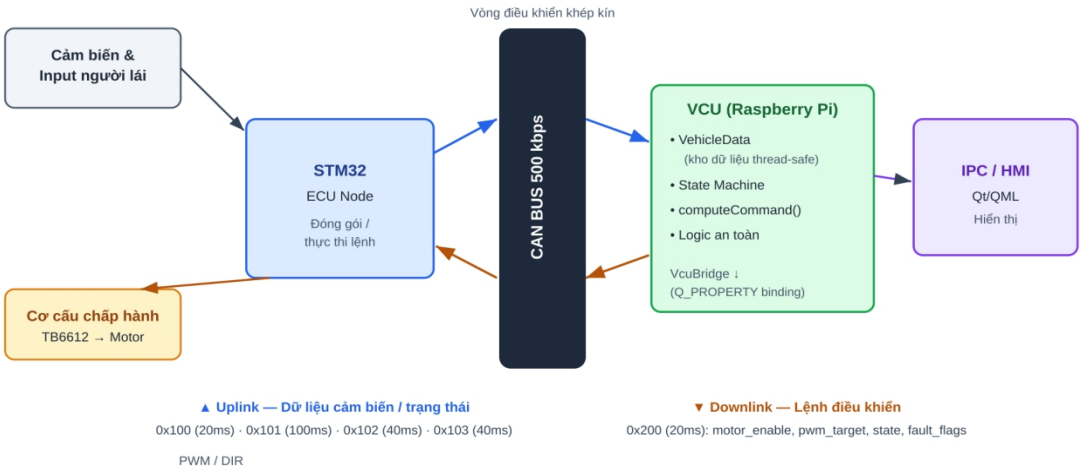
3.1.4 Luồng dữ liệu tổng thể

Dữ liệu trong hệ thống lưu thông theo hai chiều, tạo thành một vòng điều khiển khép kín như minh họa ở Hình 3.2.

Chiều đi lên mang dữ liệu cảm biến và trạng thái phần cứng. STM32 đọc input người lái, điện áp pin và encoder, sau đó đóng gói thành các khung CAN theo lịch định sẵn: khung 0x100 (tốc độ và độ mở ga) mỗi 20 ms, khung 0x101 (điện áp và SoC) mỗi 100 ms, khung 0x102 (trạng thái các input số) và khung 0x103 (quãng đường) mỗi 40 ms. VCU nhận các khung này, giải mã và cập nhật vào kho dữ liệu trung tâm VehicleData. Từ đây, lớp cầu nối VcuBridge đẩy dữ liệu lên giao diện QML thông qua cơ chế property binding để IPC hiển thị theo thời gian thực.

Chiều đi xuống mang lệnh điều khiển. Sau mỗi chu kỳ, state machine của VCU tính ra lệnh điều khiển gồm cờ cho phép motor, mức PWM mục tiêu, trạng thái xe hiện tại và cờ lỗi, rồi đóng gói vào khung CAN 0x200 gửi xuống STM32 mỗi 20 ms. STM32 nhận lệnh và điều khiển động cơ tương ứng qua driver TB6612FNG.

Sự kết hợp của hai chiều dữ liệu tạo thành một vòng kín: tín hiệu người lái → STM32 → VCU → quyết định điều khiển → STM32 → động cơ → encoder phản hồi → STM32 → VCU. Toàn bộ chu trình được lặp lại ổn định ở chu kỳ 20 ms, bảo đảm xe phản ứng mượt mà và ổn định với thao tác của người lái.

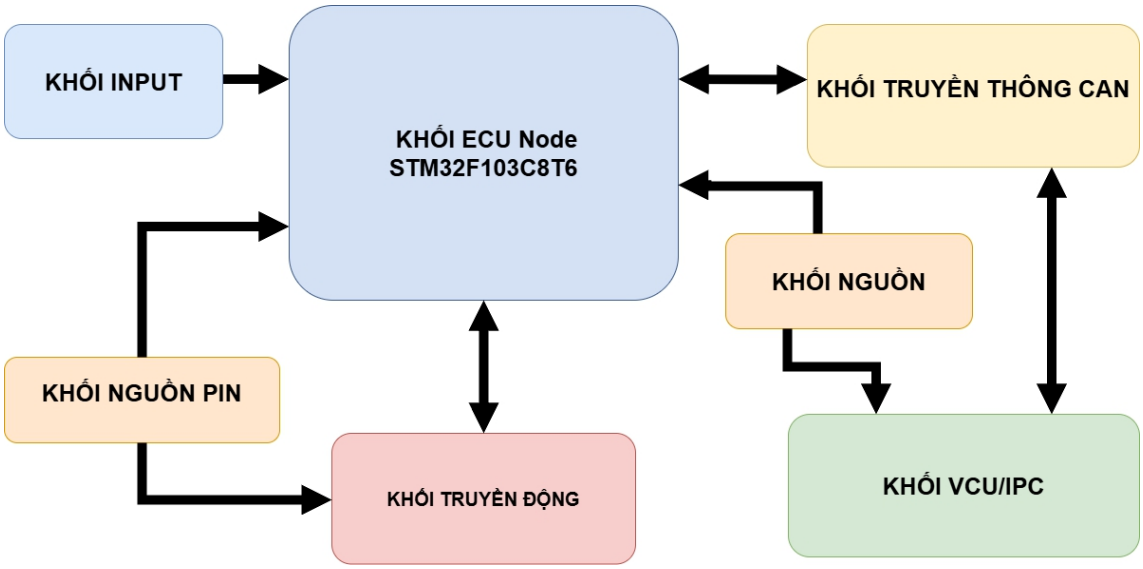


Hình 3.2 Luồng dữ liệu tổng thể của hệ thống

3.2 THIẾT KẾ PHẦN CỨNG

3.2.1 Sơ đồ khối phần cứng tổng thể

Phần cứng hệ thống gồm bảy khối chức năng: khối ECU Node trung tâm (STM32), khối nguồn pin, khối truyền động, khối input (số và analog), khối truyền thông CAN, khối VCU/HMI (Raspberry Pi - màn hình) và khối nguồn. STM32 là điểm hội tụ của hệ thống: nó thu thập tín hiệu từ khối input, encoder của khối truyền động và khối nguồn pin, điều khiển khối truyền động, đồng thời trao đổi dữ liệu với khối VCU/HMI thông qua khối truyền thông CAN. Khối nguồn dùng để cấp nguồn cho STM32 và khối VCU/HMI. Quan hệ kết nối giữa các khối được minh họa ở Hình 3.3.



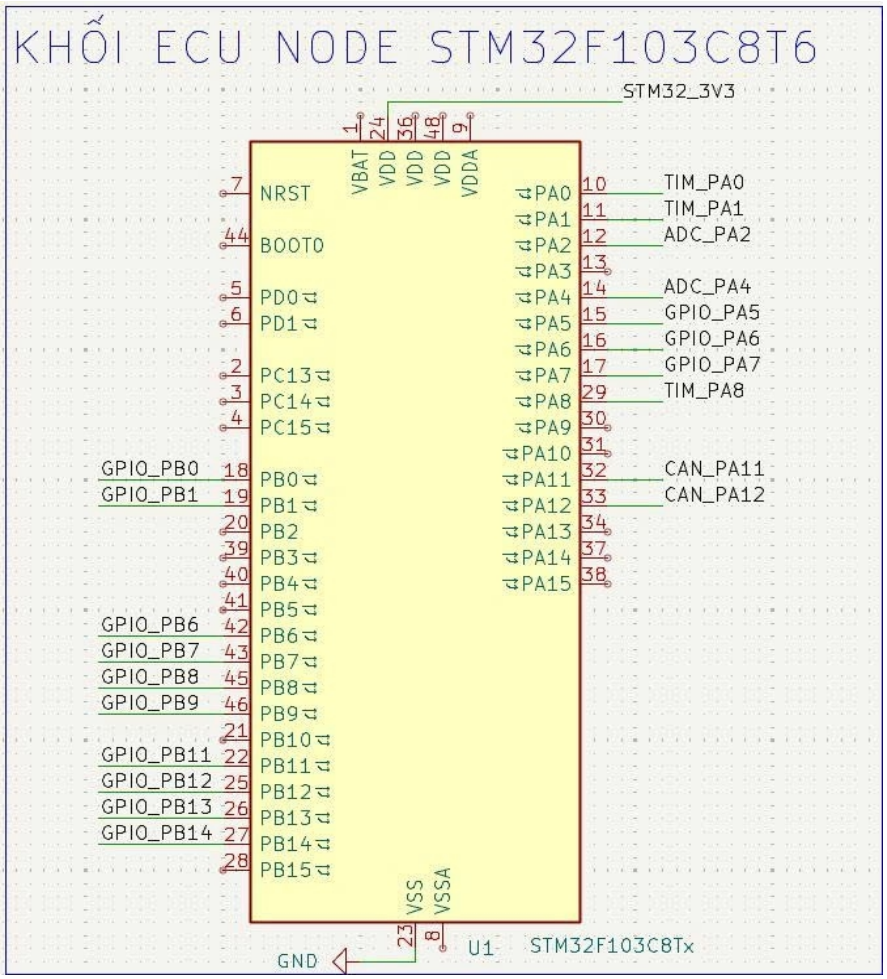
Hình 3.3 Sơ đồ khối phần cứng tổng thể của hệ thống

3.2.2 Khối ECU Node (STM32F103C8T6)

STM32F103C8T6 (board Blue Pill) là nút điều khiển của hệ thống. Trong kiến trúc đã chọn, khối này đảm nhiệm toàn bộ phần tương tác phần cứng mức thấp: đọc input số và analog, đọc cảm biến phát hiện sạc pin, đo điện áp pin, điều khiển động cơ, đọc encoder và truyền nhận khung CAN.

Để thực hiện các chức năng trên, firmware khai thác một tập ngoại vi của STM32 (chi tiết cấu hình ở mục 3.2.8): ngoại vi bxCAN cho truyền thông, ADC1 cho hai ngõ analog, TIM1 cho PWM điều khiển động cơ, TIM2 cho đọc encoder

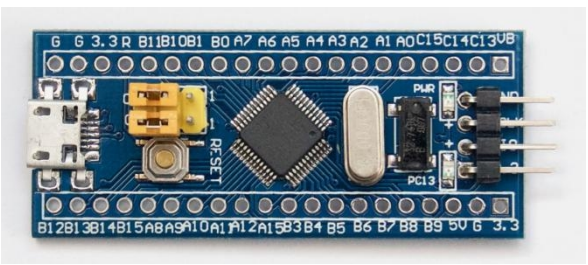
ở chế độ Encoder Interface, USART1 cho debug, cùng nhiều chân GPIO cho tín hiệu vào/ra số. Việc lựa chọn STM32F103C8T6 cho tầng ECU xuất phát từ khả năng đáp ứng thời gian thực ổn định và sự tích hợp sẵn các ngoại vi điều khiển cần thiết, đặc biệt là ngoại vi CAN ngay trên chip.



Hình 3.4 Sơ đồ nối dây khối ECU Node

Danh sách thành phần dùng trong khối ECU Node:

1. STM32F103C8T6 (board Blue Pill)



Hình 3.5 STM32F103C8T6 (board Blue Pill)

Thông số kỹ thuật

MCU: STM32F103C8T6 (ARM Cortex-M3 32-bit)

Xung nhịp tối đa: 72 MHz

Bộ nhớ: 64 KB Flash, 20 KB SRAM

Điện áp hoạt động (I/O): 3.3 V

Ngoại vi sử dụng: bxCAN, ADC, Timer (PWM/Encoder), USART, GPIO

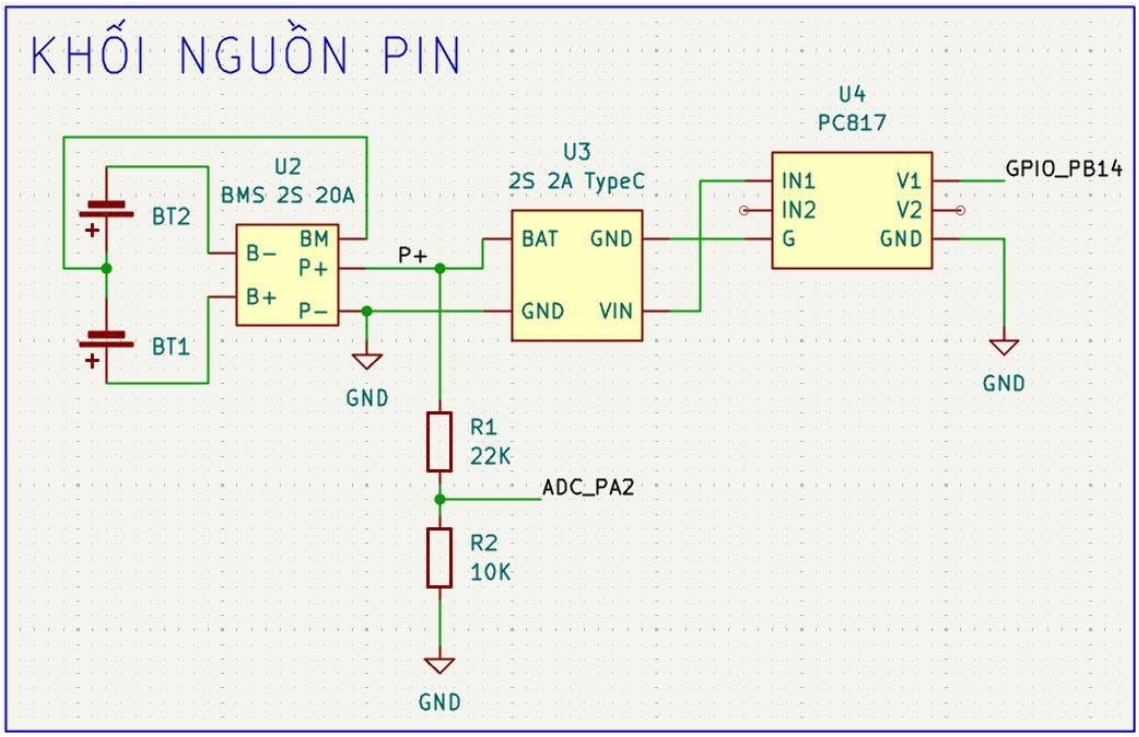
Lý do lựa chọn thành phần: STM32F103C8T6 được chọn trước hết vì tích hợp sẵn ngoại vi bxCAN ngay trên chip, giúp giao tiếp CAN mà không cần thêm chip điều khiển ngoài. Chip cũng cung cấp đủ ngoại vi cho các chức năng còn lại: TIM1 cho PWM, TIM2 cho đọc encoder, ADC1 hai kênh và USART1 cho debug . Mức logic 3,3 V tương thích trực tiếp với phần lớn module trong hệ thống . Ngoài ra board Blue Pill có giá thành thấp, rất phổ biến, tài liệu và cộng đồng hỗ trợ dồi dào nên phù hợp với phạm vi đồ án.

3.2.3 Khối nguồn pin

Nguồn của mô hình là pack pin gồm hai cell 18650 ghép nối tiếp (cấu hình 2S), điện áp danh định 7,4 V và điện áp đầy khoảng 8,4 V. Pack được bảo vệ bởi mạch BMS 2S 20A - chống quá sạc, quá xả, quá dòng và ngắn mạch - với các chân cân bằng B+, BM, B-; đầu ra P+/P- sau BMS cấp nguồn động lực cho khối truyền động và đồng thời được đưa đi qua mạch chia áp để trích đo điện áp pin.

Vì điện áp pack (tối đa 8,4 V) vượt ngưỡng chịu đựng của chân ADC STM32 (tối đa 3,3 V), cần một mạch chia áp để hạ điện áp đo về dải an toàn. Mạch sử dụng hai điện trở nối tiếp R1 = 22 kΩ (nối từ P+) và R2 = 10 kΩ (nối xuống GND), điểm giữa hai điện trở đưa vào chân PA2.

Mạch sạc 2S Type-C đảm nhiệm nạp lại cho pack đồng thời phát hiện được trạng thái sạc của pin. Mạch sạc 2S Type-C gồm 4 chân: 2 chân BAT, GND được nối thẳng vào P+, P- của BMS để nạp điện áp cho pack pin; hai chân phía còn lại: GND và VIN được nối vào kênh 1 của ngõ vào của mạch Opto Cách Ly 2 Kênh PC817, rồi tại ngõ ra của mạch Opto: chân V1 được đưa về chân charging detect của STM32 để phát hiện trạng thái sạc của pin.



Hình 3.6 Sơ đồ nối dây khối nguồn pin

Danh sách thành phần dùng trong khối nguồn pin:

1. Pack pin gồm hai cell 18650 ghép nối tiếp

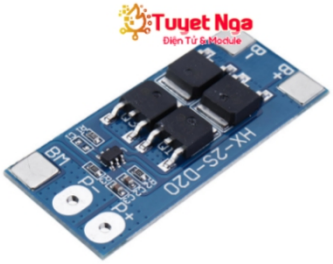
   Thông số kỹ thuật

   Cấu hình: 2 cell Li-ion mắc nối tiếp (2S)

   Điện áp danh định: 7,4 V

   Điện áp sạc đầy: 8,4 V

2. Mạch bảo vệ pin BMS 2S 20A



Hình 3.7 Mạch bảo vệ pin BMS 2S 20A

Thông số kỹ thuật

Loại: BMS cho pack Li-ion 2S

Điện áp sạc: 8,4 V

Dòng bảo vệ tối đa: 20 A

3. Mạch chia áp gồm 2 điện trở khác nhau nối tiếp

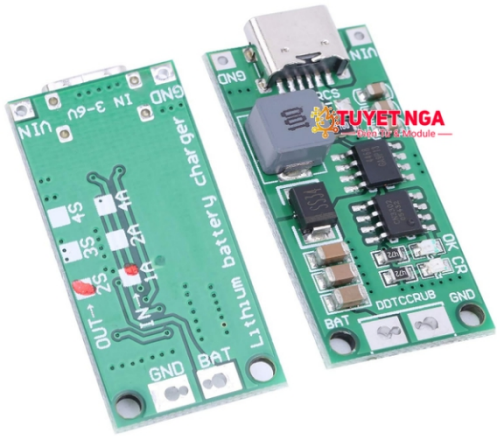
Thông số kỹ thuật

Điện trở: 22 kΩ và 10 kΩ (±1%)

Tỉ lệ chia áp: 0,3125

Dải đo tối đa tại ADC: 10,56 V

4. Mạch sạc Pin 2S 2A Cổng TypeC



Hình 3.8 Mạch sạc Pin 2S 2A Cổng TypeC

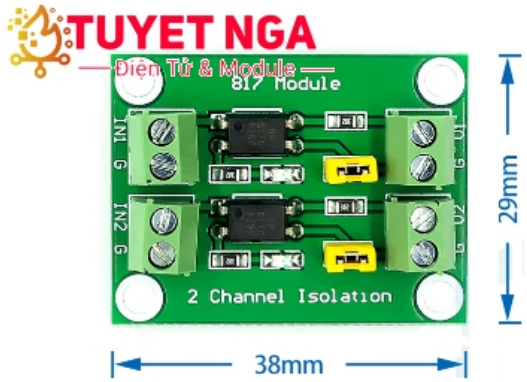
Thông số kỹ thuật

Điện áp sạc đầu ra: 8,4 V

Dòng sạc: ~1,1 A

Chức năng: sạc pin 2S, đèn báo trạng thái sạc

5. Mạch Opto Cách Ly 2 Kênh PC817



Hình 3.9 Mạch Opto Cách Ly 2 Kênh PC817

Thông số kỹ thuật

Loại: Opto cách ly 2 kênh (PC817)

Điện áp ngõ vào: 3,6–24 VDC

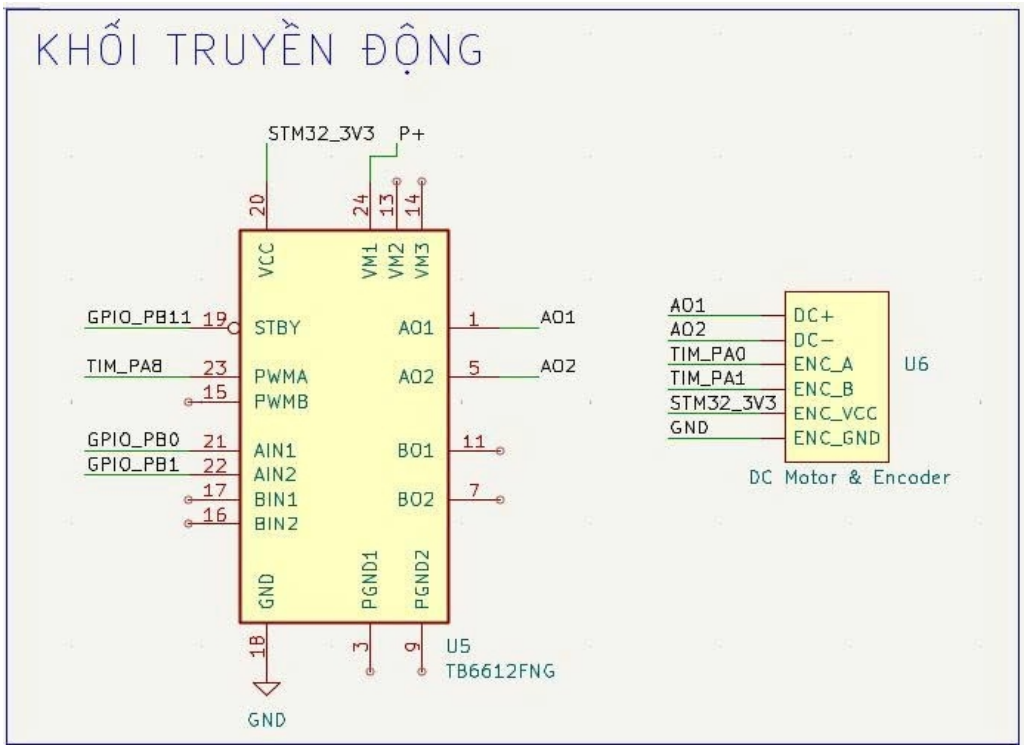
Chức năng: cách ly tín hiệu phát hiện trạng thái sạc

Lý do lựa chọn các thành phần: Cấu hình 2S được chọn để có điện áp 7,4–8,4 V, đủ cấp cho động cơ 6 V kèm biên dự phòng. Mạch BMS là bắt buộc đối với pin Li-ion vì lý do an toàn - chống quá sạc, quá xả, quá dòng và ngắn mạch. Cặp điện trở chia áp 22 kΩ/10 kΩ được chọn sao cho điện áp đưa vào ADC luôn ≤ 3,3 V (8,4 V → 2,625 V), đồng thời tổng trở 32 kΩ giữ dòng rò qua mạch chia rất nhỏ (~0,26 mA) để không hao pin; sai số ±1% giúp giảm sai số đo điện áp.

3.2.4 Khối truyền động

Bộ truyền động của mô hình là động cơ DC giảm tốc GA12-N20 6V có tích hợp encoder, đóng vai trò mô phỏng bánh xe. Động cơ được điều khiển thông qua driver cầu H TB6612FNG.

TB6612FNG nhận nguồn động lực VM từ đầu ra P+ của pack (sau BMS) và nguồn logic VCC 3,3 V từ STM32. STM32 điều khiển driver qua bốn tín hiệu: PWMA (PA8, lấy từ TIM1) quy định độ lớn tốc độ; AIN1 (PB0) và AIN2 (PB1) quy định chiều quay; STBY (PB11) cho phép hoặc ngắt driver. Quan hệ logic điều khiển chiều quay được trình bày ở Bảng 3.2.



Hình 3.10 Sơ đồ nối dây khối truyền động

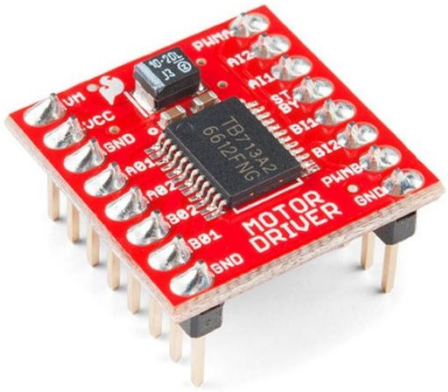
Bảng 3.2 Bảng chân lý điều khiển chiều quay motor qua TB6612FNG

STBY	AIN1	AIN2	PWMA	Trạng thái motor
0	x	x	x	Standby — driver tắt, không cấp điện cho motor
1	1	0	PWM	Quay thuận (tiến) — tốc độ theo độ rộng xung PWMA
1	0	1	PWM	Quay nghịch (lùi) — tốc độ theo độ rộng xung PWMA
1	0	0	x	Dừng — phanh ngắn mạch
1	1	1	x	Dừng — phanh ngắn mạch

Encoder quadrature gắn trên trục động cơ phát ra hai kênh A và B lệch pha nhau 90°, đưa về hai chân PA0 và PA1. STM32 đọc encoder bằng TIM2 ở chế độ Encoder Interface, đếm cả bốn cạnh của hai kênh (độ phân giải ×4) cho độ phân giải gấp bốn lần số xung cơ bản. Bộ đếm 16-bit của TIM2 tự tăng hoặc giảm theo chiều quay; sau mỗi chu kỳ, firmware lấy hiệu số đếm để suy ra tốc độ và quãng đường(công thức quy đổi trình bày ở mục 3.4).

Danh sách thành phần dùng trong khối truyền động:

1. Driver motor TB6612FNG



Hình 3.11 Driver motor TB6612FNG

Thông số kỹ thuật

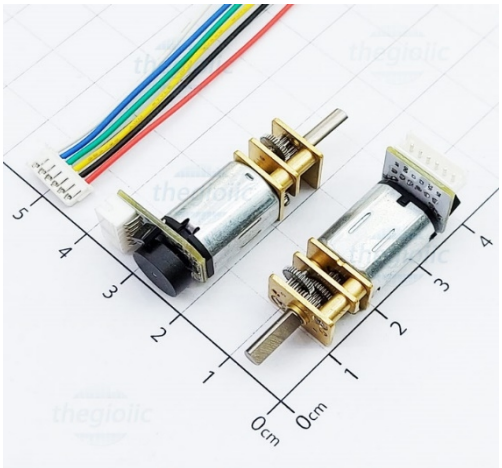
Điện áp logic: 2,7–5,5 V

Điện áp cấp động cơ (VM): tối đa 15 V

Dòng ngõ ra liên tục: 1,2 A/kênh

Hỗ trợ: điều khiển 2 cầu H (động cơ DC hoặc động cơ bước)

2. GA12-N20 động cơ giảm tốc có encoder 6VDC, 100 rpm



Hình 3.12 GA12-N20 động cơ giảm tốc có encoder 6VDC, 100 rpm

Thông số kỹ thuật

Loại: Động cơ DC giảm tốc tích hợp encoder quadrature

Điện áp định mức: 6 V

Encoder: 2 kênh A/B (đọc tốc độ và chiều quay)

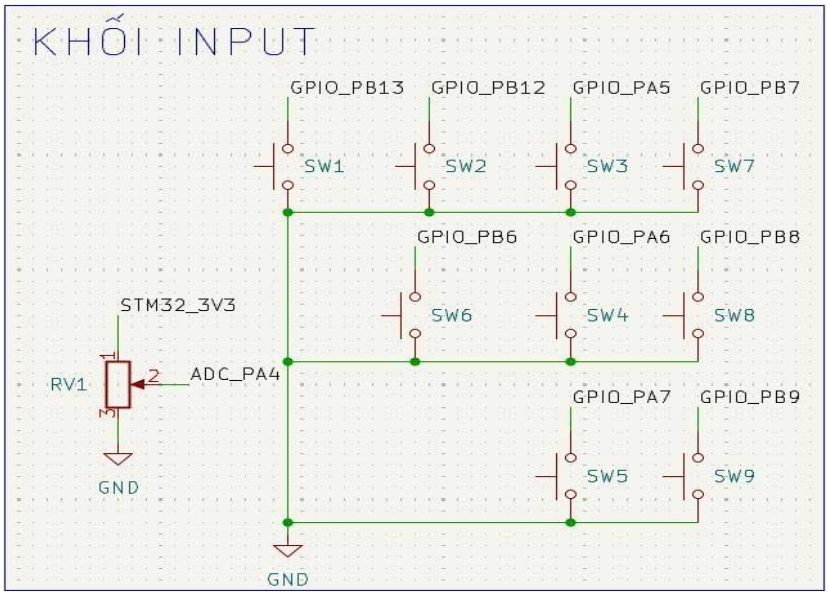
Voltage		No Load		Load Torque			Stall	Reducer	Size	Hall
Workable	Rated	Speed	Current	Speed	Current	Torque	Torque	Ratio	Length	feedback
Range	Volt.V	rpm	MA	rpm	MA	Kg.cm	Kg.cm	1:00	mm	resolution
3-12V	3V	780	≤30	450	≤90	0.03	0.08	10	9	19.36
3-12V	3V	265	≤30	150	≤90	0.1	0.23	30	9	58.94
3-12V	3V	155	≤30	90	≤90	0.17	0.4	50	9	102.2
3-12V	3V	77	≤30	45	≤90	0.35	0.75	100	9	198.6
3-12V	3V	52	≤30	30	≤90	0.5	1.1	150	9	301.9
3-12V	3V	37	≤30	21	≤90	0.7	1.5	210	9	419.5
3-12V	3V	26	≤30	15	≤90	1	≤2.8	298	9	595.8
3-12V	3V	20	≤30	12	≤90	1.2		380	9	758.3
3-12V	6V	1550	≤60	900	≤170	0.07	0.16	10	9	19.36
3-12V	6V	530	≤60	300	≤170	0.2	0.45	30	9	58.94
3-12V	6V	310	≤60	180	≤170	0.35	0.8	50	9	102.2
3-12V	6V	155	≤60	90	≤170	0.7	1.5	100	9	198.6
3-12V	6V	105	≤60	60	≤170	1	2.2	150	9	301.9
3-12V	6V	75	≤60	42	≤170	1.4		210	9	419.5
3-12V	6V	52	≤60	30	≤170	2	≤2.8	298	9	595.8
3-12V	6V	41	≤60	24	≤170	2.5		380	9	758.3

Hình 3.13 Thông số kỹ thuật của GA12-N20 động cơ giảm tốc có encoder 6VDC, 100 rpm

Lý do lựa chọn các thành phần: Driver TB6612FNG được chọn thay cho L298N (một lựa chọn phổ biến khác) vì dùng MOSFET cho hiệu suất cao và sụt áp thấp (~0,5 V so với khoảng 2 V của L298N dùng transistor lưỡng cực), nhờ đó ít tỏa nhiệt và tận dụng tốt điện áp pack; kích thước nhỏ gọn và mức logic điều khiển tương thích trực tiếp 3,3 V của STM32. Động cơ GA12-N20 được chọn vì nhỏ gọn phù hợp mô hình tỉ lệ và đã tích hợp sẵn encoder, cho phép đo tốc độ và quãng đường mà không cần lắp thêm cảm biến rời.

3.2.5 Khối input

Hệ thống nhận lệnh người lái qua một tập tín hiệu số gồm: công tắc Key/Power (PB13), nút Brake (PB12), bốn vị trí cần số Gear P/R/N/D (PA5, PA6, PA7, PB6), ba chế độ lái Mode ECO/NORMAL/SPORT (PB7, PB8, PB9) và tín hiệu Charging detect (PB14) . Tất cả ngõ vào số được cấu hình kéo lên nội (pull-up) và đấu kiểu active-low: một đầu nút nối chân vi điều khiển, đầu còn lại nối GND chung; khi nhấn, chân bị kéo xuống mức logic 0. Cách đấu này tiết kiệm điện trở ngoài và chống nhiễu tốt hơn so với kiểu kéo xuống.



Hình 3.14 Sơ đồ nối dây khối input

Hai ngõ analog được đưa vào ADC1: chân ga là biến trở 10 kΩ với hai đầu nối 3,3 V và GND, chân giữa đưa vào PA4 và cho điện áp thay đổi trong dải 0–3,3 V theo độ mở ga; điện áp pin đưa vào PA2 qua mạch chia áp đã mô tả ở mục 3.2.3. Cả hai kênh được ADC1 chuyển đổi tuần tự và truyền vào bộ đệm qua DMA, giảm tải cho CPU và cho phép lấy mẫu liên tục.

Danh sách thành phần dùng trong khối input:

Bảng 3.3 Thành phần khối input

Linh kiện	Thông số kỹ thuật chính
Công tắc / nút nhấn (key, brake, gear ×4, mode ×3)	Loại cơ, tiếp điểm thường mở; đấu kiểu active-low
Biến trở 10 kΩ (chân ga)	Tuyến tính; 3 chân (hai đầu + con chạy)

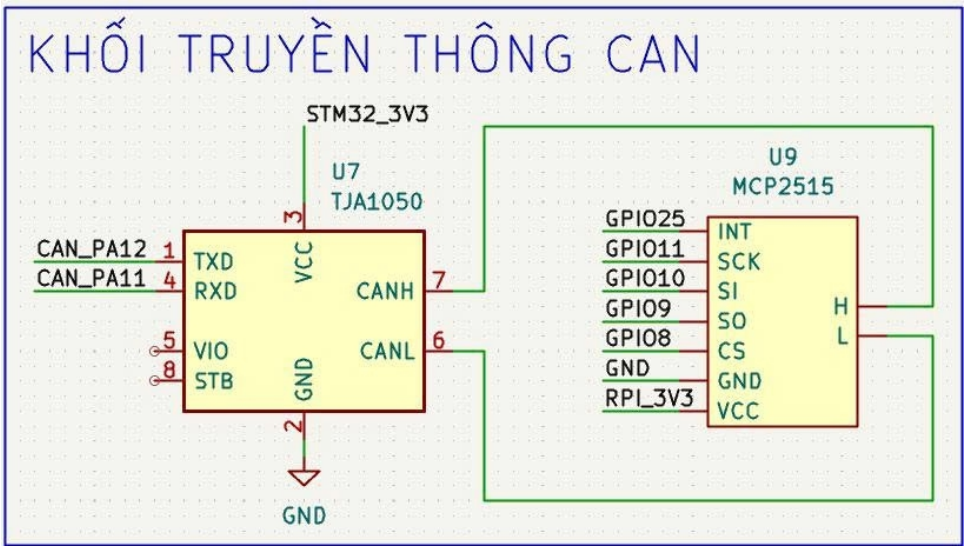
Lý do lựa chọn các thành phần: Các nút nhấn và công tắc cơ được chọn vì đơn giản, rẻ và đủ tin cậy cho mô hình ; chúng được đấu kiểu active-low để tận dụng điện trở kéo lên nội của STM32, nhờ đó không cần điện trở ngoài. Biến trở 10 kΩ cho chân ga được chọn vì cho dải điện áp 0–3,3 V tuyến tính tương thích trực tiếp với ADC, trở kháng đủ thấp để ADC lấy mẫu ổn định nhưng đủ cao để dòng tiêu thụ không đáng kể.

3.2.6 Khối truyền thông CAN

Khối CAN Bus là kênh truyền thông duy nhất giữa ECU Node và VCU. Tại mỗi đầu bus có một bộ thu phát làm nhiệm vụ chuyển tín hiệu logic của bộ điều khiển CAN thành cặp tín hiệu vi sai CANH/CANL trên đường truyền, và ngược lại.

Phía STM32, ngoại vi bxCAN tích hợp sẵn trên chip (PA11 = CAN_RX, PA12 = CAN_TX) kết nối tới transceiver TJA1050; TJA1050 đưa ra cặp dây CANH/CANL của bus. Phía Raspberry Pi - vốn không có ngoại vi CAN tích hợp - một module MCP2515 đảm nhiệm vai trò bộ điều khiển CAN, giao tiếp với Pi qua bus SPI và đã tích hợp sẵn transceiver để nối ra bus (chi tiết kết nối ở mục 3.2.7).

Theo chuẩn ISO 11898-2, bus CAN phải có hai điện trở kết cuối 120 Ω đặt tại hai đầu xa nhất của bus — một đầu gần STM32, một đầu gần Pi — nhằm hấp thụ phản xạ tín hiệu và bảo đảm tổng trở đặc tính của đường truyền. Thiếu hoặc sai giá trị điện trở kết cuối là một trong những nguyên nhân phổ biến nhất gây lỗi truyền CAN.



Hình 3.15 Sơ đồ nối dây khối truyền thông Can

Ngoài hai dây tín hiệu CANH/CANL, toàn bộ các khối phải nối chung mass: GND của STM32, TB6612, BMS (P-), mạch chia áp, TJA1050, MCP2515 và Raspberry Pi. GND chung tạo ra một điện áp tham chiếu thống nhất cho tín hiệu vi sai và cho phép ADC; nếu các khối có mass trôi khác nhau, CAN dễ lỗi bit và

ADC đọc sai giá trị. Tốc độ bus được chọn là 500 kbit/s; thiết kế khung dữ liệu và định thời bit sẽ được trình bày ở mục 3.3.

Danh sách thành phần dùng trong khối truyền thông CAN:

1. Mạch truyền thông Can Bus TJA1050



Hình 3.16 Mạch truyền thông Can Bus TJA1050

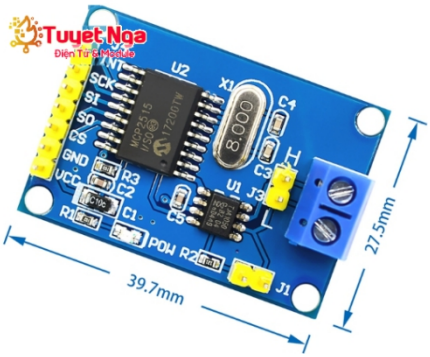
Thông số kỹ thuật

Chức năng: CAN transceiver (chuyển mức logic ↔ CANH/CANL)

Điện áp hoạt động: 5 V

Hỗ trợ chuẩn: ISO 11898

2. Mạch chuyển giao tiếp Can Bus MCP2515



Hình 3.17 Mạch chuyển giao tiếp Can Bus MCP2515

Thông số kỹ thuật

Chức năng: Bộ điều khiển CAN giao tiếp SPI

Điện áp hoạt động: 5 V

Hỗ trợ chuẩn: CAN 2.0B

Tốc độ CAN tối đa: 1 Mb/s

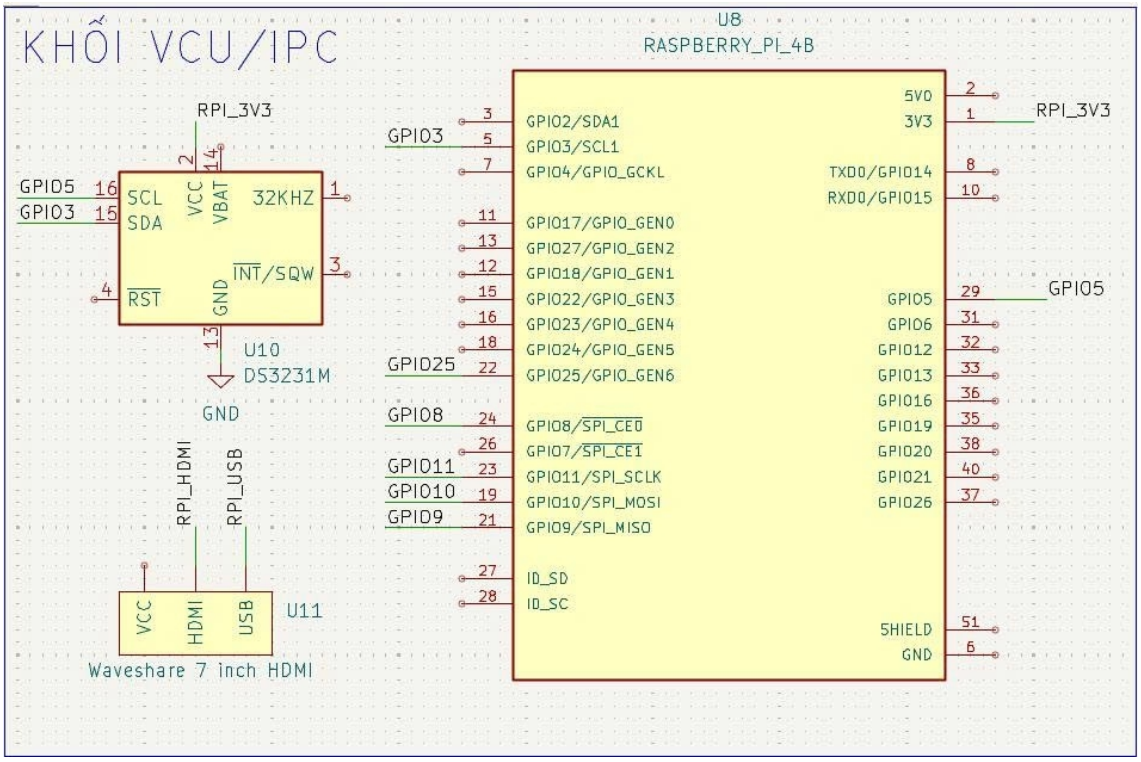
Lý do lựa chọn các thành phần: Phía STM32 đã có sẵn bộ điều khiển bxCAN nhưng vẫn cần một transceiver vật lý để ghép vào bus; TJA1050 được chọn vì phổ biến, rẻ và đúng chuẩn ISO 11898 . Phía Raspberry Pi không có ngoại vi CAN nên dùng module MCP2515 giao tiếp qua SPI — đây là giải pháp chuẩn cho Pi; module thương mại đã tích hợp sẵn transceiver và thạch anh nên giảm công đấu nối. Hai điện trở kết cuối 120 Ω được lắp theo chuẩn ISO 11898-2 để chống phản xạ tín hiệu.

3.2.7 Khối VCU/IPC

Tầng điều khiển trung tâm và giao diện người dùng chạy trên máy tính nhúng Raspberry Pi 4B (4 GB RAM). Vì Pi không có ngoại vi CAN, module MCP2515 được ghép qua giao tiếp SPI0 với ánh xạ chân: GPIO10 (MOSI) → SI, GPIO9 (MISO) → SO, GPIO11 (SCK) → SCK, GPIO8 (CE0) → CS và GPIO25 → INT; Pi được cấp nguồn 3,3 V và nối GND chung với toàn hệ thống. Chân INT báo cho Pi mỗi khi có khung CAN mới về, cho phép xử lý theo ngắt thay vì hỏi vòng liên tục điều quan trọng vì Linux trên Pi không bảo đảm thời gian thực cứng.

Giao diện cụm đồng hồ số được hiển thị trên màn hình Waveshare 7 inch kết nối qua cổng HDMI; nếu sử dụng tính năng cảm ứng thì bổ sung cáp USB. Ở giai đoạn bring-up, Raspberry Pi và màn hình dùng nguồn riêng 5 V để tránh sụt áp và nhiễu, trong khi pack pin 2S chỉ cấp cho khối truyền động và được trích đo để hiển thị.

Ngoài ra, hệ thống dùng module RTC DS3231 ghép qua bus I²C của Raspberry Pi (GPIO3 = SDA, GPIO5 = SCL), cấp nguồn 3,3 V và GND chung. DS3231 có pin backup CR2032 nên giữ được thời gian thực ngay cả khi mất nguồn chính — bảo đảm đồng hồ trên cụm hiển thị (góc TIME của giao diện) luôn đúng giờ.



Hình 3.18 Sơ đồ nối dây khối VCU/IPC

Danh sách thành phần dùng trong khối truyền thông CAN:

1. Raspberry Pi 4 Model B 4GB



Hình 3.19 Raspberry Pi 4 Model B 4GB

Thông số kỹ thuật

CPU: Broadcom BCM2711, Quad-core ARM Cortex-A72 1.5 GHz

RAM: 4 GB LPDDR4

Giao tiếp sử dụng: SPI, I²C, HDMI, GPIO 40 chân

Nguồn cấp: 5 VDC

2. Màn hình Waveshare 7 inch HDMI Capacitive Touch Screen LCD (H)



Hình 3.20 Màn hình Waveshare 7 inch HDMI Capacitive Touch Screen LCD (H)

Thông số kỹ thuật

Kích thước: 7 inch

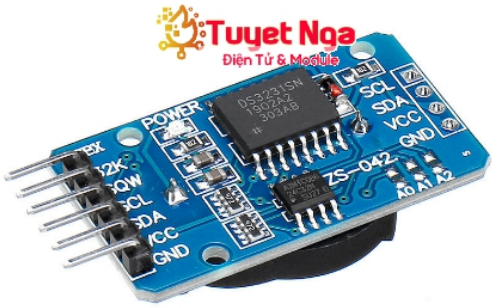
Độ phân giải: 1024 × 600 pixels

Giao tiếp: HDMI (hiển thị), USB (cảm ứng)

Nguồn cấp: 5 VDC

Hỗ trợ cảm ứng điện dung

3. Mạch thời gian thực RTC DS3231



Hình 3.21 Mạch thời gian thực RTC DS3231

Thông số kỹ thuật

Điện áp hoạt động: 3,3–5 VDC

Giao tiếp: I^2C

Chức năng: lưu và duy trì thời gian thực (RTC) khi mất nguồn

Lý do lựa chọn các thành phần: Raspberry Pi 4B được chọn vì đủ năng lực để chạy giao diện Qt/QML mượt mà (cập nhật ~20 fps) và xử lý logic VCU đa luồng; có sẵn GPIO SPI để ghép module MCP2515 và cổng HDMI để xuất màn hình; nền tảng Linux hỗ trợ sẵn SocketCAN giúp lập trình CAN thuận tiện. Màn hình Waveshare 7 inch độ phân giải 1024×600 cảm ứng được chọn vì vừa khít bố cục cụm đồng hồ và cắm HDMI/USB là dùng ngay.

3.2.8 Pin mapping tổng hợp và cấu hình ngoại vi

Bảng 3.4 tổng hợp toàn bộ ánh xạ chân của STM32F103C8T6 theo từng nhóm chức năng. Trên cơ sở đó, Bảng 3.5 trình bày cấu hình các ngoại vi tương ứng trên công cụ STM32CubeMX — làm cơ sở để sinh mã khởi tạo HAL cho phần mềm STM32 ở mục 3.4.

Bảng 3.4 Pin mapping tổng hợp khối STM32F103C8T6

Chân	Chức năng	Kết nối ngoài	Cấu hình
PA0	Encoder A	Kênh A encoder GA12-N20	TIM2_CH1
PA1	Encoder B	Kênh B encoder GA12-N20	TIM2_CH2
PA2	Điện áp pin (ADC)	Điểm giữa mạch chia áp	ADC1_IN2
PA4	Chân ga (ADC)	Chân giữa biến trở 10kΩ	ADC1_IN4
PA5	Gear P	Công tắc cần số	GPIO In, pull-up
PA6	Gear R	Công tắc cần số	GPIO In, pull-up
PA7	Gear N	Công tắc cần số	GPIO In, pull-up
PA8	PWM motor	PWMA của TB6612	TIM1_CH1
PA9	UART TX (debug)	Bộ chuyển USB–UART	USART1_TX
PA10	UART RX (debug)	Bộ chuyển USB–UART	USART1_RX
PA11	CAN_RX	RXD của TJA1050	CAN_RX
PA12	CAN_TX	TXD của TJA1050	CAN_TX
PB0	Motor AIN1	AIN1 của TB6612	GPIO Out
PB1	Motor AIN2	AIN2 của TB6612	GPIO Out
PB6	Gear D	Công tắc cần số	GPIO In, pull-up
PB7	Mode ECO	Công tắc chế độ lái	GPIO In, pull-up
PB8	Mode NORMAL	Công tắc chế độ lái	GPIO In, pull-up

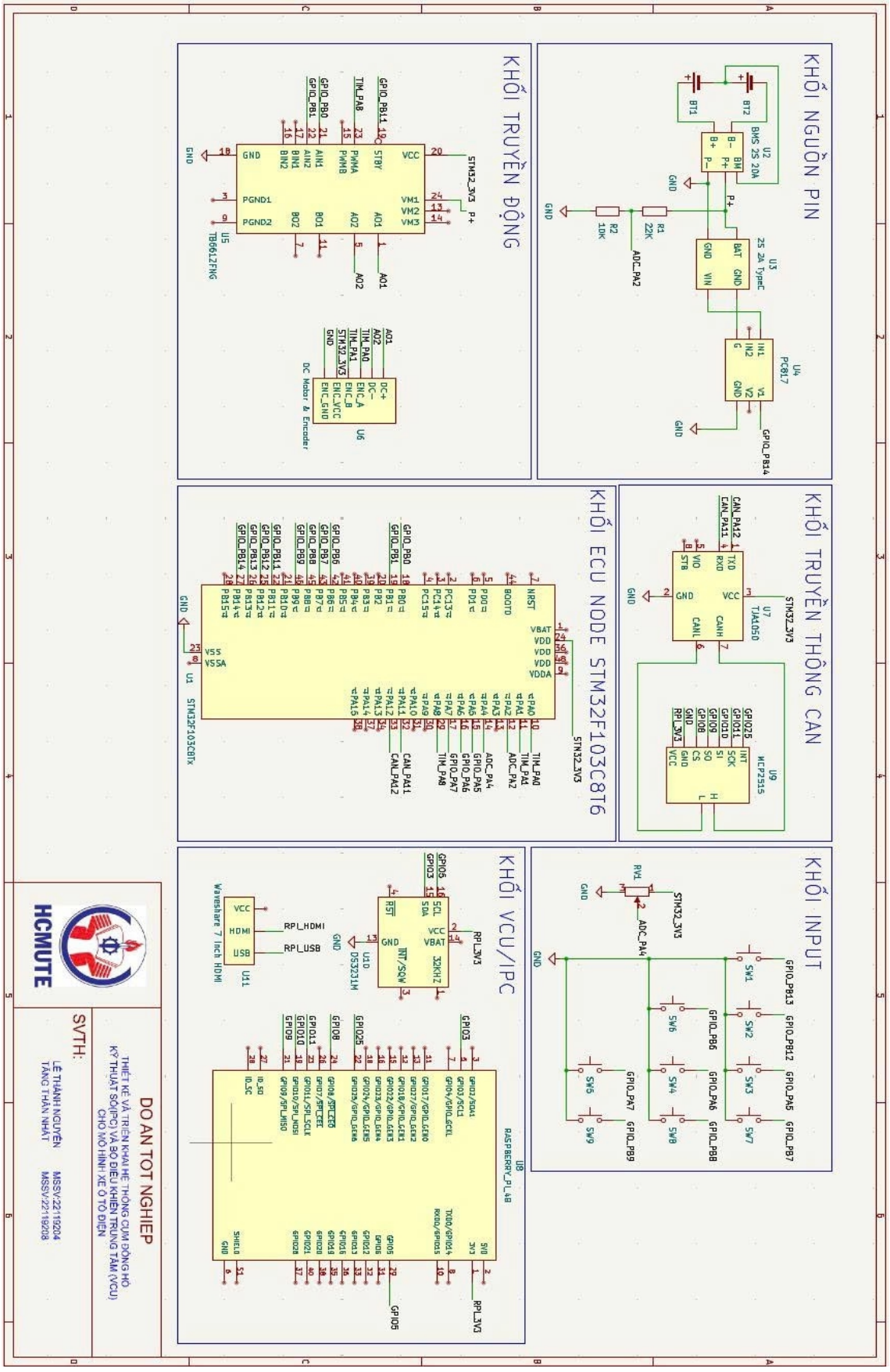
Chân	Chức năng	Kết nối ngoài	Cấu hình
PB9	Mode SPORT	Công tắc chế độ lái	GPIO In, pull-up
PB11	Motor STBY	STBY của TB6612	GPIO Out
PB12	Brake	Nút phanh	GPIO In, pull-up
PB13	Key / Power	Công tắc nguồn	GPIO In, pull-up
PB14	Charging detect	Tín hiệu detect sạc	GPIO In, pull-up

Bảng 3.5 Cấu hình ngoại vi STM32 trên STM32CubeMX

Ngoại vi	Cấu hình chính	Chân
Clock	HSE 8 MHz → PLL ×9 → SYSCLK 72 MHz; APB1 = 36 MHz	—
TIM1 (PWM)	PWM Generation CH1; ARR = 999 → ~1 kHz, 1000 mức	PA8
TIM2 (Encoder)	Encoder Interface Mode (×4), bộ đếm 16-bit	PA0, PA1
ADC1	2 kênh quét (IN4 ga, IN2 pin), 12-bit, truyền qua DMA	PA4, PA2
bxCAN	CAN 2.0A, 500 kbit/s, Normal mode, lọc nhận ID 0x200	PA11, PA12
USART1	115200 baud, 8N1 — debug log	PA9, PA10
GPIO Out	Push-pull: AIN1, AIN2, STBY	PB0, PB1, PB11
GPIO In	Pull-up, active-low: key, brake, gear, mode, detect, fault	PA5–PA7, PB6–PB9, PB12–PB15
DMA1	ADC1 → bộ đệm circular, giảm tải CPU	Nội bộ

Hình 3.22 thể hiện: đầy đủ sơ đồ nguyên lý của các khối trong kiến trúc phần cứng, đầy đủ các thành phần của từng khối và cách nối dây chi tiết của từng thành phần đó.

Như vậy, mục này đã hoàn tất thiết kế phần cứng của hệ thống ở mức khối chức năng và pin mapping. Trên nền tảng phần cứng đó, mục 3.3 tiếp theo sẽ trình bày thiết kế giao thức truyền thông CAN.



Hình 3.22 Sơ đồ nguyên lý tổng thể cho phần cứng

3.3 THIẾT KẾ GIAO THỨC TRUYỀN THÔNG CAN

Giao thức CAN là lớp giao tiếp duy nhất giữa ECU Node (STM32) và VCU (Raspberry Pi), đồng thời là "hợp đồng giao diện" mà cả phần mềm STM32 (mục 3.4) lẫn phần mềm VCU (mục 3.5) phải tuân thủ. Trên nền phần cứng CAN đã trình bày ở mục 3.2.6, mục này định nghĩa tham số bus, bảng thông điệp CAN, bố cục từng byte của mỗi khung và các quy ước mã hóa dữ liệu.

3.3.1 Tham số bus và nguyên tắc thiết kế

Bus được cấu hình ở tốc độ 500 kbit/s với định danh chuẩn 11-bit (CAN 2.0A). Tốc độ 500 kbit/s là mức rất phổ biến trong xe, đủ nhanh so với lưu lượng nhẹ của hệ thống đồng thời đủ biên độ chống nhiễu trên đoạn bus ngắn của mô hình; định danh 11-bit cung cấp đủ không gian cho số lượng thông điệp ít và giúp đơn giản hóa bộ lọc nhận. Tham số định thời bit được cấu hình để đạt 500 kbit/s phải đồng nhất giữa bxCAN của STM32 (clock APB1 = 36 MHz) với MCP2515 của Pi (thạch anh 16 MHz) — nếu hai node lệch bit timing, bus sẽ báo lỗi và không truyền được.

Việc thiết kế tập thông điệp tuân theo năm nguyên tắc. Thứ nhất, mã hóa bằng số nguyên và tránh số thực trên bus: các đại lượng thực được nhân hệ số rồi truyền dưới dạng số nguyên (tốc độ nhân 100, điện áp tính theo mV), tránh sai khác biểu diễn dấu phẩy động giữa hai nền tảng. Thứ hai, mỗi khung dùng cấu trúc đóng gói chặt với độ dài cố định 4 byte, ánh xạ trực tiếp giữa byte và trường dữ liệu bằng memcpy thay vì tuần tự hóa thủ công. Thứ ba, cả hai node đều dùng little-endian nên không cần đảo thứ tự byte. Thứ tư, định nghĩa kiểu dữ liệu được dùng chung: tệp vehicle_types.h (phía STM32) và vehicle_types_pi.h (phía Pi) là hai bản phản chiếu của cùng một đặc tả. Thứ năm, định danh được phân vùng theo chức năng và ưu tiên: dải 0x1xx dành cho dữ liệu trạng thái từ STM32, dải 0x2xx dành cho lệnh từ VCU; trong nhóm telemetry, dữ liệu thời gian thực tốc độ (0x100) có mức ưu tiên phân xử cao nhất.

3.3.2 Bảng thông điệp CAN

Toàn hệ thống sử dụng năm thông điệp như tổng hợp ở Bảng 3.6: bốn thông điệp telemetry theo hướng STM32 → VCU và một thông điệp lệnh theo hướng VCU → STM32. Mỗi thông điệp có chu kỳ gửi riêng tùy theo mức độ quan trọng và tốc độ biến thiên của dữ liệu — tốc độ/ga và lệnh điều khiển ở 20 ms vì ảnh hưởng trực tiếp đến cảm giác lái và độ mượt hiển thị, input và odometer ở 40 ms, còn điện áp pin ở 100 ms vì biến thiên chậm.

Bảng 3.6 Bảng thông điệp CAN của hệ thống

CAN ID	Tên thông điệp	Hướng	Chu kỳ	DLC	Mô tả
0x100	Speed & Accel	STM32 → VCU	20 ms	4	Tốc độ xe, độ mở ga, mức PWM motor hiện tại
0x101	Battery	STM32 → VCU	100 ms	4	Điện áp pack, SoC, cờ cảnh báo pin
0x102	Hardware Inputs	STM32 → VCU	40 ms	4	Cần số, chế độ lái, các cờ input số
0x103	Odometer	STM32 → VCU	40 ms	4	Quãng đường tích lũy
0x200	VCU Command	VCU → STM32	20 ms	4	Cờ điều khiển, PWM mục tiêu, trạng thái xe, cờ lỗi

3.3.3 Đặc tả bố cục byte của từng khung

Mỗi khung có độ dài cố định 4 byte. Các trường nhiều byte (uint16) được sắp xếp theo little-endian. Bố cục chi tiết của năm khung được trình bày từ Bảng 3.7 đến Bảng 3.11.

Bảng 3.7 Bố cục byte khung 0x100 — Speed & Accel

Byte	Trường	Kiểu	Ý nghĩa
0–1	speed_x100	uint16 (LE)	Tốc độ = giá trị / 100 (km/h); độ phân giải 0,01 km/h
2	accel_pct	uint8	Độ mở chân ga, 0–100%
3	motor_pwm_pct	uint8	Mức PWM đang cấp cho motor, 0–100%

Bảng 3.8 Bố cục byte khung 0x101 — Battery

Byte	Trường	Kiểu	Ý nghĩa
0–1	batt_mv	uint16 (LE)	Điện áp pack pin (mV)
2	soc_pct	uint8	Dung lượng còn lại ước lượng, 0–100%
3	batt_flags	uint8	bit0 = cảnh báo pin yếu; bit1 = lỗi cảm biến điện áp

Bảng 3.9 Bố cục byte khung 0x102 — Hardware Inputs

Byte	Trường	Kiểu	Ý nghĩa
0	gear	uint8	Vị trí cần số (mã hóa ở Bảng 3.11)
1	drive_mode	uint8	Chế độ lái (mã hóa ở Bảng 3.11)
2	input_flags	uint8	bit0 = key; bit1 = brake; bit2 = charging; bit3 = fault_inject
3	reserved	uint8	Dự phòng (= 0)

Bảng 3.10 Bố cục byte khung 0x103 — Odometer

Byte	Trường	Kiểu	Ý nghĩa
0–1	odometer_m	uint16 (LE)	Quãng đường tích lũy (m); VCU xử lý tràn 16-bit để cộng dồn
2–3	reserved	-	Dự phòng (= 0)

Bảng 3.11 Bố cục byte khung 0x200 — VCU Command

Byte	Trường	Kiểu	Ý nghĩa
0	ctrl_flags	uint8	bit0 = motor_enable; bit1 = direction_fwd (1 = tiến, 0 = lùi)
1	pwm_target_pct	uint8	Mức PWM mục tiêu VCU yêu cầu, 0–100%
2	vehicle_state	uint8	Trạng thái xe hiện hành (mã hóa ở Bảng 3.11)
3	fault_flags	uint8	Bitmask lỗi đang hoạt động (Bảng 3.12)

3.3.4 Quy ước mã hóa trường liệt kê và cờ lỗi

Các trường liệt kê (cần số, chế độ lái, trạng thái xe) được mã hóa bằng giá trị số nguyên cố định, đồng nhất ở cả hai node, như tổng hợp ở Bảng 3.12. Riêng trường fault_flags là một bitmask: mỗi bit ứng với một loại lỗi và có thể OR nhiều lỗi cùng lúc, cho phép một khung 0x200 mang đồng thời nhiều tình trạng lỗi đang hoạt động (Bảng 3.13).

Bảng 3.12 Mã hóa các trường liệt kê

Trường	Giá trị	Ý nghĩa
GearPosition	0 / 1 / 2 / 3	P (Park) / R (Reverse) / N (Neutral) / D (Drive)
DriveMode	0 / 1 / 2	ECO / NORMAL / SPORT
VehicleState	0 / 1 / 2	OFF / STANDBY / READY
	3 / 4 / 5	DRIVING / CHARGING / FAULT

Bảng 3.13 Mã hóa bitmask trường fault_flags

Giá trị bit	Tên cờ	Ý nghĩa
0x00	NONE	Không có lỗi
0x01	OVERCURRENT	Quá dòng
0x02	OVERVOLTAGE	Quá áp pack pin
0x04	UNDERVOLTAGE	Sụt áp / pin yếu nghiêm trọng
0x08	OVERTEMP	Quá nhiệt
0x10	MOTOR_STALL	Kẹt / đứng motor
0x20	CAN_TIMEOUT	Mất khung CAN (watchdog)
0x40	INJECTED	Lỗi kích thủ công khi kiểm thử

Toàn bộ các giá trị mã hóa, vị trí bit và kích thước trường trong các bảng trên được khai báo một lần trong tệp định nghĩa dùng chung và phản chiếu sang cả hai phía. Cơ chế static_assert kiểm tra kích thước mỗi cấu trúc khung đúng 4 byte tại thời điểm biên dịch, bảo đảm hai node luôn hiểu khung theo cùng một cách và loại trừ lỗi lệch định dạng — một lớp lỗi rất khó truy vết nếu chỉ phát hiện lúc chạy.

3.3.5 Lập lịch truyền và lọc nhận

Ở phía phát, các thông điệp telemetry được STM32 gửi định kỳ theo một bộ đếm phân tần trong tác vụ truyền CAN: khung 0x100 mỗi 20 ms, khung 0x102 và 0x103 mỗi 40 ms, khung 0x101 mỗi 100 ms; VCU phát khung lệnh 0x200 mỗi 20 ms. Việc gộp nhiều chu kỳ vào một bộ đếm chung giúp các thông điệp không phát cùng lúc gây dồn bus.

Ở phía nhận, bộ lọc được đặt ngay tại tầng phần cứng: STM32 cấu hình bộ lọc chỉ chấp nhận khung 0x200, bỏ qua mọi khung khác kể cả khung do chính nó phát; phía VCU chỉ xử lý các khung 0x100–0x103. Lọc ở phần cứng giúp CPU không phải kiểm tra định danh bằng phần mềm cho những khung không liên quan, giảm tải xử lý. Chi tiết thực hiện của lịch gửi và quy trình nhận theo ngắt được trình bày ở mục 3.4.2.

Bộ giao thức định nghĩa trong mục này là nền tảng để hiện thực hai phía phần mềm: phần mềm STM32 (mục 3.4) đóng gói và giải mã telemetry, đồng thời thực thi lệnh nhận được; phần mềm VCU (mục 3.5) giải mã telemetry, chạy state machine và phát khung lệnh 0x200.

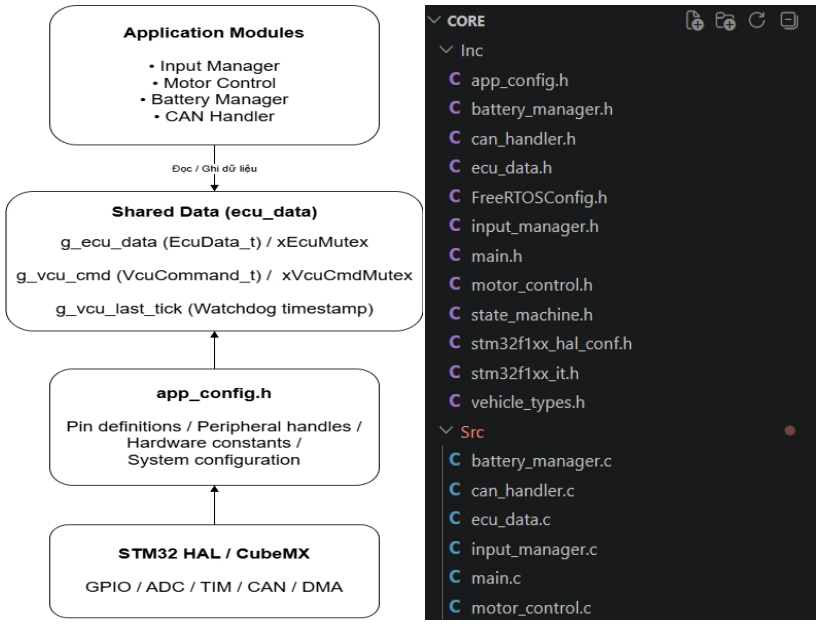
3.4 THIẾT KẾ VÀ THI CÔNG PHẦN MỀM STM32 (FREERTOS)

Phần mềm chạy trên vi điều khiển STM32F103C8T6 dưới hệ điều hành thời gian thực FreeRTOS (thông qua CMSIS-RTOS v2 wrapper layer), thực hiện vai trò ECU Node: đọc cảm biến và input, điều khiển động cơ, đo điện áp pin và truyền nhận khung CAN. Toàn bộ phần mềm tuân thủ nguyên tắc nguồn quyết định duy nhất đã nêu ở mục 3.1.3 - STM32 chỉ thực thi lệnh, mọi quyết định điều khiển thuộc về VCU - và được viết theo chuẩn MISRA C (tập quy tắc cơ bản). Mục này trình bày kiến trúc module, mô hình đa nhiệm, cơ chế đồng bộ dữ liệu thời gian thực và thi công chi tiết từng module.

3.4.1 Kiến trúc phần mềm phân chia module

Phần mềm được tổ chức phân lớp rõ ràng. Lớp dưới cùng là thư viện HAL do STM32CubeMX sinh ra, trừu tượng hóa thanh ghi ngoại vi. Toàn bộ định nghĩa phần cứng: số chân, handle ngoại vi, hằng số được gom vào một tệp cấu hình tập

trung app_config.h, nhờ đó khi thay đổi phần cứng chỉ cần sửa một nơi. Bên trên là bốn module chức năng (Input Manager, Motor Control, Battery Manager, CAN Handler), mỗi module gồm một hàm khởi tạo và một tác vụ FreeRTOS. Tất cả module trao đổi dữ liệu qua một kho dữ liệu chia sẻ là ecu_data, gồm biến g_ecu_data (kiểu EcuData_t, bảo vệ bởi xEcuMutex), g_vcu_cmd (kiểu VcuCommand_t, bảo vệ bởi xVcuCmdMutex) và mốc thời gian g_vcu_last_tick phục vụ watchdog.



Hình 3.23 Sơ đồ kiến trúc phân lớp phần mềm STM32

Về thi công, trình tự khởi động được tổ chức theo mô hình chuẩn của ứng dụng CubeMX kết hợp FreeRTOS. Hàm main() lần lượt gọi HAL_Init, các hàm khởi tạo ngoại vi (GPIO, DMA, ADC1, USART1, CAN, TIM1, TIM2), khởi động CAN bằng HAL_CAN_Start, sau đó khởi tạo nhân RTOS và tạo một tác vụ khởi động trước khi gọi osKernelStart. Khi nhân chạy, defaultTask thực thi hàm App_Init: khởi tạo lần lượt ecu_data, các module và tạo năm tác vụ ứng dụng, rồi tự kết thúc bằng osThreadExit để nhường toàn bộ tài nguyên cho các tác vụ chức năng.

3.4.2 Thiết kế tác vụ

Hệ thống dùng năm tác vụ ứng dụng cùng một tác vụ khởi động, chạy trên bộ lập lịch ưu tiên có chiếm quyền với nhịp tick 1 ms và vùng heap 10 KB. Việc

phân bổ mức ưu tiên dựa trên tính chất cần thiết của từng tác vụ: điều khiển động cơ là tác vụ thời gian thực quan trọng nhất nên được đặt ưu tiên cao nhất; đọc input và nhận lệnh CAN ở mức trung bình; đo pin và gửi CAN ở mức thấp hơn vì biến thiên chậm và không tới hạn. Các tác vụ chu kỳ giữ nhịp bằng osDelayUntil dựa trên mốc thời gian tuyệt đối nên đáp ứng thời gian ổn định, còn tác vụ nhận CAN hoạt động theo sự kiện chờ RTOS Queue có frame. Bảng 3.14 tổng hợp thiết kế các tác vụ.

Bảng 3.14 Thiết kế các tác vụ firmware STM32

Tác vụ	Chu kỳ	Ưu tiên	Stack	Chức năng chính
Task_MotorControl	5 ms	AboveNormal (4)	1 kB	Thực thi lệnh PWM, watchdog VCU, đọc encoder tính tốc độ và quãng đường
Task_InputScan	10 ms	Normal (3)	1 kB	Đọc ADC (ga, pin) và digital, debounce, latching gear/mode
Task_CANTransmit	20 ms	BelowNormal (2)	1.5 kB	Đóng gói và gửi telemetry 0x100–0x103 theo lịch phân tần
Task_CANReceive	RTOS Queue có Frame	Normal (3)	1 kB	Nhận và giải mã lệnh 0x200, cập nhật watchdog timestamp
Task_BatteryManager	100 ms	BelowNormal (2)	1 kB	Đo điện áp (đường bao), lọc EMA, ước lượng SoC
defaultTask	Một lần	Normal (3)	2 kB	Chạy App_Init (khởi tạo module và tạo task) rồi tự thoát

3.4.3 Cơ chế đồng bộ dữ liệu thời gian thực

Vì nhiều tác vụ cùng truy cập kho dữ liệu chia sẻ, phần mềm dùng hai mutex để bảo đảm nhất quán: xEcuMutex bảo vệ g_ecu_data và xVcuCmdMutex bảo vệ g_vcu_cmd cùng mốc g_vcu_last_tick. Nguyên tắc xuyên suốt là snapshot trong

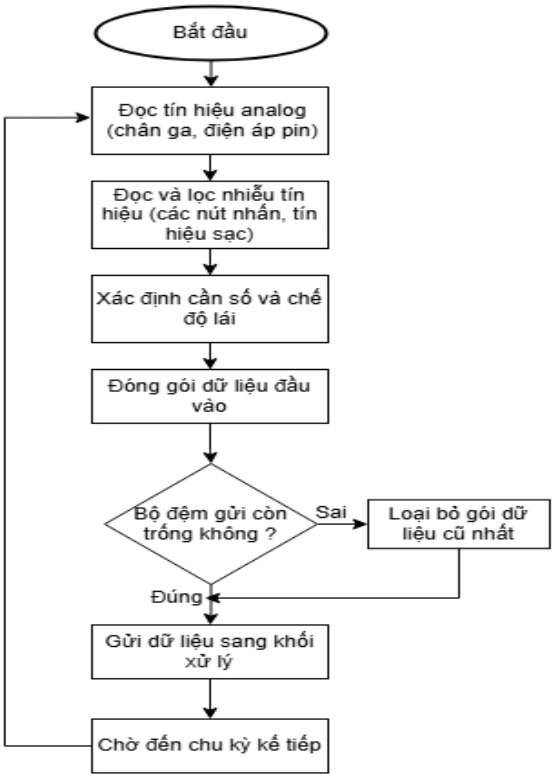
mutex: mỗi tác vụ chỉ giữ mutex trong thời gian cực ngắn để sao chép dữ liệu ra biến cục bộ rồi nhả ngay. Chẳng hạn tác vụ gửi CAN sao chép toàn bộ EcuData một lần rồi gửi các khung từ bản sao đó; tác vụ điều khiển động cơ ghi đồng thời PWM, tốc độ và quãng đường trong một lần giữ mutex duy nhất để phía CAN luôn đọc được một snapshot nhất quán.

Đối với dữ liệu đến từ ngắt, phần mềm áp dụng nguyên tắc đẩy xử lý ra khỏi ISR: trình phục vụ ngắt nhận CAN chỉ đọc khung và đẩy vào một hàng đợi với thời gian chờ bằng 0 (không chặn, không in log trong ngắt), còn việc giải mã được tác vụ Task_CANReceive đảm nhiệm. Cách này giữ thời gian ở trong ngắt ở mức tối thiểu và tránh mọi thao tác chặn bên trong ngữ cảnh ngắt. Hệ thống dùng hai hàng đợi: hàng đợi nhận CAN (sức chứa 8 khung) và hàng đợi input (sức chứa 2, ghi đè khi đầy). Cuối cùng, cơ chế watchdog dữ liệu được hiện thực bằng mốc g_vcu_last_tick: mốc này được cập nhật mỗi khi nhận khung lệnh 0x200, và tác vụ điều khiển động cơ căn cứ vào nó để phát hiện mất liên lạc với VCU.

3.4.4 Module Input Manager

Tác vụ đọc input chạy mỗi 10 ms. Hai kênh analog (chân ga ở Rank 1, điện áp pin ở Rank 2) được ADC1 chuyển đổi liên tục và nạp vào bộ đệm qua DMA vòng, nên tác vụ chỉ việc đọc giá trị mới nhất từ bộ đệm mà không phải chờ chuyển đổi; độ mở ga được quy đổi theo tỉ lệ phần trăm từ giá trị ADC. Các ngõ số được lọc rung phím bằng phần mềm: một trạng thái chỉ được xác nhận sau ba lần đọc liên tiếp giống nhau, tương đương thời gian chống dội khoảng 30 ms.

Hệ thống phân biệt hai loại input. Các tín hiệu tức thời: key, brake, charging dùng để phản ánh trạng thái giá trị sau lọc rung. Trong khi đó cần số và chế độ lái dùng logic chốt kết hợp phát hiện sườn lên: mỗi lần nhấn mới chuyển sang chế độ tương ứng và giữ nguyên khi nhả nút, với thứ tự ưu tiên khi nhấn đồng thời là D > R > P > N cho cần số và SPORT > ECO > NORMAL cho chế độ lái. Sau khi xử lý, tác vụ cập nhật toàn bộ input vào g_ecu_data trong mutex, đồng thời đăng lên hàng đợi input (ghi đè khi đầy) để phục vụ cho việc kiểm tra giá trị.



Hình 3.24 Lưu đồ giải thuật cho tác vụ Input Manager

3.4.5 Module Motor Control

Tác vụ điều khiển động cơ chạy mỗi 5 ms ở mức ưu tiên cao nhất, mỗi chu kỳ gồm năm bước: sao chép lệnh VCU và mốc thời gian trong mutex; kiểm tra watchdog; tính tốc độ từ encoder; thực thi PWM; và ghi kết quả vào kho dữ liệu. Cơ chế watchdog là lớp an toàn duy nhất nằm trên STM32: động cơ bị coi là mất quyền chạy nếu g_vcu_last_tick vẫn bằng 0 (VCU chưa từng gửi lệnh kể từ khi khởi động - an toàn theo mặc định) hoặc nếu khoảng cách thời gian từ lệnh gần nhất vượt quá ngưỡng 2000 ms. Khi watchdog kích hoạt, động cơ bị dừng ngay; ngược lại, nếu lệnh cho phép thì động cơ chạy với mức PWM quy đổi từ phần trăm mục tiêu và chiều quay theo lệnh. Thao tác dừng là đặt cả hai chân điều hướng về 0 và PWM = 0.

Tốc độ và quãng đường được suy ra từ bộ đếm encoder của TIM2. Sau khi đọc hiệu số đếm và xử lý hiện tượng tràn của bộ đếm 16-bit, firmware quy đổi qua các bước sau, với số xung trên một vòng bánh PULSE/REV = PPR × tỉ số truyền × 4 = 7 × 150 × 4 = 4200 xung và chu vi bánh C_wheel = 188 mm:

$$rev/s = \left(|\Delta count| \,/\, (PULSE/REV)\right) \times \left(\frac{1000}{T_task}\right) \; [vòng/s] \tag{3.1}$$

$$v_mm = \; rev/s \; \times C_wheel \qquad [mm/s] \tag{3.2}$$

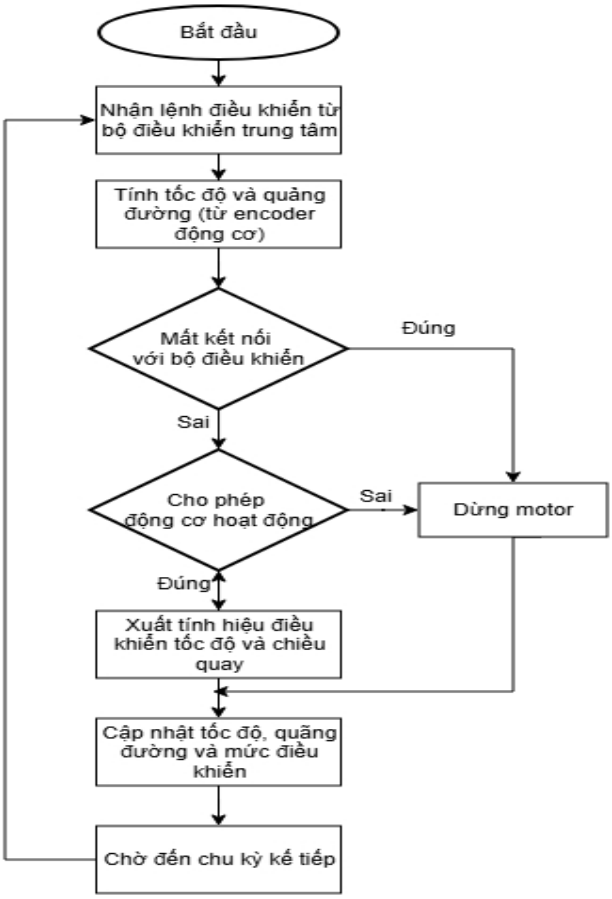
$$v = \; v_mm \; \times \; \frac{3{,}6}{1000} \qquad [km/h] \tag{3.3}$$

Giá trị tốc độ thô được làm mượt bằng bộ lọc thông thấp EMA để khử jitter lượng tử của phép đếm xung trong chu kỳ đếm xung 5 ms, với hệ số α = 0,2 (hằng số thời gian khoảng 25 ms); khi gần đứng yên (dưới 0,05 km/h) tốc độ được ép về đúng 0 để tránh hiện tượng kẹt ở giá trị nhỏ:

$$v_filt \leftarrow \; v_filt \; + \; \alpha \; \times \; (v - \; v_filt\,) \tag{3.4}$$

Quãng đường mỗi chu kỳ được tính theo công thức (3.5) và tích lũy ở đơn vị micromet; mỗi khi đủ một mét nguyên, phần nguyên được cộng vào odometer còn phần dư được giữ lại cho chu kỳ sau, nhờ đó không mất mát do làm tròn.

$$\Delta s = \; |\Delta count| \; \times \; C_wheel \; \times \frac{1000}{PULSE/REV} \; [\mu m/\text{chu kỳ}] \tag{3.5}$$



Hình 3.25 Lưu đồ giải thuật cho tác vụ Motor Control

3.4.6 Module Battery Manager

Tác vụ đo pin chạy mỗi 100 ms. Giá trị ADC được quy đổi sang điện áp pack bằng hệ số khôi phục ×3,2 của mạch chia áp theo công thức:

$$V_ref = \left(\frac{ADC_raw}{4095}\right) \times V_ref \times \frac{(R1+R2)}{R2} = \frac{ADC_raw}{4095} \times 3{,}3 \times 3{,}2 \quad (3.6)$$

Trong đó ADC_raw là giá trị số 12-bit (dải 0–4095), V_ref = 3,3 V là điện áp tham chiếu của ADC, và hệ số (R1 + R2)/R2 = 3,2 là hệ số khôi phục. Dải đo tối đa của mạch là 3,3 × 3,2 = 10,56 V, đủ biên dự phòng trên mức 8,4 V của pack 2S.

Thách thức chính khi đo trên mô hình là hiện tượng dội mass: khi động cơ kéo dòng, điểm tham chiếu GND của STM32 bị nâng lên khiến ADC đọc thấp đúng lúc PWM đang dẫn dòng; chỉ trong khe nghỉ giữa mỗi chu kỳ PWM (khoảng 1 kHz), dòng về 0 thì ADC mới đọc đúng điện áp hở mạch (OCV). Vì nhiễu chỉ kéo giá trị xuống, điện áp thật nằm ở các mẫu cao nhất.

Để khôi phục OCV, firmware dùng kỹ thuật đường bao trên: mỗi chu kỳ lấy 48 mẫu liên tiếp cách nhau một khe trễ cỡ vài chục micro-giây được cố ý đặt lệch chu kỳ PWM nhằm tránh hiện tượng trùng nhịp, sắp xếp tăng dần bằng insertion sort, rồi lấy mẫu ở phân vị cao (bỏ bốn mẫu cao nhất, lấy mẫu thứ N−1−4 ứng với khoảng 92%) làm điện áp OCV thật. Giá trị này tiếp tục được làm mượt bằng bộ lọc EMA số nguyên với α = 1/16 (hằng số thời gian khoảng 1,6 giây), và được bù khoảng 100 mV khi đang sạc để khử phần điện áp tăng do dòng sạc trên dây. Phần trăm dung lượng được tra từ bảng tra thực nghiệm 11 điểm (Bảng 3.15) bằng nội suy tuyến tính:

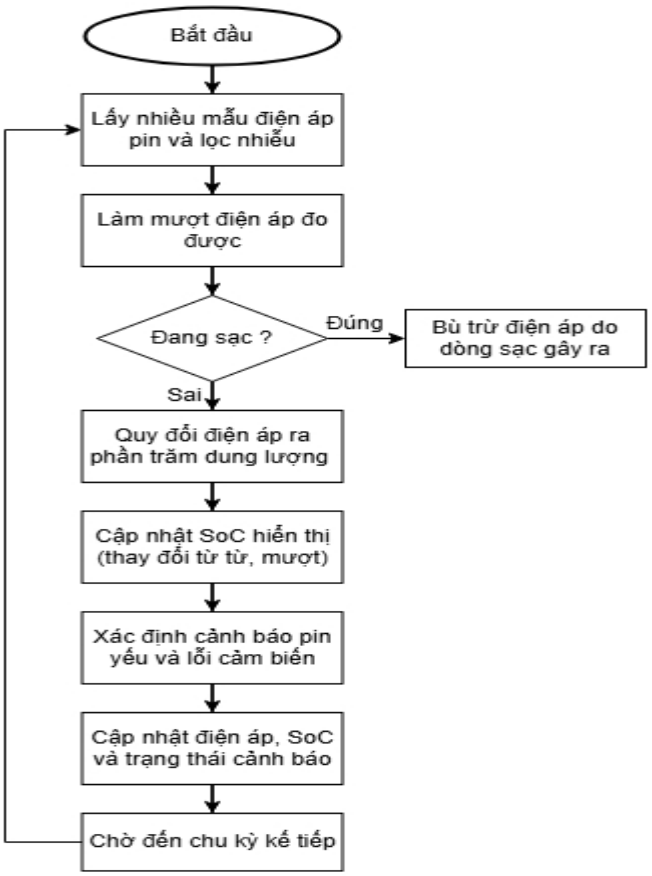
$$SoC = S_lo + (V - V_lo) \times \frac{S_hi - S_lo}{V_hi - V_lo} \quad (3.7)$$

Bảng 3.15 Bảng tra SoC theo điện áp pack 2S (nội suy tuyến tính giữa các điểm)

Điện áp pack (mV)	SoC (%)	Điện áp pack (mV)	SoC (%)
8400	100	6800	20
8200	90	6600	10
8000	80	6400	0
7800	70	—	—

Điện áp pack (mV)	SoC (%)	Điện áp pack (mV)	SoC (%)
7600	60	—	—
7400	50	—	—
7200	40	—	—
7000	30	—	—

Để hiển thị mượt, giá trị SoC đưa ra màn hình được giới hạn tốc độ thay đổi tối đa khoảng 1% mỗi chu kỳ chờ (tương đương ~1,6%/giây) theo cả hai chiều — giảm khi pin vơi thật, tăng khi điện áp hồi hoặc khi sạc — và không bao giờ bị đóng băng. Ngoài ra, cờ lỗi cảm biến được bật khi điện áp đã lọc nằm ngoài dải hợp lý (dưới 5,8 V hoặc trên 8,9 V), và cờ cảnh báo pin yếu được bật khi SoC xuống tới ngưỡng 20%. Toàn bộ kết quả được ghi vào g_ecu_data trong mutex để phía CAN gửi đi.



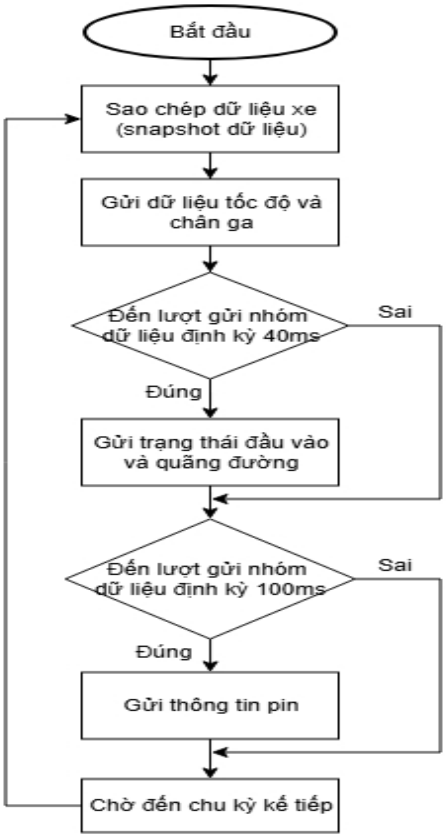
Hình 3.26 Lưu đồ giải thuật cho tác vụ Battery Manager

3.4.7 Module CAN Handler

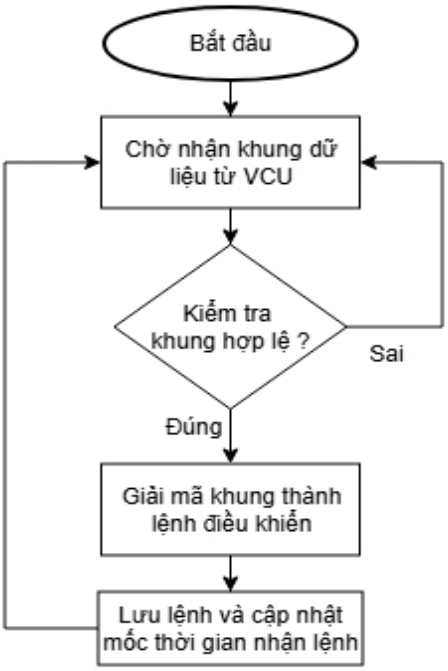
Module CAN hiện thực giao thức đã thiết kế ở mục 3.3. Khi khởi tạo, firmware cấu hình bộ lọc phần cứng (bank 0, chế độ IDMASK 32-bit) khớp tuyệt đối định danh 0x200 và gán vào FIFO0, bật ngắt nhận FIFO0, đồng thời tạo hàng đợi nhận có sức chứa 8 khung.

Ở chiều phát, tác vụ Task_CANTransmit chạy mỗi 20 ms. Đầu mỗi chu kỳ, tác vụ sao chép toàn bộ g_ecu_data dưới mutex rồi gửi các khung từ bản sao đó theo lịch phân tần dựa trên bộ đếm: khung 0x100 gửi mỗi chu kỳ (20 ms), khung 0x102 và 0x103 gửi mỗi hai chu kỳ (40 ms), khung 0x101 gửi mỗi năm chu kỳ (100 ms). Dữ liệu được mã hóa dưới dạng số nguyên đúng theo quy ước ở mục 3.3 (tốc độ nhân 100, điện áp theo mV, PWM theo phần trăm).

Ở chiều nhận, trình phục vụ ngắt chỉ đọc khung và đẩy vào hàng đợi (đẩy khỏi ISR), còn tác vụ Task_CANReceive chặn chờ trên hàng đợi để xử lý. Khi lấy được khung, tác vụ kiểm tra định danh đúng 0x200 và độ dài tối thiểu 4 byte, giải mã theo từng byte thành cấu trúc lệnh VcuCommand, giới hạn mức PWM mục tiêu không vượt 100%, rồi ghi lệnh vào g_vcu_cmd đồng thời cập nhật mốc g_vcu_last_tick trong mutex (cấp dữ liệu cho watchdog ở mục 3.4.5). Log chỉ được in khi lệnh thay đổi để tránh tràn cổng UART.



Hình 3.27 Lưu đồ giải thuật cho Task_CANTransmit



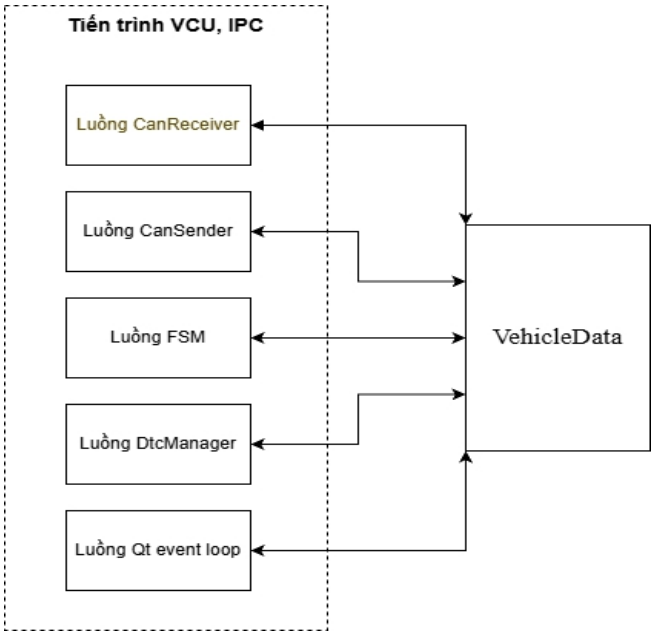
Hình 3.28 Lưu đồ giải thuật cho Task_CANReceive

3.5 THIẾT KẾ VÀ THI CÔNG PHẦN MỀM VCU

VCU là bộ điều khiển trung tâm, chạy trên Raspberry Pi dưới hệ điều hành Linux và được viết bằng C++. Nó nhận dữ liệu trạng thái từ STM32 qua CAN, chạy máy trạng thái để ra quyết định điều khiển, phát khung lệnh 0x200 xuống STM32, đồng thời cấp dữ liệu cho giao diện cụm đồng hồ. Mục này trình bày kiến trúc tiến trình và đa luồng, kho dữ liệu thread-safe, truyền thông CAN trên Linux, máy trạng thái sáu trạng thái, kiến trúc xử lý lỗi hai tầng, hệ thống chẩn đoán DTC và cơ chế lưu trữ bền vững.

3.5.1 Kiến trúc tiến trình và mô hình đa luồng

Như đã nêu ở mục 3.1, phần mềm VCU và giao diện IPC cùng chạy trong một tiến trình duy nhất trên Raspberry Pi. Bên trong tiến trình đó là nhiều luồng song song: luồng chính của Qt chạy vòng lặp sự kiện và giao diện QML; luồng CanReceiver nhận và giải mã telemetry; luồng máy trạng thái (FSM) chạy chu kỳ 20 ms; luồng CanSender phát khung lệnh mỗi 20 ms; và luồng của bộ chẩn đoán DtcManager chạy chu kỳ 200 ms. Toàn bộ các luồng trao đổi dữ liệu qua một kho dữ liệu trung tâm thread-safe là VehicleData (mục 3.5.2).



Hình 3.29 Kiến trúc tiến trình và mô hình đa luồng của VCU

Về thi công, hàm main khởi tạo các thành phần theo thứ tự: tạo ứng dụng Qt, tạo lớp cầu nối VcuBridge, khởi động CanReceiver, khởi động máy trạng thái,

khởi động CanSender, nạp giao diện QML, nạp odometer đã lưu, rồi khởi động DtcManager trước khi vào vòng lặp sự kiện. Hai callback được đăng ký để nối luồng nền với giao diện: mỗi khi nhận khung CAN, CanReceiver gọi notifyDataReady của VcuBridge; mỗi khi đổi trạng thái, máy trạng thái gọi notifyStateChanged. Phần mềm cũng có cơ chế suy giảm mềm: nếu không mở được giao diện can0, hệ thống chỉ cảnh báo và chạy ở chế độ demo với giá trị mặc định thay vì thoát. Khi đóng ứng dụng, các luồng được dừng và chờ kết thúc theo thứ tự ngược lại để giải phóng tài nguyên sạch sẽ.

3.5.2 VehicleData — kho dữ liệu trung tâm thread-safe

VehicleData là kho dữ liệu chia sẻ giữa mọi luồng, được hiện thực theo mẫu singleton (biến cục bộ tĩnh, được C++11 bảo đảm khởi tạo an toàn đa luồng). Nó đóng vai trò tương đương cặp g_ecu_data và xEcuMutex trên STM32. Để giảm tranh chấp giữa các luồng độc lập, lớp này dùng cơ chế khóa với bốn mutex riêng biệt bảo vệ bốn nhóm dữ liệu: telemetry từ STM32, lệnh điều khiển, quãng đường và dữ liệu chẩn đoán.

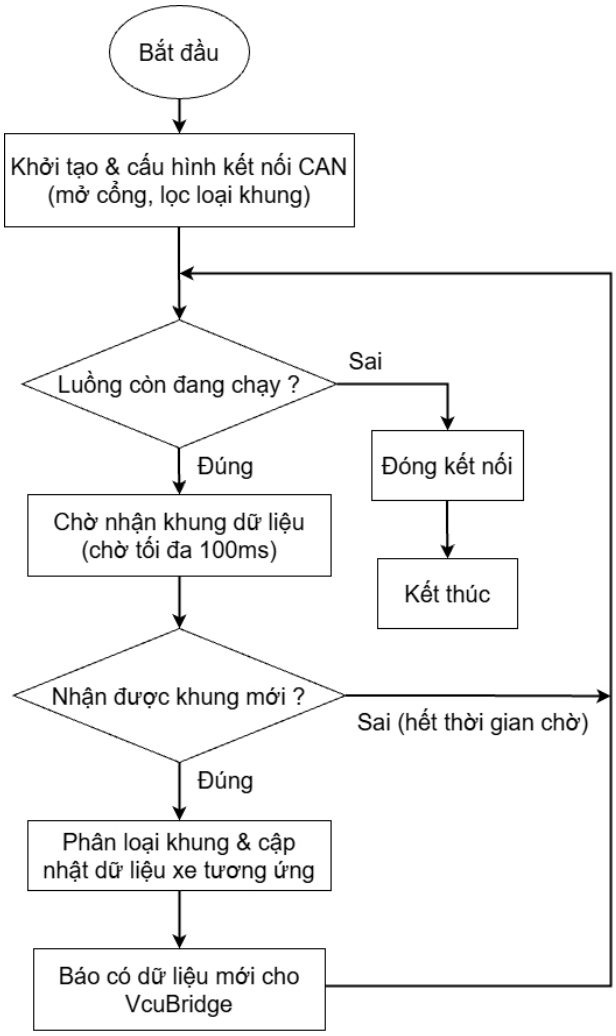
Nguyên tắc truy cập là snapshot theo giá trị: phương thức đọc telemetry trả về một bản sao của toàn bộ cấu trúc dữ liệu trong khi đang giữ khóa rồi nhả ngay, nhờ đó luồng đọc làm việc trên bản sao mà không giữ khóa lâu - đồng nhất với nguyên tắc snapshot dưới mutex của firmware STM32. Các phương thức cập nhật (do CanReceiver gọi) vừa giải mã vừa kiểm tra hợp lệ và kẹp giá trị bất thường về mặc định an toàn (cần số ngoài dải về P, chế độ ngoài dải về NORMAL), đồng thời ghi mốc thời gian nhận khung phục vụ watchdog. Kho dữ liệu cũng cung cấp hàm kiểm tra timeout CAN, hàm tích lũy quãng đường, và cặp đọc/ghi dữ liệu chẩn đoán cho DtcManager.

3.5.3 Truyền thông CAN trên Linux (SocketCAN)

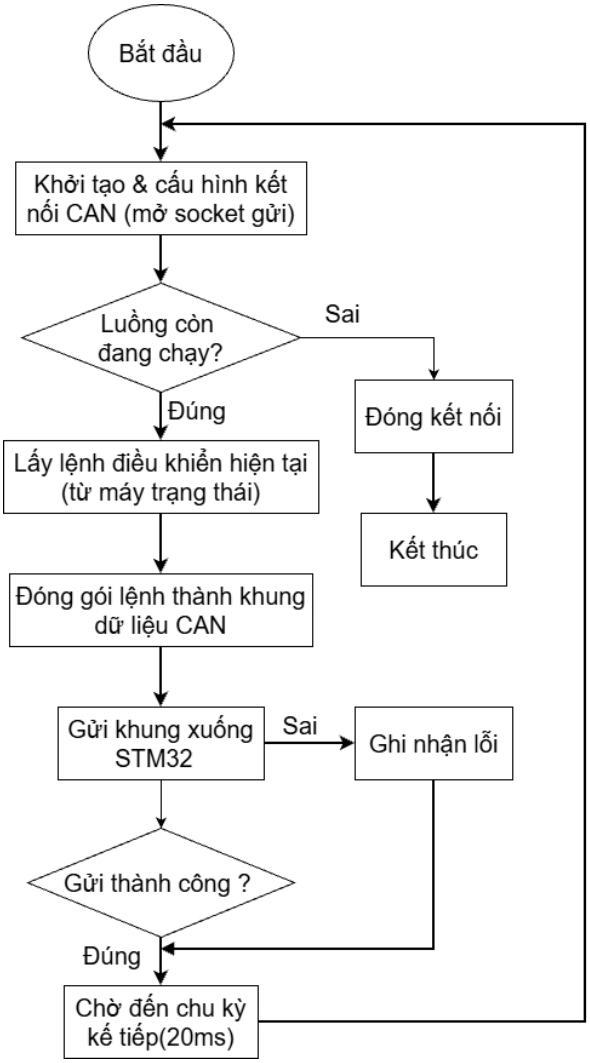
Vì Raspberry Pi không có ngoại vi CAN, hệ thống dùng module MCP2515 cùng khung SocketCAN của nhân Linux, qua đó bus CAN xuất hiện như một giao diện mạng tên can0. Lớp CanReceiver mở một socket CAN thô, gắn vào can0, có bộ lọc ở phần mềm với bốn luật khớp tuyệt đối các định danh 0x100–

0x103 và một thời gian chờ đọc 100 ms để luồng có thể thoát mượt khi tắt. Luồng nhận đọc khung theo kiểu chặn, giải mã theo định danh, sao chép vào cấu trúc tương ứng và cập nhật VehicleData, sau đó gọi callback báo cho VcuBridge cập nhật giao diện.

Ở chiều phát, lớp CanSender dùng một socket riêng và một luồng gửi khung lệnh 0x200 mỗi 20 ms theo mốc thời gian tuyệt đối (sleep_until trên đồng hồ steady_clock) nên không bị trôi nhịp. Mỗi chu kỳ, luồng này lấy lệnh hiện tại từ VehicleData, đóng gói thành khung 0x200 và ghi ra socket. Chi tiết bố cục byte và quy ước mã hóa đã trình bày ở mục 3.3; phần này là hiện thực của giao thức đó trên nền Linux.



Hình 3.30 Lưu đồ giải thuật cho luồng CanReceive



Hình 3.31 Lưu đồ giải thuật cho luồng CanSender

3.5.4 Máy trạng thái sáu trạng thái

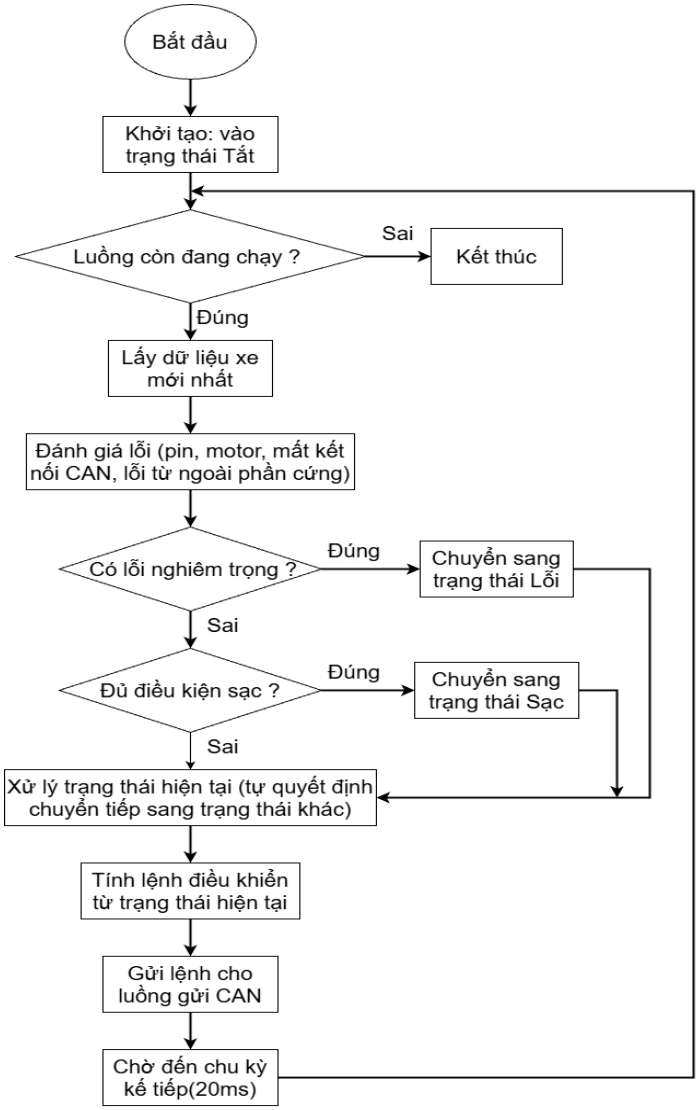
Trái tim của VCU là máy trạng thái hữu hạn chạy trong một luồng riêng với chu kỳ cố định 20 ms. Trạng thái hiện hành được lưu bằng biến nguyên tử m_state để các thành phần khác đọc được mà không cần khóa. Hệ thống có sáu trạng thái: OFF, STANDBY, READY, DRIVING, CHARGING và FAULT, luôn khởi động ở OFF - trạng thái an toàn nhất.

Mỗi trạng thái được tổ chức theo mẫu Entry/Run: hàm Entry chạy một lần khi vào trạng thái để khởi tạo, còn hàm Run chạy mỗi chu kỳ để kiểm tra điều kiện chuyển. Mỗi vòng lặp 20 ms thực hiện năm bước: lấy snapshot dữ liệu; đánh giá lỗi (gộp kết quả của evaluateFaults với cờ timeout CAN và cờ lỗi nghiêm trọng từ chẩn đoán); áp dụng thứ tự ưu tiên (nếu có lỗi và không ở OFF/FAULT

thì chuyển FAULT, ngược lại nếu đủ điều kiện sạc thì chuyển CHARGING, còn lại chạy hàm Run của trạng thái hiện hành); tính lệnh điều khiển; và ghi lệnh vào kho dữ liệu cho CanSender.

Bảng 3.16 Các điều kiện chuyển trạng thái của máy trạng thái VCU

Từ trạng thái	Điều kiện chuyển	Đến trạng thái
OFF	key_on (bật khóa điện)	STANDBY
STANDBY	Tắt khóa điện	OFF
STANDBY	Đạp phanh và cần số D/ R	READY
READY	Tắt khóa điện	OFF
READY	Cần số về P/N	STANDBY
READY	Độ mở ga ≥ 5%	DRIVING
DRIVING	Tắt khóa điện	OFF
DRIVING	Cần số = N	STANDBY
DRIVING	Cần số = P và tốc độ ≈ 0	STANDBY
DRIVING	Nhả ga < 2% liên tục 200 ms	READY
Mọi state (khác OFF, khác FAULT)	Xuất hiện cờ lỗi bất kỳ	FAULT
FAULT	Tắt rồi bật lại khóa điện (key cycle)	STANDBY
OFF hoặc STANDBY (đứng yên)	Có tín hiệu cáp sạc	CHARGING
CHARGING	Rút sạc và khóa điện đang bật	STANDBY
CHARGING	Rút sạc và khóa điện tắt	OFF



Hình 3.32 Lưu đồ vòng lặp StateMachine

Hàm tính lệnh điều khiển (computeCommand) là nơi duy nhất quyết định cho phép động cơ chạy và mức PWM mục tiêu, đúng nguyên tắc nguồn quyết định duy nhất: STM32 chỉ thực thi lệnh này. Chỉ ở trạng thái DRIVING động cơ mới được phép chạy, chiều quay phụ thuộc cần số (lùi khi gear = R), còn mức PWM được tính theo chế độ lái. Ở mọi trạng thái khác, lệnh ép động cơ tắt và PWM về 0. Mức PWM theo chế độ lái được tính tuyến tính theo độ mở ga nhưng bị kẹp trần khác nhau: ECO tối đa 50%, NORMAL 80%, SPORT 100% - phản ánh đặc tính tiết kiệm pin của chế độ ECO và sức mạnh tối đa của SPORT. Khi hệ thống ở chế độ chạy cầm chừng (limp-home, do chẩn đoán mức HIGH), mức PWM bị kẹp thêm về 30%.

3.5.5 Kiến trúc xử lý lỗi hai tầng và logic an toàn

Một quyết định thiết kế cốt lõi của hệ thống là tách xử lý lỗi thành hai tầng độc lập, đúng theo cách cụm đồng hồ ô tô thật vận hành, vì hai loại lỗi có bản chất khác nhau. Tầng A - Bảo vệ - nhằm ngắt an toàn tức thời, hiển thị màu đỏ phủ toàn màn hình, phản ứng nhanh với debounce ngắn, không lưu trữ và thoát bằng chu trình khóa điện; tầng này do máy trạng thái xử lý trong hàm evaluateFaults mỗi 20 ms. Tầng B - Chẩn đoán - nhằm ghi nhận, theo dõi và báo bảo dưỡng, hiển thị đèn vàng MIL, chín dần theo cơ chế two-trip và lưu bền vững ra file; tầng này do DtcManager xử lý (mục 3.5.6). Nguyên tắc ưu tiên hiển thị tuân theo chuẩn ngành: đỏ đè lên vàng đè lên thông tin.

Các lỗi tầng A được biểu diễn bằng một bitmask và phát hiện qua bộ đếm debounce - điều kiện lỗi phải đúng liên tục đủ thời gian mới được công nhận, nhằm chống cảnh báo giả (Bảng 3.17). Đặc biệt, lỗi tầng A được chốt: một khi đã vào FAULT, hệ thống không tự thoát ngay cả khi điều kiện lỗi biến mất, buộc người vận hành phải tắt rồi bật lại khóa điện - mô phỏng đúng hành vi xe thật, tránh việc xe tự chạy lại sau một sự cố thoáng qua.

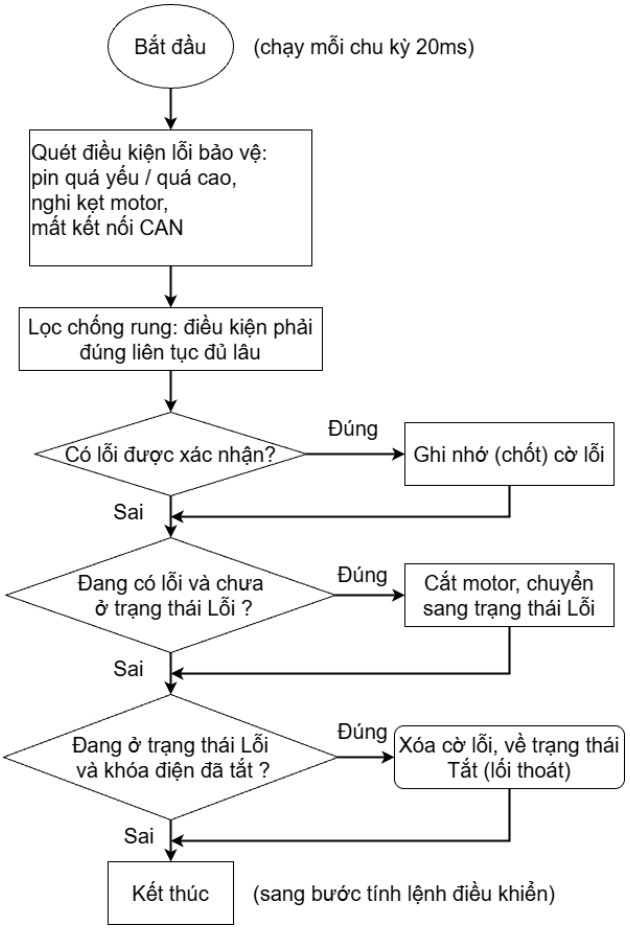
Bảng 3.17 Danh mục lỗi tầng A (Bảo vệ) do máy trạng thái xử lý

Lỗi	Điều kiện phát hiện	Debounce	Hành vi
UNDERVOLTAGE	Cảm biến tin cậy và điện áp ≤ 6,50 V	2 s	FAULT, cắt motor
OVERVOLTAGE	Cảm biến tin cậy và điện áp > 8,70 V	2 s	FAULT, cắt motor
MOTOR_STALL	DRIVING, PWM ≥ 30% nhưng tốc độ ≈ 0	1,5 s	FAULT, cắt motor
CAN_TIMEOUT	Mất khung CAN từ STM32 quá 2 s	—	FAULT, cắt motor
INJECTED	Từ DTC mức CRITICAL (cờ fault_active)	—	FAULT, cắt motor

Một điểm kỹ thuật tinh tế là phân biệt cảm biến điện áp bị hỏng với pin thật sự yếu - hai sự kiện rất dễ nhầm nếu chỉ nhìn một con số điện áp. Giải pháp là để STM32, nơi có dữ liệu ADC gốc, tự phán độ tin cậy của cảm biến: khi điện áp đo

nằm ngoài dải vật lý của cụm 2S (dưới 5,8 V hoặc trên 8,9 V), tức chân ADC đang trôi tự do do đứt dây, STM32 đặt cờ sensor_fault và gửi lên qua khung 0x101. Máy trạng thái chỉ đánh giá quá áp/sụt áp khi cảm biến đáng tin; khi cảm biến hỏng, hai lỗi này bị khóa lại, nhường cho tầng B bắt thành mã chẩn đoán P0463 (đèn vàng, không dừng xe), còn giao diện hiển thị "--%" thay vì báo pin cạn giả.

Điều kiện chuyển sang sạc cũng được kiểm soát chặt: hàm checkChargingCondition chỉ cho vào CHARGING khi hội đủ ba yếu tố - có tín hiệu cáp sạc, xe đang đứng yên và đang ở trạng thái đỗ (OFF hoặc STANDBY). Đây là mô phỏng đúng xe điện thật: không bao giờ vào sạc khi đang READY hoặc DRIVING; muốn sạc, người dùng phải đưa xe về đỗ trước.



Hình 3.33 Lưu đồ giải thật của Tầng A trong kiến trúc xử lý lỗi

3.5.6 Hệ thống chẩn đoán DtcManager (Tầng B)

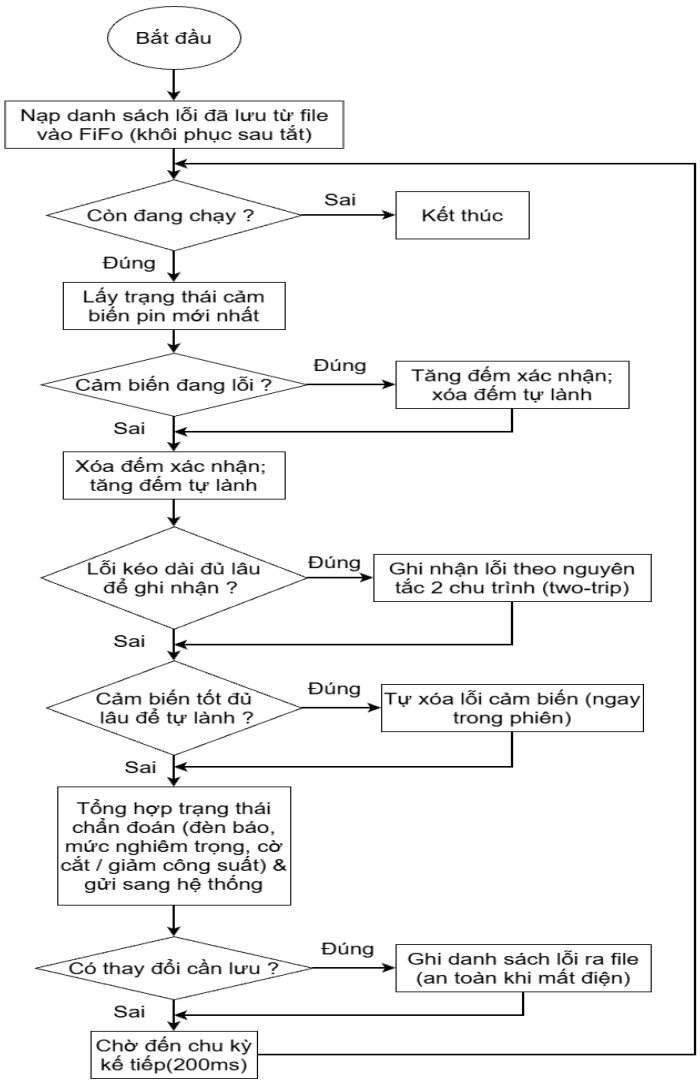
DtcManager chạy trong một luồng riêng chu kỳ 200 ms, đóng vai một cổng chẩn đoán: thu thập, theo dõi vòng đời và lưu trữ các mã lỗi DTC. Vòng đời mỗi DTC mô phỏng quy trình two-trip của chuẩn OBD-II, đi qua bốn trạng thái CLEARED → PENDING → CONFIRMED → PERMANENT, trong đó một "chu trình lái" được định nghĩa là một phiên chạy ứng dụng. Triết lý two-trip là: một lỗi thoáng qua chỉ xuất hiện một chu trình rồi tự hết sẽ không bật đèn báo (tránh làm phiền người lái), chỉ lỗi tái diễn ở chu trình sau mới được xác nhận và bật đèn MIL. Riêng các lỗi nghiêm trọng được xác nhận ngay sang PERMANENT mà không chờ hai chu trình, vì lý do an toàn. Catalog gồm bảy mã lỗi đặt theo chuẩn OBD-II (Bảng 3.18).

Bảng 3.18 Danh mục mã chẩn đoán DTC của hệ thống

Mã	Tên lỗi	Mức	Phát hiện	Hành vi khi xác nhận
P0463	Battery Voltage Sensor	MEDIUM	Tự động	MIL vàng, xe chạy bình thường
P0AA6	HV Isolation Fault	CRITICAL	Inject	FAULT đỏ, cắt motor (permanent)
U0111	Lost Comm with BMS	CRITICAL	Inject	FAULT đỏ, cắt motor (permanent)
P0A3F	Inverter Over-Temperature	HIGH	Inject	Limp-home (giảm PWM còn 30%)
P0A80	Replace HV Battery	HIGH	Inject	Limp-home
P0D0A	Onboard Charger Fault	MEDIUM	Inject	MIL vàng, xe chạy bình thường
P0560	Auxiliary 12V Voltage	MEDIUM	Inject	MIL vàng, xe chạy bình thường

Mức độ nghiêm trọng quyết định DTC tác động thế nào lên xe: mức CRITICAL đặt cờ fault_active buộc máy trạng thái vào FAULT và cắt động cơ; mức HIGH đặt cờ limp_active khiến hàm tính lệnh kẹp PWM xuống 30% (chạy cầm chừng để người lái còn về được xưởng); mức MEDIUM chỉ bật đèn MIL mà không ảnh hưởng vận hành. Trong bảy mã, chỉ P0463 được phát hiện tự động từ

bit sensor_fault của STM32 (liên tục khoảng 2 giây thì ghi nhận), và mã này còn có cơ chế tự lành sống: khi cảm biến tốt trở lại liên tục khoảng 3 giây, P0463 tự xóa ngay trong phiên mà không cần khởi động lại. Sáu mã còn lại được Inject thủ công để trình diễn các lỗi không có phần cứng tạo ra (rò cách điện cao áp, mất liên lạc BMS, quá nhiệt inverter…), qua hai kênh chẩn đoán: một named pipe nhận lệnh văn bản (inject/clear/clearall) phục vụ phát triển, và một menu service ẩn trên giao diện (giữ lâu vào vùng odometer) phục vụ vận hành khi không có terminal.



Hình 3.34 Lưu đồ giải thật của Tầng B trong kiến trúc xử lý lỗi

Để sống sót qua tắt nguồn, các DTC được lưu ra file bằng kỹ thuật ghi nguyên tử (ghi ra file tạm rồi đổi tên đè lên file thật) — thao tác đổi tên là nguyên

tử ở mức hệ điều hành nên file không bao giờ hỏng dở dang dù mất điện đột ngột. Nhờ lưu bền vững, một lỗi PERMANENT sẽ tự đưa xe vào FAULT lại mỗi lần khởi động cho đến khi được xóa bằng lệnh chẩn đoán. Sau mỗi chu kỳ, DtcManager công bố một bản tóm tắt chẩn đoán (đèn MIL, mã ưu tiên hiển thị, cờ fault_active, cờ limp_active) vào VehicleData để máy trạng thái và giao diện cùng sử dụng.

3.6 THIẾT KẾ VÀ THI CÔNG GIAO DIỆN IPC (QT/QML)

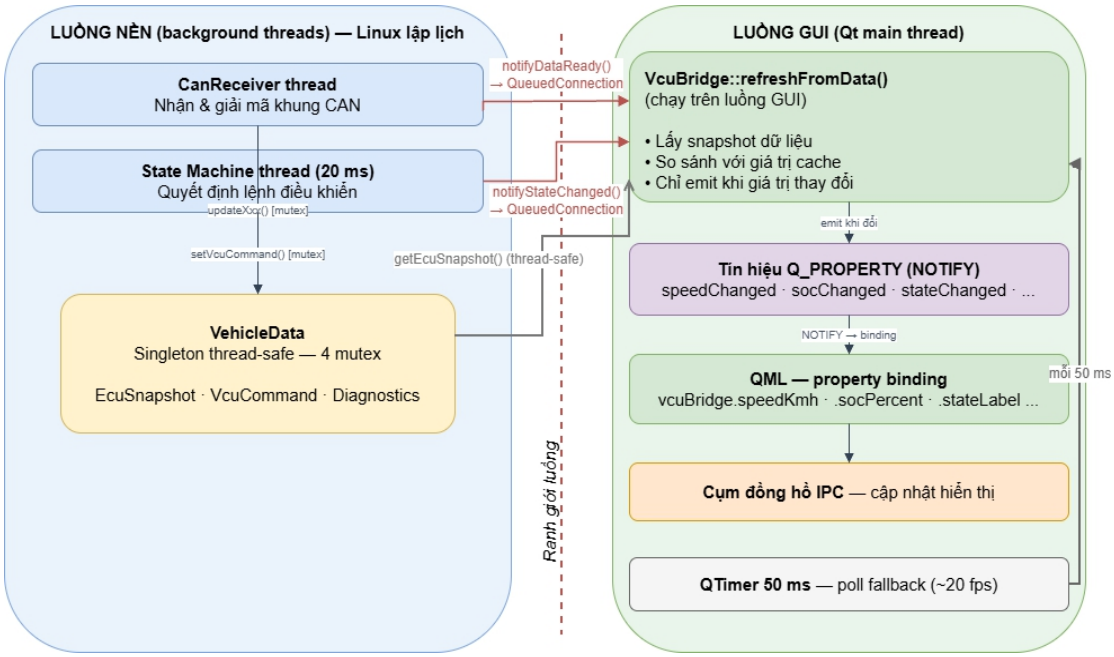
Cụm đồng hồ số (IPC) là thành phần hiển thị của hệ thống, được xây dựng bằng Qt/QML và chạy chung tiến trình với phần mềm VCU trên Raspberry Pi, xuất ra màn hình Waveshare 7 inch ở chế độ toàn màn hình. IPC tách bạch hoàn toàn khỏi logic điều khiển: nó chỉ trực quan hóa dữ liệu và trạng thái do VCU cung cấp, không tự ra quyết định. Mục này trình bày lớp cầu nối giữa C++ và QML, cơ chế điều khiển màn hình theo trạng thái, thiết kế các màn hình, các hoạt ảnh chuyển cảnh, hệ thống cảnh báo – chẩn đoán và bố cục cụm đồng hồ.

3.6.1 Lớp cầu nối VcuBridge (C++ ↔ QML)

Toàn bộ dữ liệu hiển thị được đưa từ C++ sang QML thông qua lớp cầu nối VcuBridge - một đối tượng Qt đẩy dữ liệu của VehicleData ra QML dưới dạng các thuộc tính Q_PROPERTY, mỗi thuộc tính kèm một tín hiệu thông báo (NOTIFY). Nhờ cơ chế ràng buộc thuộc tính của QML, mọi thành phần giao diện tham chiếu tới các thuộc tính này sẽ tự cập nhật khi tín hiệu được phát.

Thách thức thi công nằm ở đồng bộ đa luồng: dữ liệu đến từ luồng CanReceiver và luồng máy trạng thái, nhưng các đối tượng đồ họa Qt chỉ được phép truy cập từ luồng giao diện (GUI thread). Để giải quyết, các hàm thông báo của VcuBridge không cập nhật trực tiếp mà dùng cơ chế hàng đợi liên luồng (kết nối kiểu Qt::QueuedConnection) để chuyển việc làm tươi dữ liệu sang đúng luồng GUI một cách an toàn. Ngoài ra, một bộ định thời 50 ms (khoảng 20 khung hình mỗi giây) cũng định kỳ làm tươi giao diện để hình ảnh luôn mượt ngay cả khi tạm thời không có sự kiện CAN. Một tối ưu quan trọng là chỉ phát tín hiệu khi giá trị thực sự thay đổi: mỗi lần làm tươi, cầu nối so sánh giá trị mới với giá

trị đã lưu và chỉ phát tín hiệu thay đổi khi khác nhau, nhờ đó tránh việc QML vẽ lại không cần thiết và giữ giao diện nhẹ. Cầu nối còn cung cấp các thuộc tính truy xuất tiện dụng cho QML (nhãn trạng thái, nhãn cần số và chế độ, màu theo trạng thái, nhãn lỗi, cờ đèn MIL, mã DTC) cùng các hàm gọi được từ QML để xóa hoặc inject DTC, chuyển tiếp xuống DtcManager.



Hình 3.35 Cơ chế chuyển C++ qua QML thông qua lớp cầu nối VcuBridge

3.6.2 Điều khiển màn hình theo trạng thái

Tệp gốc main.qml tạo một cửa sổ toàn màn hình nền đen và dùng một bộ nạp để hiển thị màn hình tương ứng với trạng thái xe. Hàm ánh xạ trạng thái sang màn hình cho thấy một quyết định thiết kế đáng chú ý: ba trạng thái STANDBY, READY và DRIVING dùng chung một màn hình StandbyScreen (phân biệt bằng nhãn và màu sắc), trạng thái CHARGING dùng màn hình riêng, còn lại hiển thị OffScreen; nhờ đó tránh phải duy trì ba tệp giao diện gần giống nhau (Bảng 3.19).

Khi trạng thái thay đổi, giao diện áp dụng một logic chuyển cảnh có thứ tự ưu tiên rõ ràng: FAULT có ưu tiên tuyệt đối (dừng mọi hoạt ảnh, lớp phủ lỗi tự hiện); kế đến là chuyển từ OFF sang STANDBY (chạy hoạt ảnh khởi động); rồi đến rời CHARGING (chạy hoạt ảnh ngừng sạc); rồi đến về OFF (chạy hoạt ảnh tắt máy); cuối cùng các chuyển cảnh thông thường chỉ hòa hình trong 300 ms.

Lớp phủ cảnh báo lỗi được đặt ở một bộ nạp riêng, kích hoạt theo cờ lỗi và có thứ tự hiển thị cao nhất nên luôn nằm trên mọi màn hình.

Bảng 3.19 Ánh xạ trạng thái xe sang màn hình hiển thị và hiệu ứng chuyển cảnh

Trạng thái	Màn hình hiển thị	Hiệu ứng khi chuyển vào - Độ ưu tiên
OFF	OffScreen	ShutdownAnimation (khi từ trạng thái khác về) - 1
STANDBY	StandbyScreen	BootAnimation (từ OFF) - 3 hoặc ChargingStopAnimation (từ CHARGING) - 2
READY	StandbyScreen	Hòa hình 300 ms
DRIVING	StandbyScreen	Hòa hình 300 ms
CHARGING	ChargingScreen	Hòa hình 300 ms
FAULT	Giữ màn hình hiện tại + FaultOverlay	Lớp phủ đỏ hiện lên trên cùng - 4

3.6.3 Thiết kế các màn hình

Hệ thống có ba màn hình chính. OffScreen tối giản với nền đen và dòng chữ trạng thái tắt nguồn. StandbyScreen là màn hình cụm đồng hồ chính, dùng chung cho ba trạng thái chờ, sẵn sàng và đang lái — sự khác biệt giữa các trạng thái được thể hiện qua nhãn trạng thái, dòng gợi ý thao tác và màu sắc thay vì tạo màn hình riêng (bố cục chi tiết ở mục 3.6.6). ChargingScreen là màn hình sạc riêng, giữ thanh trên cùng giống StandbyScreen (các đèn báo, điện áp, huy hiệu trạng thái và đồng hồ) nhưng phần thân hiển thị tiến trình sạc gồm mức pin, điện áp và hoạt ảnh nạp, trong khi động cơ bị khóa hoàn toàn.

3.6.4 Hoạt ảnh chuyển màn hình

Để chuyển cảnh mượt mà như cụm đồng hồ ô tô thật, hệ thống dùng ba lớp hoạt ảnh phủ lên trên màn hình (nhưng dưới lớp cảnh báo lỗi), mỗi lớp có một hàm khởi chạy và một tín hiệu báo kết thúc. Hoạt ảnh khởi động (khi bật khóa điện từ OFF sang STANDBY) hiện logo và khẩu hiệu hệ thống trên nền chuyển sắc rồi mờ đi để lộ màn hình chính. Hoạt ảnh tắt máy (khi về OFF) phủ nhanh một lớp đen, hiện dòng chữ tắt nguồn rồi tối hẳn trước khi chuyển sang màn hình

tắt. Hoạt ảnh ngừng sạc (khi rời CHARGING) phủ lớp tối và hiển thị thông báo kết thúc sạc kèm mức pin và điện áp hiện tại.

3.6.5 Hệ thống cảnh báo và chẩn đoán

Giao diện thể hiện kiến trúc lỗi hai tầng (mục 3.5.5) đúng theo chuẩn hiển thị ô tô (ISO 2575), với nguyên tắc màu đỏ đè vàng đè thông tin. Lỗi bảo vệ tầng A được thể hiện bằng lớp phủ đỏ chiếm phần lớn màn hình: biểu tượng cảnh báo, dòng chữ phát hiện lỗi, tên lỗi cụ thể lấy từ cầu nối, hướng dẫn tắt – bật khóa điện để khôi phục và một nút xóa DTC; lớp phủ này nhấp nháy liên tục để thu hút sự chú ý. Lỗi chẩn đoán tầng B được thể hiện bằng đèn MIL màu vàng trên thanh trên cùng, sáng khi có DTC được xác nhận; chạm vào đèn MIL sẽ mở bảng DTC chi tiết liệt kê các mã đang hoạt động kèm trạng thái và nút xóa toàn bộ. Ngoài ra còn có một menu service ẩn để inject lỗi mô phỏng (giữ lâu vào vùng đồng hồ quãng đường), phục vụ trình diễn khi không có terminal. Bảng 3.20 liệt kê các đèn báo trên thanh trên cùng. Riêng khi cảm biến điện áp hỏng, giao diện hiển thị "--%" thay cho mức pin để không báo pin cạn giả.

Bảng 3.20 Các đèn báo trên thanh trên cùng của cụm đồng hồ

Biểu tượng	Ý nghĩa	Điều kiện sáng
	Khóa điện	Khóa điện đang bật (key)
⚡	Pin yếu	SoC ≤ 20%
⚠	Lỗi bảo vệ (tầng A)	Đang có lỗi bảo vệ (hasFault)
⚙ (MIL)	Đèn check-engine (tầng B)	Có DTC ở trạng thái CONFIRMED/PERMANENT (milOn)

3.6.6 Bố cục cụm đồng hồ

Màn hình chính StandbyScreen được bố trí theo cấu trúc quen thuộc của cụm đồng hồ xe. Thanh trên cùng gồm khu vực các đèn báo, giá trị điện áp, huy hiệu trạng thái xe ở giữa và đồng hồ thời gian (lấy từ thời gian hệ thống được đồng bộ từ RTC DS3231). Vùng bên trái hiển thị nhãn chế độ lái, tốc độ cỡ lớn cùng hình ảnh chiếc xe trên nền đường phối cảnh. Vùng giữa là đồng hồ tốc độ dạng cung

tròn được vẽ bằng Canvas (cung quét, vạch chia, kim chỉ đổi màu theo chế độ lái), số tốc độ ở tâm, thanh hiển thị độ mở ga, bộ chọn cần số P/R/N/D làm nổi vị trí hiện tại và chỉ báo phanh. Vùng bên phải hiển thị đồng hồ quãng đường. Thanh dưới cùng hiển thị dòng gợi ý thao tác thay đổi theo trạng thái xe cùng thanh pin và phần trăm SoC đổi màu theo mức (nhấp nháy khi pin yếu).

Về mặt thi công, màn hình áp dụng một số kỹ thuật để đạt trải nghiệm gần với cụm đồng hồ thật: nền đổi màu theo chế độ lái bằng hoạt ảnh chuyển màu; tách giá trị tốc độ thành hai mạch — một giá trị mượt cho kim chỉ và một giá trị ổn định cho chữ số — để kim quay êm trong khi con số không nhảy loạn; chạy một chuỗi tự kiểm tra khi khởi động với các đèn báo sáng lần lượt và kim quét hết thang; và kiểm tra tính hợp lệ của điện áp trước khi hiển thị.

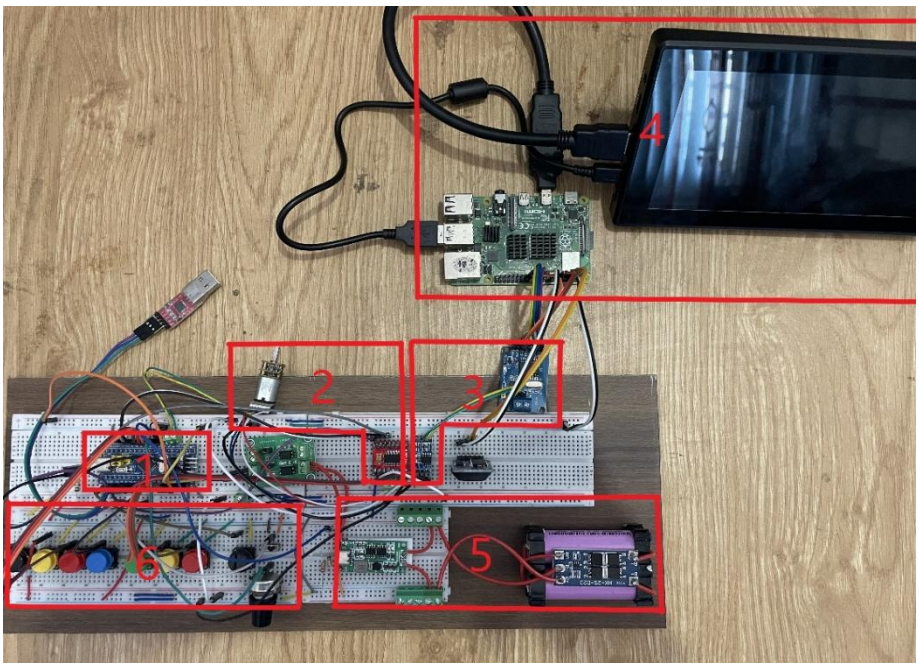
CHƯƠNG 4

KẾT QUẢ THỰC HIỆN

Sau quá trình nghiên cứu, thiết kế và thi công đã trình bày ở các chương trước, nhóm đã hoàn thiện mô hình hệ thống cụm đồng hồ kỹ thuật số (IPC) và bộ điều khiển trung tâm (VCU) cho xe ô tô điện. Hệ thống đã thực hiện được các mục tiêu đã đề ra thông qua kết quả của từng khối firmware trên ECU node, kết quả truyền thông CAN Bus, kết quả vận hành của bộ điều khiển trung tâm và cơ chế chẩn đoán, giao diện cụm đồng hồ IPC, và cuối cùng là kết quả tích hợp toàn hệ thống theo các kịch bản vận hành cùng các đánh giá định lượng về mức độ đáp ứng so với yêu cầu đã đặt ra.

4.1 KẾT QUẢ THI CÔNG PHẦN CỨNG

Trên cơ sở sơ đồ nguyên lý, sơ đồ nối chân và danh mục linh kiện đã trình bày ở chương 3, nhóm tiến hành lắp ráp và đấu nối các khối chức năng thành một mô hình hoàn chỉnh trên breadboard. Việc bố trí bảo đảm các yêu cầu về đấu nối tín hiệu, chia sẻ nguồn và điểm đất chung giữa các node, đồng thời thuận tiện cho việc quan sát, gỡ lỗi qua UART và trình diễn hệ thống.



Hình 4.1 Mô hình hệ thống VCU–IPC cho mô hình xe điện

Hình 4.1 thể hiện toàn cảnh mô hình sau khi thi công, với các khối chức năng được đánh số tương ứng:

(1) Khối ECU node: kit STM32F103C8T6 — đọc tín hiệu người lái, điều khiển động cơ và truyền dữ liệu lên mạng CAN.

(2) Khối truyền động: động cơ giảm tốc GA12-N20 có encoder cùng driver TB6612FNG — chấp hành lệnh PWM và phản hồi tốc độ.

(3) Khối giao tiếp CAN: gồm cả hai bộ thu phát đặt cạnh nhau — TJA1050 (phía STM32) và MCP2515 (phía Raspberry Pi, giao tiếp SPI) nối với nhau qua cặp dây CANH – CANL.

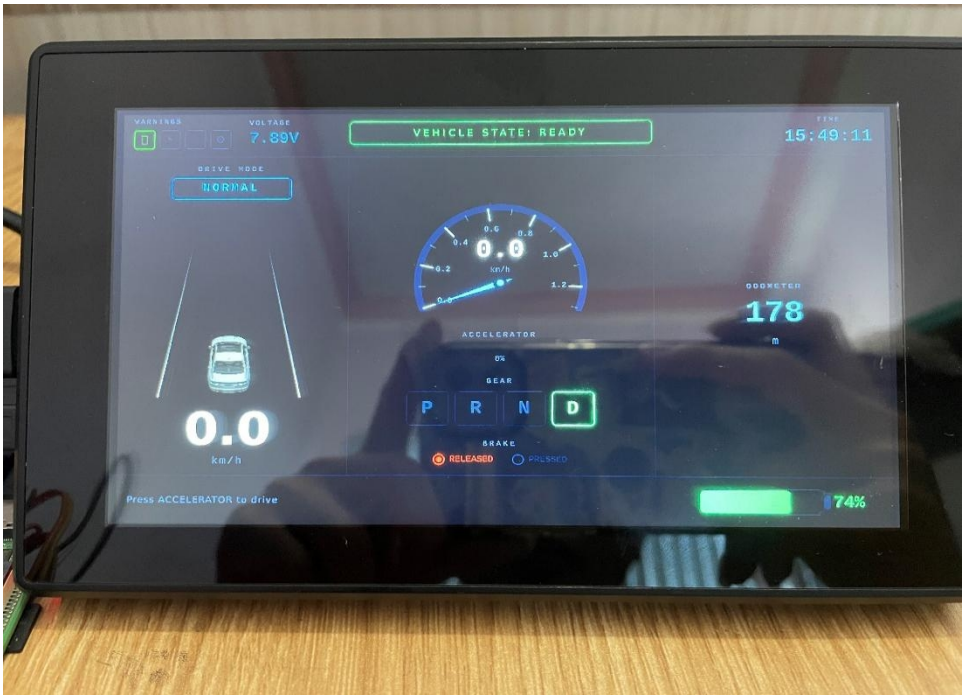
(4) Khối VCU/IPC: Raspberry Pi 4B chạy máy trạng thái và logic điều khiển, xuất giao diện cụm đồng hồ lên màn hình cảm ứng Waveshare 7 inch qua HDMI.

(5) Khối nguồn pin: pack 2S gồm hai cell 18650, mạch bảo vệ BMS 2S (HX-2S-D20), module sạc Type-C và mạch chia áp đưa điện áp pack về ADC của STM32.

(6) Khối đầu vào người dùng: các nút nhấn (Brake, Gear P/R/N/D, Mode), công tắc gạt (Key) và biến trở ga.

Ngoài ra, tín hiệu phát hiện sạc được lấy qua module cách ly quang PC817 (2 kênh) rồi mới đưa về chân PB14 của STM32, nhằm cách ly điện giữa phía mạch sạc và đầu vào của vi điều khiển.

Toàn bộ tín hiệu logic dùng chung mức 3,3V; nguồn truyền động lấy từ pack pin 2S sau BMS, còn Raspberry Pi và màn hình được cấp nguồn riêng nhằm tránh sụt áp và nhiễu.



Hình 4.2 Màn hình giao diện hệ thống

Bên cạnh phần cứng, kết quả trực quan nhất của hệ thống là cụm đồng hồ kỹ thuật số (IPC) hiển thị trên màn hình. Hình 4.2 minh hoạ giao diện khi hệ thống ở trạng thái sẵn sàng vận hành (VEHICLE STATE: READY). Toàn bộ thông tin được bố trí trên một màn hình duy nhất, mô phỏng cụm đồng hồ trên xe điện thực tế: chính giữa là đồng hồ tốc độ dạng kim hiển thị tốc độ hiện tại (km/h); phía trên là thanh trạng thái gồm dung lượng pin, điện áp pack (7,80 V), trạng thái xe và đồng hồ thời gian thực; góc trái hiển thị chế độ lái đang chọn (DRIVE MODE: NORMAL) cùng mô hình xe; khu vực trung tâm thể hiện vị trí ga (ACCELERATOR), cần số đang ở vị trí D (P–R–N–D) và trạng thái phanh (BRAKE: RELEASED); bên phải là quãng đường đã đi (ODOMETER: 178 m) và thanh dung lượng pin (74%).

Giao diện được xây dựng bằng Qt/QML, cập nhật theo thời gian thực từ dữ liệu nhận được qua mạng CAN. Tại thời điểm chụp, xe đang ở trạng thái READY: chân ga ở mức 0%, tốc độ 0,0 km/h, hệ thống hiển thị dòng nhắc "Press ACCELERATOR to drive" báo hiệu sẵn sàng chuyển sang trạng thái vận hành. Kết quả cho thấy toàn bộ thông số phần cứng được phản ánh đầy đủ, trực quan và chính xác trên màn hình, khẳng định sự phối hợp đồng bộ giữa khối thu thập dữ liệu (ECU node) và khối hiển thị (IPC).

4.2 KẾT QUẢ TRUYỀN THÔNG CAN

4.2.1 Khởi động và log UART

Sau khi nạp firmware, hoạt động khởi động của ECU node được giám sát qua cổng UART1 (PA9/PA10) ở tốc độ 115200 baud bằng phần mềm terminal. Hình 4.3 ghi lại toàn bộ chuỗi log mà firmware xuất ra ngay sau khi cấp nguồn, cho phép xác nhận từng bước khởi tạo diễn ra đúng thứ tự thiết kế.



Hình 4.3 Log khởi động của ECU node trên cổng UART

Trình tự log phản ánh chính xác luồng khởi tạo trong chương trình: dòng [CAN] Start OK xác nhận ngoại vi bxCAN được kích hoạt thành công ở 500 kbit/s trước khi nhân FreeRTOS chạy; khối banner xác nhận nền tảng STM32F103C8T6 hoạt động ở 72 MHz; tiếp đó [CAN] Init OK báo hiệu bộ lọc, ngắt nhận và hàng đợi CAN đã được thiết lập; [INIT] All modules initialized xác nhận bốn module (Input, Motor, Battery, CAN) khởi tạo xong; và cuối cùng [INIT] All tasks created - Scheduler starting cho thấy toàn bộ các task đã được tạo thành công (không có thông báo [ERR]) và quyền điều phối được bàn giao cho bộ lập lịch. Kết quả này khẳng định chuỗi khởi tạo phần cứng và phần mềm của ECU node hoạt động ổn định và tin cậy.

4.2.2 Kết quả VCU nhận gói tin CAN từ ECU

Sau khi khởi tạo thành công, ECU node trên STM32 liên tục đóng gói dữ liệu thu thập được thành bốn nhóm khung CAN và phát lên mạng: khung 0x100 (tốc độ, vị trí ga, độ rộng xung PWM) với chu kỳ 20 ms, khung 0x101 (điện áp pin, dung lượng SoC) chu kỳ 100 ms, khung 0x102 (các đầu vào người lái) và 0x103 (quãng đường) chu kỳ 40 ms. Ở chiều ngược lại, bộ điều khiển trung tâm VCU trên Raspberry Pi phát khung lệnh 0x200 để điều khiển ECU node thực thi. Để

kiểm chứng hoạt động truyền nhận, lưu lượng trên bus được giám sát trực tiếp trên Raspberry Pi bằng công cụ SocketCAN.



Hình 4.4 Lưu lượng khung CAN bắt được trên Raspberry Pi

Hình 4.4 cho thấy đầy đủ năm định danh khung (0x100, 0x101, 0x102, 0x103 và 0x200) đều xuất hiện đều đặn trên bus, xác nhận cả hai chiều truyền thông giữa ECU node và VCU đều hoạt động. Các khung dữ liệu đang ở dạng mã thập lục phân; phía VCU tiến hành giải mã từng trường về giá trị vật lý theo đúng cấu trúc thông điệp đã thiết kế ở Chương 3. Bảng 4.1 đối chiếu một thời điểm tiêu biểu khi xe đang vận hành ở số D, chế độ SPORT với chân ga gần tối đa.

Bảng 4.1 Đối chiếu giải mã các khung CAN

STT	CAN ID	Ý nghĩa	Chiều	Giá trị Hex	Giá trị sau giải mã
1	0x100	Tốc độ – Ga – PWM	STM32 → Pi	8D 00 62 62	Tốc độ 1,41 km/h; ga 98%; PWM motor 98%
2	0x101	Pin (áp / SoC)	STM32 → Pi	C5 1E 49 00	Điện áp 7,877 V; SoC 73%; cờ bình thường
3	0x102	Input	STM32 → Pi	03 02 01 00	Số D; chế độ SPORT; công tắc KEY bật
4	0x103	Quãng đường	STM32 → Pi	04 00 00 00	Odometer 4 m
5	0x200	Lệnh điều khiển	Pi → STM32	03 62 03 00	Motor bật, chiều tiến; PWM đích 98%; trạng thái DRIVING

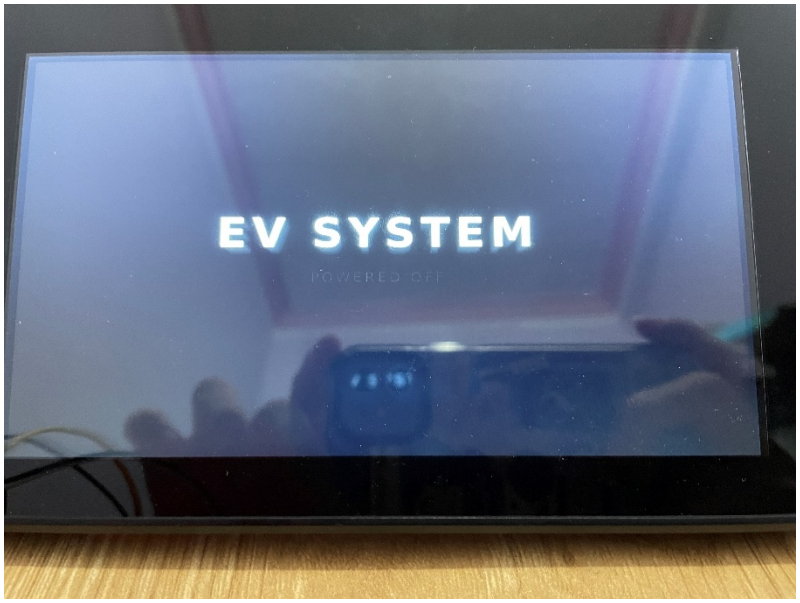
Kết quả giải mã ở Bảng 4.1 hoàn toàn nhất quán: dữ liệu telemetry mà STM32 gửi lên (0x100 – 0x103) phản ánh đúng thao tác đang thực hiện, đồng thời khung lệnh 0x200 mà VCU gửi xuống khớp với trạng thái điều khiển, motor

được phép quay theo chiều tiến với mức PWM tương ứng vị trí ga, trong khi VCU xác định trạng thái xe là DRIVING. Phía Raspberry Pi tiếp nhận và giải mã chính xác toàn bộ các khung này (xác nhận qua log [CAN RX] của ứng dụng), cập nhật vào kho dữ liệu trung tâm để phục vụ máy trạng thái và hiển thị. Quá trình truyền nhận diễn ra liên tục, ổn định, không ghi nhận hiện tượng mất khung trong điều kiện thử nghiệm. Điều này khẳng định trục truyền thông CAN hai chiều, nền tảng của toàn hệ thống VCU – IPC, hoạt động đúng thiết kế.

4.3 KẾT QUẢ HOẠT ĐỘNG VCU VÀ GIAO DIỆN IPC

Bộ điều khiển trung tâm VCU trên Raspberry Pi vận hành một máy trạng thái sáu trạng thái, đồng thời kết xuất toàn bộ giao diện cụm đồng hồ IPC. Tùy theo trạng thái hiện tại của xe, giao diện tự động chuyển đổi nội dung hiển thị, các nhãn cảnh báo và hiệu ứng chuyển cảnh tương ứng. Bố cục chung của màn hình đã được mô tả ở mục 4.1; phần này trình bày kết quả hiển thị thực tế của hệ thống qua từng trạng thái vận hành.

4.3.1 Khởi động và tự kiểm tra hệ thống



Hình 4.5 Giao diện trạng thái OFF

Khi chưa cấp nguồn điều khiển (công tắc Key ở vị trí tắt), hệ thống ở trạng thái OFF với màn hình nền tối hiển thị dòng "EV SYSTEM – POWERED OFF" (Hình 4.5), thể hiện trạng thái an toàn mặc định khi xe chưa được kích hoạt. Khi

người dùng bật công tắc Key, VCU chuyển trạng thái OFF → STANDBY và kích hoạt hiệu ứng khởi động (boot animation) với logo "EVSYS – Electric Vehicle System" (Hình 4.6) trong khoảng 2 giây.



Hình 4.6 Giao diện hiệu ứng khởi động hệ thống

Ngay sau khi khởi động, hệ thống tự động thực hiện chu trình tự kiểm tra: toàn bộ các đèn báo trên thanh trạng thái — đèn Key, đèn pin, đèn cảnh báo và đèn báo lỗi động cơ (MIL) — đồng loạt sáng lên, kết hợp với hiệu ứng kim đồng hồ quét hết thang đo (Hình 4.7). Chu trình này tương tự thao tác kiểm tra đèn báo trên cụm đồng hồ ô tô thực tế khi bật chìa khóa, cho phép người dùng xác nhận tất cả đèn báo và đồng hồ đều hoạt động bình thường trước khi vận hành.



Hình 4.7 Giao diện chu trình tự kiểm tra

4.3.2 Trạng thái STANDBY và READY

Sau chu trình tự kiểm tra, hệ thống vào trạng thái STANDBY (Hình 4.8). Lúc này nhãn "VEHICLE STATE: STANDBY" hiển thị màu lam nhạt, các đèn báo đã tắt, xe đứng yên ở 0,0 km/h với cần số ở vị trí N và phanh ở trạng thái nhả (RELEASED). Dòng nhắc "Press BRAKE + D or R to continue" hướng dẫn điều kiện để chuyển sang trạng thái sẵn sàng. Đây là trạng thái hệ thống đã được cấp nguồn nhưng chưa cho phép truyền động, nhằm bảo đảm an toàn khi xe đang đỗ.



Hình 4.8 Giao diện trạng thái STANDBY

Khi người dùng đạp phanh đồng thời gạt cần số về vị trí D (hoặc R), VCU chuyển trạng thái STANDBY → READY (Hình 4.9). Nhãn trạng thái đổi sang "VEHICLE STATE: READY" màu xanh lá, ô số D được làm nổi bật, và dòng nhắc chuyển thành "Press ACCELERATOR to drive". Ở trạng thái READY, VCU đã cho phép động cơ hoạt động nhưng xe vẫn đứng yên do chưa có tín hiệu ga. Kết quả này xác nhận máy trạng thái của VCU phản ứng đúng theo điều kiện chuyển trạng thái đã thiết kế, đồng thời giao diện cập nhật chính xác và tức thời theo từng thao tác của người lái.



Hình 4.9 Giao diện trạng thái READY

4.3.3 Trạng thái DRIVING và các chế độ lái



Hình 4.10 Giao diện trạng thái DRIVING – chế độ ECO

Khi người dùng đạp ga vượt ngưỡng quy định ở trạng thái READY, VCU chuyển sang trạng thái DRIVING và phát lệnh điều khiển động cơ xuống ECU node. Lúc này thanh ACCELERATOR hiển thị mức ga theo thời gian thực, kim đồng hồ và giá trị tốc độ phản ánh chuyển động thực tế của mô hình. Hệ thống hỗ trợ ba chế độ lái — ECO, NORMAL và SPORT — chuyển đổi bằng nút Mode; mỗi chế độ tương ứng một bộ giới hạn công suất và một dải thang đo tốc độ riêng, đồng thời được phân biệt bằng màu sắc giao diện.



Hình 4.11 Giao diện trạng thái DRIVING – chế độ NORMAL

Cả 3 ghi lại ba chế độ khi xe đang vận hành ở số D với cùng một mức ga khoảng 98%. Ở chế độ ECO (Hình 4.10, màu xanh lá, tốc độ đạt 0,7 km/h); ở chế độ NORMAL (Hình 4.11, màu lam, tốc độ đạt 1,1 km/h); và ở chế độ SPORT (Hình 4.12, màu cam–đỏ, tốc độ đạt 1,4 km/h). Có thể thấy với cùng vị trí ga, tốc độ đạt được tăng dần theo thứ tự ECO < NORMAL < SPORT. Kết quả này minh chứng trực quan cho cơ chế giới hạn công suất theo chế độ lái: chế độ ECO giới hạn độ rộng xung PWM ở mức thấp để tiết kiệm năng lượng, trong khi SPORT cho phép công suất tối đa. Bên cạnh đó, giá trị ODOMETER tăng dần trong suốt quá trình vận hành (từ 184 m khi đứng yên lên 202 m sau khi lái), xác nhận chức năng tích lũy quãng đường hoạt động đúng. Toàn bộ thông số trên giao diện cập nhật mượt mà, đồng bộ với dữ liệu nhận từ mạng CAN, khẳng định sự phối hợp chính xác giữa ECU node, VCU và khối hiển thị IPC.



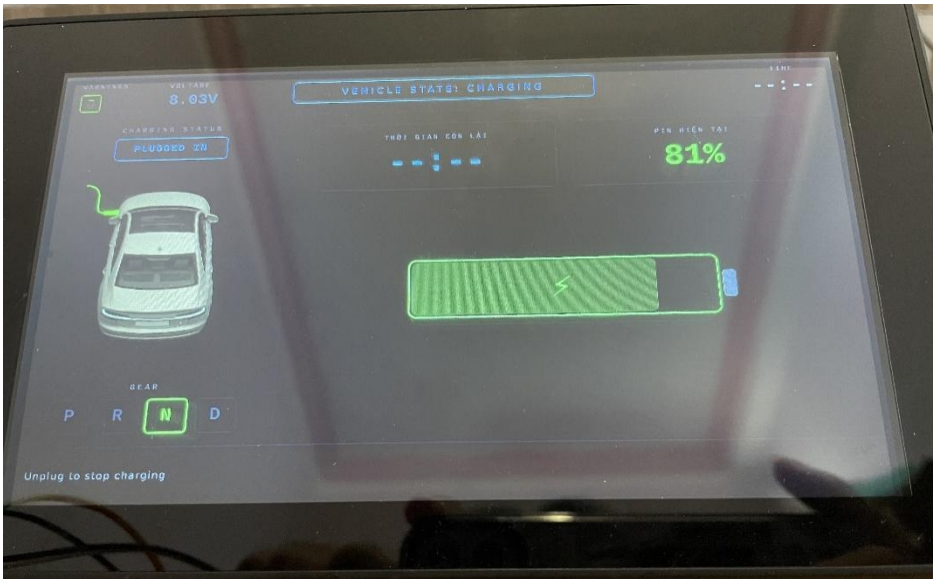
Hình 4.12 Giao diện trạng thái DRIVING – chế độ SPORT

4.3.4 Trạng thái sạc pin

CHARGING là một trạng thái độc lập trong máy trạng thái của VCU, được thiết kế theo nguyên tắc an toàn của xe điện thực tế: hệ thống chỉ cho phép vào sạc khi xe đang đứng yên và ở trạng thái đỗ (OFF hoặc STANDBY). Khi xe đang ở READY hoặc DRIVING, tín hiệu cắm sạc sẽ bị bỏ qua — người dùng buộc phải đưa cần số về P/N hoặc tắt khóa điện trước khi sạc, đúng như cơ chế khóa an toàn trên xe thật. Khi phát hiện tín hiệu cắm sạc (charging detect, lấy qua bộ cách ly quang PC817) trong điều kiện cho phép, VCU chuyển sang trạng thái CHARGING và giao diện tự động chuyển sang màn hình sạc chuyên dụng.

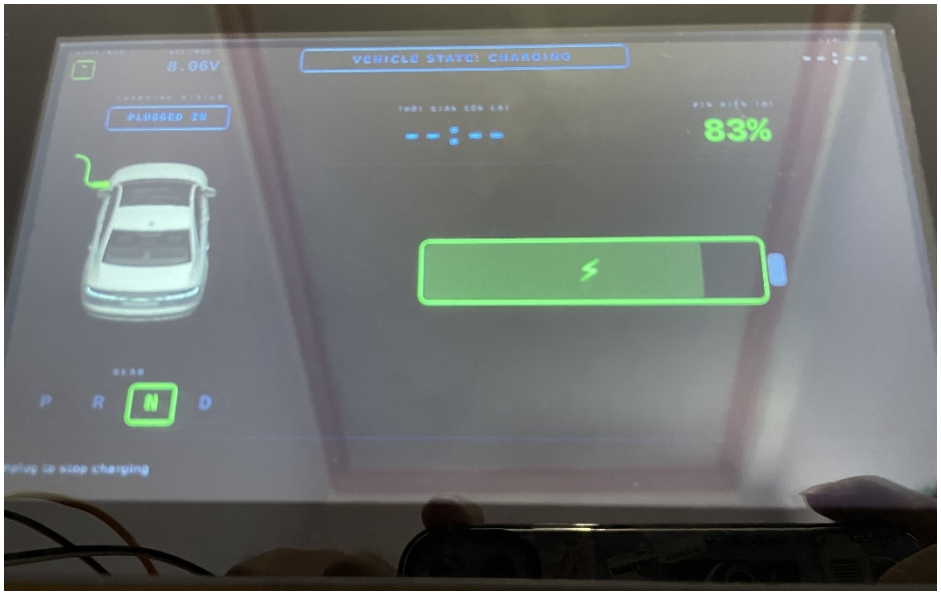
Hình 4.13 thể hiện giao diện ở trạng thái CHARGING. Nhãn "VEHICLE STATE: CHARGING" hiển thị ở trung tâm, trạng thái sạc "CHARGING STATUS: PLUGGED IN" xác nhận cáp sạc đã được kết nối, kèm hình ảnh chiếc xe cắm cáp sạc minh hoạ trực quan. Khu vực trung tâm là biểu tượng pin lớn với hiệu ứng nạp (thanh dung lượng màu xanh lá kèm tia sét) cập nhật theo thời gian thực, bên cạnh là mức pin hiện tại (PIN HIỆN TẠI) và điện áp pack. Dòng nhắc "Unplug to stop charging" hướng dẫn người dùng cách kết thúc sạc. Trong suốt

quá trình sạc, động cơ được cưỡng bức tắt cứng (lệnh VCU đặt motor_enable = 0) nhằm bảo đảm tuyệt đối an toàn — xe không thể di chuyển khi đang cắm sạc.



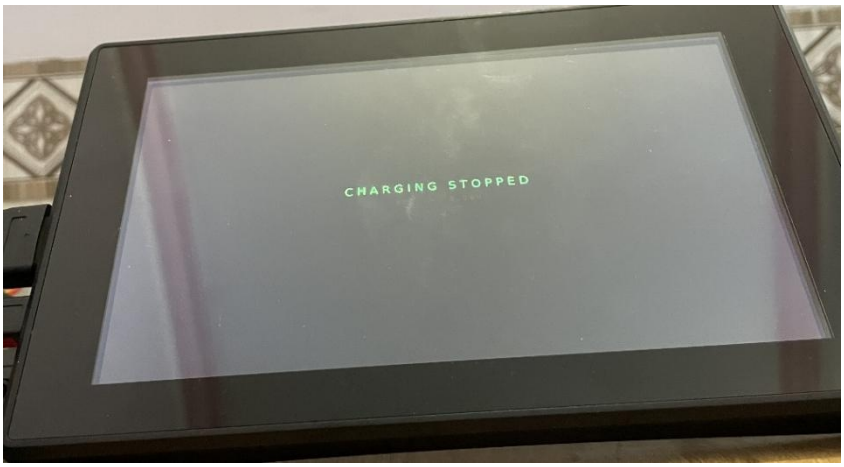
Hình 4.13 Giao diện trạng thái CHARGING

Để kiểm chứng quá trình nạp diễn ra thực sự, mức pin được theo dõi tại hai thời điểm liên tiếp trong cùng một phiên sạc. Hình 4.13 và Hình 4.14 cho thấy dung lượng pin tăng từ 81% (8,03 V) lên 83% (8,06 V): cả phần trăm SoC lẫn điện áp pack đều nhích lên đồng thời, xác nhận khối nguồn đang được nạp và VCU phản ánh đúng diễn biến này lên giao diện theo thời gian thực.



Hình 4.14 Kiểm chứng quá trình nạp pin

Khi người dùng rút cáp sạc, tín hiệu charging detect mất, VCU kết thúc trạng thái CHARGING và kích hoạt hiệu ứng chuyển cảnh "CHARGING STOPPED" (Hình 4.15) trước khi đưa hệ thống trở về trạng thái STANDBY hoặc OFF tương ứng. Hiệu ứng này giúp người dùng nhận biết rõ ràng thời điểm kết thúc phiên sạc. Kết quả cho thấy toàn bộ chu trình sạc — từ lúc cắm cáp, nạp pin, đến lúc rút cáp — được máy trạng thái quản lý chặt chẽ và phản ánh chính xác, đầy đủ trên giao diện cụm đồng hồ.



Hình 4.15 Hiệu ứng kết thúc sạc

4.3.5 Kết quả phát hiện lỗi và chẩn đoán

Hệ thống xử lý lỗi được tổ chức theo kiến trúc hai tầng độc lập, mô phỏng đúng cách cụm đồng hồ ô tô thật phân loại sự cố: tầng bảo vệ xử lý các lỗi an toàn cần ngắt tức thời (cảnh báo đỏ), và tầng chẩn đoán ghi nhận, theo dõi các mã lỗi DTC phục vụ bảo dưỡng (đèn báo vàng). Nguyên tắc ưu tiên hiển thị tuân theo chuẩn ngành: đỏ (nguy hiểm) đè lên vàng (cảnh báo) đè lên xanh (thông tin).

Tầng bảo vệ do máy trạng thái xử lý ở chu kỳ 20 ms, giám sát liên tục bốn điều kiện: thấp áp (điện áp $\leqslant$ 6,50 V), quá áp (> 8,70 V), kẹt động cơ (đang DRIVING, PWM $\geqslant$ 30% nhưng tốc độ $\approx$ 0), và mất truyền thông CAN (không nhận khung từ STM32 quá 2 giây). Mỗi điều kiện đều qua bộ lọc thời gian (debounce 1,5–2 giây) để chống cảnh báo giả do nhiễu thoáng qua. Khi phát hiện bất kỳ lỗi nào, VCU lập tức chuyển sang trạng thái FAULT và cưỡng bức tắt cứng động cơ (lệnh điều khiển đặt motor_enable = 0), đồng thời hiển thị lớp phủ cảnh báo đỏ toàn màn hình.



Hình 4.16 Màn hình FAULT khi khi động cơ bị kẹt



Hình 4.17 Màn hình FAULT mất truyền thông CAN

Hình 4.16 và Hình 4.17 minh hoạ kết quả khi kích hai lỗi tiêu biểu. Khi rút dây CAN, sau 2 giây hệ thống báo lỗi "CAN TIMEOUT" (Hình 4.15); khi giữ kẹt trục động cơ trong lúc đang tăng ga, hệ thống báo "MOTOR STALL" (Hình 4.16). Cả hai cùng dùng một lớp phủ đỏ "FAULT DETECTED" với biểu tượng cảnh báo, tên lỗi cụ thể, hướng dẫn "Turn KEY OFF then ON to reset" và nút CLEAR DTC. Hai lỗi thấp áp/quá áp dùng chung cơ chế này (không trình diễn được trên mô hình do không thể chủ động đưa điện áp pack vượt ngưỡng an toàn). Lỗi tầng bảo vệ được chốt (latch): hệ thống không tự thoát ngay cả khi điều kiện lỗi biến mất (ví dụ cắm lại dây CAN), buộc người vận hành phải gạt

khóa điện OFF → ON để xác nhận — đúng hành vi của xe thật nhằm tránh xe tự chạy lại sau một sự cố.

Một kết quả kỹ thuật đáng chú ý là khả năng phân biệt cảm biến hỏng với pin yếu thật. Vì điện áp pin được đo qua mạch chia áp vào chân ADC của STM32, khi dây tín hiệu bị đứt, chân ADC trôi tự do và cho giá trị phi lý — nếu xử lý ngây thơ sẽ bị hiểu nhầm thành "pin cạn" và kích lỗi thấp áp giả. Để giải quyết, chính STM32 (nơi có dữ liệu ADC gốc) tự đánh giá độ tin cậy của cảm biến: nếu điện áp nằm ngoài dải vật lý khả dĩ của cụm pin 2S (5,8–8,9 V), cảm biến được coi là hỏng và một bit cờ được gửi kèm khung CAN. Khi nhận cờ này, VCU khóa việc đánh giá thấp/quá áp và hiển thị "-- V" thay cho giá trị điện áp (Hình 4.18), tuyệt đối không hiển thị "0% pin cạn" giả. Nhờ vậy, sự cố cảm biến được tách bạch rõ ràng và chuyển cho tầng chẩn đoán bắt thành mã lỗi P0463 (đèn vàng, không dừng xe), thay vì gây ra một cảnh báo đỏ sai.



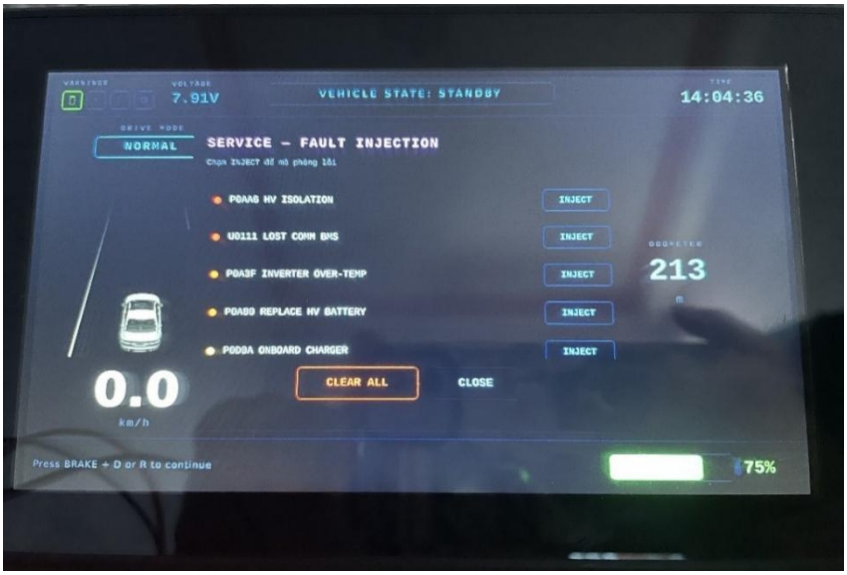
Hình 4.18 Hiển thị khi cảm biến điện áp bị ngắt

Ở chiều ngược lại, khi pin thực sự yếu (điện áp hợp lệ nhưng dung lượng xuống thấp), hệ thống bật đèn cảnh báo pin màu vàng trên thanh trạng thái để nhắc người lái (Hình 4.19), phân biệt rõ với trường hợp cảm biến hỏng ở trên.



Hình 4.19 Giao diện cảnh báo pin yếu

Tầng chẩn đoán do bộ quản lý DTC xử lý trên một luồng riêng (chu kỳ 200 ms), đóng vai một cổng chẩn đoán quản lý danh mục bảy mã lỗi đặt theo phong cách OBD-II của xe điện. Mỗi mã đi qua vòng đời hai chu trình (two-trip): xuất hiện lần đầu chuyển sang PENDING (chưa báo người lái), tái xuất ở chu trình sau mới được CONFIRMED và bật đèn MIL — triết lý này tránh làm phiền người lái bằng cảnh báo của những lỗi thoáng qua. Để trình diễn các lỗi không có phần cứng để tạo ra (rò cách điện cao áp, mất liên lạc BMS, quá nhiệt bộ nghịch lưu…), hệ thống cung cấp một menu service ẩn ngay trên cảm ứng (Hình 4.20): chạm INJECT tại mỗi mã để mô phỏng lỗi tương ứng, đúng tinh thần một máy chẩn đoán gửi lệnh vào VCU.



Hình 4.20 Menu service mô phỏng lỗi (Fault Injection)



Hình 4.21 Màn hình Lỗi CRITICAL (P0AA6)

Điểm cốt lõi của tầng chẩn đoán là phản ứng phân cấp theo mức độ nghiêm trọng của lỗi, được kiểm chứng qua ba trường hợp tiêu biểu. Với lỗi mức CRITICAL như P0AA6 (rò cách điện cao áp), hệ thống lập tức đẩy xe vào trạng thái FAULT đỏ và cắt động cơ (Hình 4.21) — tương đương lỗi tầng bảo vệ. Với lỗi mức HIGH như P0A3F (quá nhiệt bộ nghịch lưu), xe không dừng hẳn mà chuyển sang chế độ "limp-home", giới hạn công suất động cơ còn 30% để người lái còn về được xưởng: Hình 4.23 cho thấy dù chân ga ở 96% và đang ở chế độ NORMAL, tốc độ chỉ đạt 0,4 km/h (so với ~1,1 km/h khi không lỗi), kèm đèn MIL vàng và mã "DTC: P0A3F" ở dải dưới. Với lỗi mức MEDIUM như P0560 (điện áp nguồn phụ), hệ thống chỉ bật đèn MIL cảnh báo mà không hạn chế vận hành: Hình 4.22 cho thấy xe vẫn đạt tốc độ bình thường 1,1 km/h ở cùng điều kiện, chỉ khác là có đèn MIL và mã "DTC: P0560". Sự đối chứng giữa Hình 4.22 và Hình 4.23 minh chứng rõ ràng cơ chế phân cấp: cùng một chế độ lái và mức ga, lỗi HIGH ghìm công suất còn lỗi MEDIUM thì không.

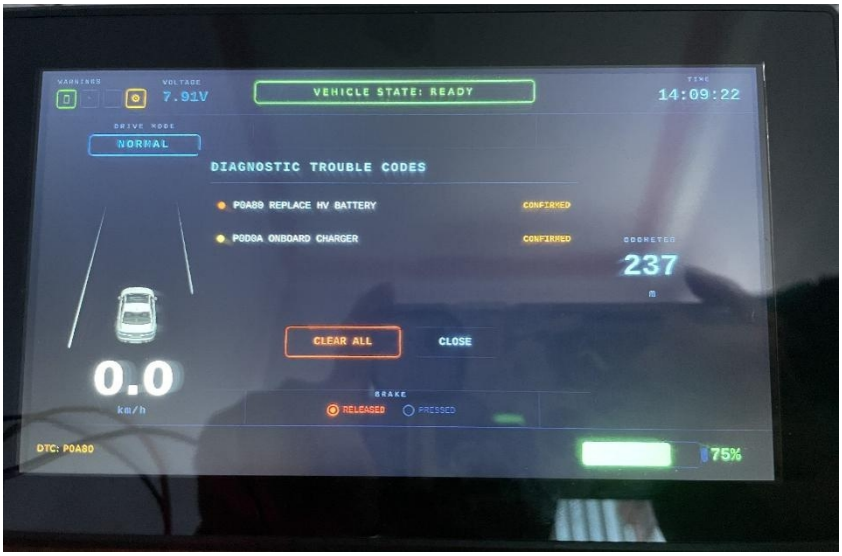


Hình 4.22 Màn hình bật đèn MIL khi lỗi MEDIUM (P0560)

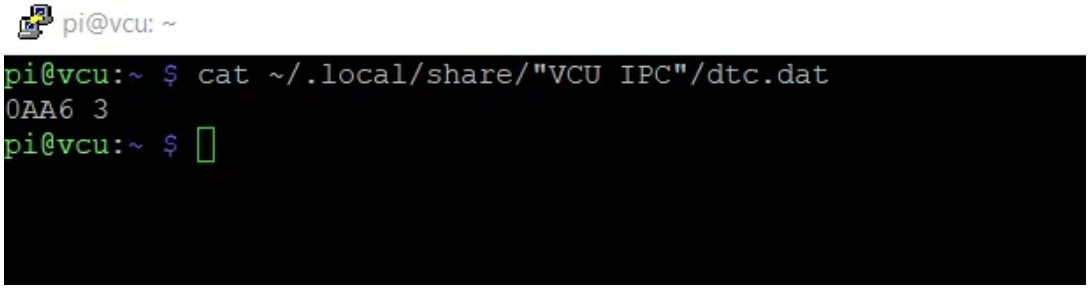


Hình 4.23 Màn hình chế độ limp-home khi lỗi HIGH (P0A3F)

Người dùng có thể xem chi tiết các mã lỗi đang hoạt động bằng cách chạm vào đèn MIL để mở bảng "DIAGNOSTIC TROUBLE CODES" (Hình 4.24), liệt kê từng mã kèm trạng thái (CONFIRMED) và nút xóa. Toàn bộ DTC được lưu bền vững ra file dtc.dat bằng kỹ thuật ghi nguyên tử (ghi ra file tạm rồi đổi tên), bảo đảm dữ liệu không hỏng dở dang ngay cả khi mất điện đột ngột. Hình 4.24 cho thấy nội dung file sau khi inject lỗi P0AA6: dòng 0AA6 3 thể hiện mã P0AA6 ở trạng thái PERMANENT (mức 3). Nhờ persistence, một lỗi nghiêm trọng sẽ tự đưa xe vào FAULT lại mỗi lần khởi động cho đến khi được xóa bằng lệnh chẩn đoán — đúng đặc tính "permanent DTC" của OBD-II thật.



Hình 4.24 Màn hình bảng chi tiết các mã lỗi DTC



Hình 4.25 Nội dung file lưu trữ DTC

Bên cạnh các lỗi cần can thiệp thủ công, hệ thống còn hỗ trợ cơ chế tự lành (self-heal) cho những lỗi mang tính tạm thời, tiêu biểu là mã lỗi cảm biến điện áp P0463. Khi nguyên nhân gây lỗi được khắc phục — cụ thể là cảm biến được cắm lại và cho giá trị hợp lệ liên tục trong khoảng 3 giây — bộ quản lý DTC tự động đưa mã lỗi này về trạng thái CLEARED và tắt đèn MIL ngay trong phiên làm việc, không cần khởi động lại hệ thống. Kết quả thử nghiệm cho thấy: sau khi ngắt dây tín hiệu cảm biến để kích lỗi (màn hình hiển thị "-- V", đèn MIL sáng), việc cắm lại dây khiến điện áp hiển thị trở về bình thường và đèn MIL tự tắt sau vài giây. Cần lưu ý rằng cơ chế tự lành chỉ áp dụng cho các lỗi two-trip mang tính phục hồi được; các lỗi nghiêm trọng ở trạng thái PERMANENT (như P0AA6) không tự lành mà bắt buộc phải xóa bằng lệnh chẩn đoán, phản ánh đúng nguyên tắc phân tầng an toàn của hệ thống.

Cuối cùng, hệ thống được kiểm chứng về logic ưu tiên hiển thị khi nhiều lỗi xuất hiện đồng thời. Theo nguyên tắc HMI an toàn của ngành ô tô, cảnh báo có

mức độ nghiêm trọng cao hơn luôn được hiển thị đè lên cảnh báo thấp hơn, theo thứ tự đỏ (nguy hiểm) > vàng (cảnh báo) > xanh (thông tin). Để kiểm chứng, trong tình huống hệ thống đang có một mã lỗi DTC mức cảnh báo (đèn MIL vàng đang sáng), người vận hành tiếp tục kích một lỗi tầng bảo vệ bằng cách rút dây CAN. Kết quả quan sát được: lớp phủ cảnh báo đỏ "FAULT DETECTED" lập tức chiếm toàn bộ màn hình, đè lên đèn MIL vàng đang hiển thị. Điều này khẳng định hệ thống xử lý đúng thứ tự ưu tiên, bảo đảm thông tin nguy hiểm nhất luôn được người lái tiếp nhận trước tiên — một yêu cầu quan trọng trong thiết kế giao diện an toàn.

Kết quả cho thấy hệ thống phát hiện và xử lý lỗi hoạt động đầy đủ, chính xác theo đúng thiết kế: phân loại được lỗi theo mức độ nghiêm trọng, phản ứng phù hợp với từng mức (cắt động cơ / giảm công suất / chỉ cảnh báo), phân biệt được cảm biến hỏng với pin yếu, lưu trữ bền vững và phục hồi đúng cách. Đây là phần thể hiện rõ nhất tính hoàn thiện và tư duy an toàn theo chuẩn ngành ô tô của hệ thống.

4.4 ĐÁNH GIÁ MỨC ĐỘ TIN CẬY

Sau khi hoàn thiện và đưa vào vận hành thử nghiệm, hệ thống VCU–IPC đã được đánh giá toàn diện về độ tin cậy thông qua quá trình chạy liên tục và kiểm thử theo các kịch bản vận hành đã trình bày ở các mục trên. Nhìn chung, hệ thống hoạt động ổn định, phản ánh chính xác trạng thái vận hành và xử lý đúng các tình huống lỗi theo đúng thiết kế. Mục này đánh giá độ tin cậy của hệ thống trên ba khía cạnh chính — truyền thông, điều khiển và an toàn, đồng thời đối chiếu mức độ đáp ứng so với các mục tiêu đã đặt ra.

4.4.1 Độ tin cậy truyền thông và đồng bộ dữ liệu

Mạng CAN Bus — trục xương sống của hệ thống, vận hành ổn định trong suốt quá trình thử nghiệm. Bốn nhóm khung dữ liệu từ ECU node (tốc độ, pin, đầu vào, quãng đường) và khung lệnh từ VCU được truyền nhận liên tục, đúng chu kỳ thiết kế, không ghi nhận hiện tượng mất khung hay sai lệch dữ liệu trong điều kiện thử nghiệm. Dữ liệu giải mã ở phía Raspberry Pi luôn nhất quán với

thao tác thực tế của người lái và với lệnh điều khiển trả về, xác nhận tính toàn vẹn của kênh truyền hai chiều. Giao diện cụm đồng hồ cập nhật mượt mà, gần như tức thời theo dữ liệu nhận được: kim đồng hồ, thanh ga, mức pin và các chỉ báo đều thay đổi đồng bộ với diễn biến vận hành, cho thấy độ trễ end-to-end đủ thấp để bảo đảm trải nghiệm thời gian thực.

4.4.2 Độ tin cậy của logic điều khiển và máy trạng thái

Máy trạng thái sáu trạng thái trên VCU chuyển trạng thái chính xác theo đúng các điều kiện đã thiết kế qua toàn bộ chu trình vận hành: từ khởi động (OFF → STANDBY), sẵn sàng (READY), vận hành (DRIVING), sạc pin (CHARGING) cho đến xử lý sự cố (FAULT). Trong tất cả các lần thử nghiệm, không ghi nhận trường hợp chuyển trạng thái sai hay kẹt trạng thái. Đặc biệt, kiến trúc phân tách "đo lường – ra quyết định – chấp hành" (STM32 thu thập và chấp hành, Raspberry Pi ra quyết định) hoạt động đúng như nguyên lý thiết kế: STM32 không tự ý điều khiển động cơ mà chỉ thực thi lệnh từ VCU, giúp tập trung toàn bộ logic an toàn về một điểm và dễ kiểm soát. Các chế độ lái ECO/NORMAL/SPORT cho kết quả nhất quán, với công suất động cơ bị giới hạn đúng theo từng chế độ.

4.4.3 Độ tin cậy của các cơ chế an toàn và chẩn đoán

Đây là khía cạnh thể hiện rõ nhất tính tin cậy của hệ thống. Các cơ chế an toàn nhiều lớp đều được kiểm chứng hoạt động đúng: bộ giám sát watchdog hai phía bảo đảm hệ thống vào trạng thái an toàn khi mất truyền thông (STM32 tự dừng động cơ nếu mất lệnh VCU, VCU vào FAULT nếu mất khung CAN); động cơ được tắt cứng tuyệt đối ở các trạng thái OFF, FAULT và CHARGING; cơ chế chốt lỗi buộc người vận hành xác nhận sự cố trước khi cho phép xe hoạt động lại; và bộ lọc thời gian loại bỏ hiệu quả các cảnh báo giả do nhiễu thoáng qua. Hệ thống chẩn đoán DTC theo chuẩn OBD-II cũng vận hành đúng: phân loại lỗi theo mức độ nghiêm trọng với phản ứng tương ứng (cắt động cơ / giảm công suất / chỉ cảnh báo), phân biệt được cảm biến hỏng với pin yếu thật, lưu trữ bền vững nhờ

kỹ thuật ghi nguyên tử và phục hồi đúng sau khi mất điện. Kết quả khẳng định hệ thống có tư duy an toàn đúng theo tinh thần của ngành công nghiệp ô tô.

4.4.4 Đối chiếu với mục tiêu đề ra

Đối chiếu với các mục tiêu đặt ra, hệ thống đã đáp ứng đầy đủ các yêu cầu, được tổng hợp trong Bảng 4.2.

Bảng 4.2 Đối chiếu kết quả với mục tiêu đề ra

Mục tiêu	Kết quả đạt được	Minh chứng
Xây dựng ECU node dùng FreeRTOS: đọc tín hiệu người lái, điều khiển động cơ, đo lường thông số, truyền CAN	Đọc đúng các đầu vào số/analog, điều khiển PWM theo lệnh, đo tốc độ/điện áp/SoC, phát đủ bốn nhóm khung CAN	Mục 4.1, 4.2
Xây dựng bộ điều khiển trung tâm VCU nhận dữ liệu từ CAN, điều hành hoạt động của xe theo máy trạng thái và xử lý logic điều khiển, an toàn.	Máy trạng thái 6 trạng thái, điều khiển hai chiều, an toàn nhiều lớp, chẩn đoán DTC	Mục 4.3
Thiết kế cụm đồng hồ IPC bằng Qt/QML hiển thị thời gian thực	Giao diện cụm đồng hồ đầy đủ chức năng, cập nhật mượt theo dữ liệu CAN	Mục 4.1, 4.3
Tích hợp và kiểm thử hệ thống theo các kịch bản vận hành	Vận hành thông suốt toàn chu trình và kiểm thử đầy đủ các tình huống lỗi	Mục 4.3

4.5 ĐÁNH GIÁ ĐỊNH LƯỢNG VỀ ĐÁP ỨNG THỜI GIAN THỰC

Bên cạnh đánh giá định tính ở mục 4.4, hệ thống được đánh giá thêm về mặt định lượng dựa trên chu kỳ các tác vụ đã cấu hình trong thiết kế, nhằm ước lượng cận trên của độ trễ điều khiển và tải truyền thông.

Bảng 4.3 Chu kỳ hoạt động của các tác vụ/luồng chính trên cả hai nền tảng

Tác vụ/ Luồng	Nền tảng	Chu kỳ
Task_MotorControl	STM32	5 ms
Task_InputScan	STM32	10 ms
Task_CANTransmit (0x100)	STM32	20 ms
Task_BatteryManager	STM32	100 ms
StateMachine (FSM)	Raspberry Pi	20 ms
CanSender (0x200)	Raspberry Pi	20 ms
DtcManager	Raspberry Pi	200 ms

Độ trễ đầu - cuối (từ lúc đọc chân ga đến lúc PWM động cơ được cập nhật) được ước lượng bằng tổng cận trên của từng khâu trong chuỗi xử lý: đọc cảm biến (10 ms) → gửi CAN lên VCU (20 ms) → truyền bus (<1 ms) → xử lý máy trạng thái (20 ms) → gửi lệnh CAN xuống ECU (20 ms) → truyền bus (<1 ms) → áp PWM (5 ms), cho cận trên lý thuyết xấp xỉ 77 ms, đạt yêu cầu đáp ứng thời gian thực dưới 100 ms đề ra ở Chương 1.

Về tải truyền thông, với tổng cộng 160 khung/giây (0x100: 50 Hz, 0x101: 10 Hz, 0x102/0x103: 25 Hz mỗi khung, 0x200: 50 Hz) trên bus 500 kbps, thời gian bus bị chiếm dụng ước lượng chỉ khoảng 2.8%, cho thấy hệ thống còn nhiều dư địa để mở rộng thêm ECU hoặc thông điệp trong tương lai.

Về thời gian phản ứng lỗi, các ngưỡng debounce/watchdog được thiết kế tường minh: mất CAN 2 s, xác nhận lỗi cảm biến ~2 s, tự lành ~3 s, watchdog lệnh VCU 2 s — cân bằng giữa phản ứng nhanh và chống cảnh báo giả.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Từ những yêu cầu đã đề ra trong đề tài "Thiết kế và triển khai hệ thống cụm đồng hồ kỹ thuật số (IPC) và bộ điều khiển trung tâm (VCU) cho mô hình xe ô tô điện", nhóm đã tiến hành nghiên cứu chuẩn truyền thông CAN, hệ điều hành thời gian thực FreeRTOS và nền tảng đồ họa Qt/QML để xây dựng một hệ thống điều khiển – hiển thị hoàn chỉnh cho mô hình xe ô tô điện.

Khác với hướng tiếp cận giám sát thụ động thường gặp (chỉ thu thập và hiển thị dữ liệu), hệ thống của đề tài được xây dựng như một vòng điều khiển khép kín hai chiều thực thụ, theo kiến trúc phân tách "đo lường – ra quyết định – chấp hành": vi điều khiển STM32 đóng vai trò ECU node thu thập tín hiệu và chấp hành lệnh, trong khi Raspberry Pi đóng vai trò bộ điều khiển trung tâm (VCU) chạy máy trạng thái và toàn bộ logic điều khiển, an toàn, chẩn đoán; đồng thời kết xuất giao diện cụm đồng hồ kỹ thuật số (IPC). Hai khối trao đổi dữ liệu qua mạng CAN Bus tốc độ 500 kbit/s.

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định, đáp ứng đầy đủ các yêu cầu đặt ra về thu thập, truyền nhận dữ liệu, điều khiển động cơ theo trạng thái, hiển thị thời gian thực, và đặc biệt là các cơ chế an toàn, chẩn đoán lỗi theo tư duy của ngành công nghiệp ô tô. Đề tài đã đạt được mục tiêu xây dựng một mô hình hệ thống VCU–IPC có chiều sâu kỹ thuật và khả năng mở rộng trong tương lai.

5.1.1 Kết quả đạt được

Nhóm đã hoàn thiện được mô hình hệ thống theo đúng các yêu cầu đề ra, thực hiện được các chức năng chính sau:

Xây dựng ECU node trên STM32 sử dụng FreeRTOS: đọc các tín hiệu người lái (nút nhấn, công tắc, biến trở ga), đo tốc độ từ encoder, đo điện áp pin và ước

lượng dung lượng SoC, điều khiển động cơ bằng xung PWM và truyền dữ liệu lên mạng CAN.

Xây dựng bộ điều khiển trung tâm VCU trên Raspberry Pi với máy trạng thái sáu trạng thái (OFF, STANDBY, READY, DRIVING, CHARGING, FAULT), điều khiển động cơ hai chiều và tập trung toàn bộ logic an toàn về một điểm.

Triển khai ba chế độ lái ECO, NORMAL và SPORT với mức giới hạn công suất khác nhau.

Thiết lập truyền thông CAN Bus hai chiều ổn định ở tốc độ 500 kbit/s giữa ECU node và VCU.

Thiết kế cụm đồng hồ kỹ thuật số IPC bằng Qt/QML hiển thị thời gian thực: đồng hồ tốc độ, vị trí ga, cần số, chế độ lái, dung lượng pin, quãng đường, cùng các màn hình trạng thái và hiệu ứng chuyển cảnh.

Triển khai các cơ chế an toàn nhiều lớp: bộ giám sát watchdog hai phía, tắt cứng động cơ ở các trạng thái không cho phép vận hành, cơ chế chốt lỗi và bộ lọc thời gian chống cảnh báo giả.

Xây dựng hệ thống chẩn đoán DTC theo chuẩn OBD-II: danh mục bảy mã lỗi, vòng đời hai chu trình, phản ứng phân cấp theo mức độ nghiêm trọng (cắt động cơ / giảm công suất / chỉ cảnh báo), phân biệt được cảm biến hỏng với pin yếu thật, lưu trữ bền vững và cơ chế tự lành.

Triển khai chức năng sạc pin với cơ chế khóa an toàn (chỉ cho sạc khi xe đứng yên và đang đỗ).

So với các đề tài trước đây, điểm khác biệt cốt lõi mà đề tài này bổ sung là: thay vì chỉ đọc và hiển thị dữ liệu, hệ thống này thực hiện vòng lặp điều khiển khép kín – STM32 đọc tín hiệu người lái, điều khiển motor thực tế và phản hồi encoder; Raspberry Pi xử lý state machine toàn xe và đưa ra quyết định điều khiển; IPC phản ánh trạng thái hệ thống theo thời gian thực. Đây là bước tiến từ "quan sát" sang "hiểu và điều khiển" một hệ thống xe điện.

5.1.2 Hạn chế

Bên cạnh những kết quả đạt được, hệ thống vẫn còn tồn tại một số hạn chế do đang trong giai đoạn nghiên cứu và phát triển trên mô hình:

Do là mô hình tỉ lệ thu nhỏ với động cơ công suất nhỏ nên dải tốc độ vật lý còn rất nhỏ, chưa phản ánh được dải tốc độ vận hành của xe thật.

Điệp áp pin đo được vẫn còn nhiễu khi động cơ hoạt động và khi cắm sạc.

Hệ thống chưa có cảm biến dòng điện nên dung lượng pin SoC được ước lượng chủ yếu dựa trên điện áp, chưa bù theo dòng tải tức thời.

Raspberry Pi đảm nhiệm đồng thời cả vai trò VCU và IPC trên cùng một thiết bị, chưa tách biệt về phần cứng giữa khối xử lý an toàn và khối hiển thị.

5.2 HƯỚNG PHÁT TRIỂN

Để nâng cao tính ứng dụng và hoàn thiện hệ thống trong tương lai, nhóm đề xuất một số hướng phát triển như sau:

Bổ sung các cảm biến thật (cảm biến dòng điện, nhiệt độ, giám sát cách điện) để phát hiện lỗi bằng đo lường trực tiếp thay vì mô phỏng bằng inject.

Mở rộng danh mục mã lỗi DTC và tích hợp giao thức chẩn đoán chuẩn (UDS/OBD-II thực thụ qua CAN) để có thể giao tiếp với các máy chẩn đoán thương mại.

Mở rộng hệ thống lên nhiều node ECU hoặc nâng cấp lên chuẩn CAN-FD nhằm mô phỏng kiến trúc mạng phân tán nhiều ECU như trên ô tô hiện đại.

Bổ sung tính năng kết nối từ xa (telemetry qua cloud, ứng dụng điện thoại) để giám sát và quản lý xe từ xa.

TÀI LIỆU THAM KHẢO

[1] IEA (International Energy Agency), "Global EV Outlook 2024: Moving towards increased affordability," IEA, Paris, 2024. [Online]. Available: https://www.iea.org/reports/global-ev-outlook-2024 [Accessed: May 2026]

[2] Embitel Technologies, "Integrated Vehicle Control Unit (VCU) in Contemporary Electric Vehicles," Embitel Blog, Nov. 2024. [Online]. Available: https://www.embitel.com/blog/embedded-blog/integrated-vehicle-control-unit-vcu-in-contemporary-electric-vehicles [Accessed: May 2026]

[3] EmbedIC, "What is Instrument Panel Clusters and Common Issues," EmbedIC Technology Blog. [Online]. Available: https://www.embedic.com/technology/details/what-is-instrument-panel-clusters-and-common-issues [Accessed: May 2026]

[4] SEA:ME Program, "DES Instrument Cluster – PiRacer Instrument Cluster Qt Application on RPi via CAN Bus," GitHub Repository, 2023. [Online] Available: https://github.com/SEA-ME/DES_Instrument-Cluster [Accessed: Mar 2026]

[5] International Journal of Electrical Engineering and Technology Research (IJEETR), "Implementation of Automotive Embedded System using STM32F103 and FreeRTOS," IJEETR, 2022.

[6] Bamboo Apps, "CAN Bus Emulation with FreeRTOS and Qt," Bamboo Apps Blog, Mar. 2024. [Online] Available: https://bambooapps.eu/blog/can-bus-testing-alternative [Accessed: Mar 2026]

[7] Developex, "Qt for Automotive Development Solutions," Developex Tech Blog. [Online] Available: https://developex.com/blog/qt-for-automotive/ [Accessed: Apr 2026]

[8] CAN in Automation (CiA), "History of CAN Technology," CiA Official Website. [Online]. Available: https://www.can-cia.org/can-knowledge/history-of-can-technology [Accessed: Apr 2026]

[9] CSS Electronics, "CAN Bus Explained – A Simple Intro [2025]," CSS Electronics. [Online]. Available: https://www.csselectronics.com/pages/can-bus-simple-intro-tutorial [Accessed: Apr 2026]

[10] Dewesoft, "What Is CAN Bus (Controller Area Network)," Dewesoft Blog. [Online]. Available: https://dewesoft.com/blog/what-is-can-bus [Accessed: Apr 2026]

[11] Altium Resources, "CAN-Bus: Introduction and History," Altium Designer Blog. [Online]. Available: https://resources.altium.com/p/Controller-Area-Network-Bus-Introduction-and-History [Accessed: Apr 2026]

[12] NXP Semiconductors, "I2C-bus specification and user manual (UM10204)".

[13] Texas Instruments, "Understanding the I2C Bus (Application Report SLVA704),".

[14] SparkFun Electronics, "Serial Peripheral Interface (SPI)," SparkFun Tutorials. [Online]. Available: https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi [Accessed: May 2026]

[15] Circuit Basics, "Basics of the SPI Communication Protocol." [Online]. Available: https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/ [Accessed: May 2026]

[16] SparkFun Electronics, "Serial Communication," SparkFun Tutorials. [Online]. Available: https://learn.sparkfun.com/tutorials/serial-communication [Accessed: May 2026]

[17] Circuit Basics, "Basics of UART Communication." [Online]. Available: https://www.circuitbasics.com/basics-uart-communication/ [Accessed: May 2026]